GRADUATION PROJECT · SOFTWARE IMPLEMENTATION HANDBOOK

# SafeVision AI

نظام ذكي لإدارة السلامة الصناعية بالاعتماد على الرؤية الحاسوبية

## الدليل العربي الشامل للتنفيذ

**Backend · Frontend · Database · AI Integration · Testing · Deployment**

المعمارية الكاملة · الـ API · قاعدة البيانات · خط معالجة الفيديو · الواجهة · النشر · خطة الفريق

`OpenCV` `TypeScript` `React` `SQLAlchemy` `PostgreSQL` `FastAPI` `Python` `Alembic` `Docker` `PyTorch`

### ما يغطّيه هذا الدليل

- المعمارية الكاملة للنظام والـ AI Pipeline
- قاعدة البيانات و PostgreSQL و Alembic
- واجهة React وربطها بالـ API
- الـ Backend بـ FastAPI والـ API Contract
- دمج الـ Models وخط معالجة الفيديو
- الاختبارات و Docker والنشر وخطة الفريق


| | |
|---|---|
| التوثيق | وثائق مشروع التخرّج |
| إعداد | فريق مشروع التخرّج — Graduation Project Team |
| القسم | قسم علوم الحاسب / الذكاء الاصطناعي |
| الجامعة | <UNIVERSITY_NAME> |
| إشراف | <SUPERVISOR_NAME> |
| العام الدراسي | 2027 – 2026 |

# فهرس المحتويات

أرقام الصفحات تشير إلى ترقيم صفحات المحتوى. يمكنك أيضاً استخدام لوحة الـ Bookmarks في قارئ الـ PDF للانتقال السريع بين الفصول.


00 مقدمة: كيف تستخدم هذا الدليل 2

01 نظرة عامة على المشروع ونطاقه 5

02 البنية الكاملة للنظام 9

03 حزمة التقنيات واختيار الأدوات 14

04 هيكل المستودع وتنظيم المجلدات 17

05 تجهيز بيئة التطوير 22

06 معمارية الـ Backend بـ FastAPI 26

07 تصميم الـ API وتوثيق الـ Endpoints 34

08 تصميم قاعدة البيانات مع PostgreSQL 39

09 رفع الفيديو وتخزين الملفات 47

10 دمج الـ AI Models المتفق عليها 51

11 خط معالجة الفيديو والـ Background Jobs 56

12 قواعد السلامة وتوليد الوقائع 61

13 دورة حياة الواقعة والمراجعة البشرية 65

14 معمارية الـ Frontend بـ React و TypeScript 69

15 بناء لوحة التحكم وربطها بالـ API 74

16 التحديثات الفورية وحالة المعالجة 80

17 التحليلات وتقارير الالتزام والتصدير 83

18 الأمان والخصوصية والمصادقة 87

19 Docker والنشر المحلي 91

20 النشر في بيئة الإنتاج 98

21 الـ CI/CD وسير عمل GitHub 101

22 الاختبارات وضمان الجودة 105

23 تقييم الـ AI وأداء النظام 109

24 معالجة الأخطاء والـ Logging والمراقبة 113

25 توزيع العمل على خمسة أعضاء 116

26 الجدول الزمني والمراحل 119

27 تعريف الـ MVP ومعايير القبول 122

28 العرض النهائي لمشروع التخرّج 125

29 قائمة التحقق النهائية واستكشاف الأخطاء 128

30 المراجع والمواد المرجعية 130

CH00 · HOW TO USE THIS HANDBOOK

# 00 مقدمة: كيف تستخدم هذا الدليل

اتفاقيات التوثيق، ودلالات الصناديق والوسوم، وكيف تحوّل كل فصل إلى مهام قابلة للتنفيذ

هذا الدليل ليس عرضاً تقديمياً للمشروع، بل **دليل تنفيذ**. كل فصل يشرح **ماذا** نبني، و**لماذا** نبني بهذه الطريقة، و**كيف** ننفّذه خطوة بخطوة، وينتهي بقائمة تحقق تجعل تقدّم الفريق قابلاً للقياس.

## 0.1 لمن هذا الدليل وكيف تقرأه

الدليل موجَّه لفريق من خمسة طلاب يعرفون أساسيات `Python` وتعلّم الآلة، لكنهم لم يبنوا قبل ذلك نظاماً كاملاً بـ `Backend` و`Frontend` وقاعدة بيانات وخط معالجة فيديو. لذلك نفترض أن القارئ **لا** يعرف مسبقاً كيف يعمل الـ Terminal، ولا معنى الـ Port، ولا كيف تُدار الـ Environment Variables — ونشرح كل ذلك عند أول استخدام.

**ترتيب القراءة المقترح**

اقرأ الفصول 1 و2 و3 و4 و7 **معاً قبل كتابة أي كود**، لأنها تحدّد النطاق والمعمارية وعقد الـ API وهيكل المستودع. بعدها يمكن لكل عضو أن ينتقل مباشرة إلى فصول دوره، مع الرجوع إلى الفصل 7 كلما احتاج التأكد من شكل الطلب أو الرد.

## 0.2 بنية كل فصل

كل فصل يبدأ بصندوق تمهيدي، ثم أقسام مرقّمة، وينتهي بخمسة أجزاء ثابتة. هذه البنية ليست شكلاً، بل طريقة لإجبارنا على التحقق من الإنجاز بدلاً من الافتراض.

**جدول 0-1: الأجزاء الخمسة الثابتة في نهاية كل فصل**

| الجزء | الغرض منه |
|---|---|
| ما تعلّمناه | ملخّص قصير للمفاهيم والقرارات التي ثبّتها الفصل. |
| ما يجب تنفيذه | قائمة مهام عملية على شكل Checklist لكل عضو معني. |
| المخرجات المتوقعة | الملفات والجداول والـ Endpoints والصفحات التي يجب أن تكون موجودة فعلاً. |
| كيف نتحقق من الإنجاز | معايير قبول يمكن اختبارها، وليست عبارات عامة. |
| أخطاء شائعة | الأخطاء التي يقع فيها الفريق عادةً في هذا الجزء وكيف نتجنبها. |

## 0.3 دلالات الصناديق والوسوم

نستخدم ألواناً ثابتة عبر الدليل كله، بحيث تعرف نوع المعلومة من لونها قبل قراءتها.

**جدول 0-2: دلالات الصناديق الملوّنة**

| الصندوق | المعنى |
|---|---|
| صندوق أزرق مخضرّ (Info) | معلومة تقنية مهمة أو توضيح لمصطلح. |
| صندوق برتقالي (Warning) | تحذير: خطأ شائع أو قرار يجب الحذر فيه. |
| صندوق أحمر (Danger) | خطر حقيقي: قد يسبب فشل النظام أو ادّعاءً غير صحيح. |
| صندوق أخضر (Tip) | نصيحة عملية توفّر وقتاً. |
| صندوق بنفسجي (Example) | مثال عملي يربط المفاهيم ببعضها. |
| صندوق رمادي (Note) | ملاحظة جانبية أو تعليق على المنهجية. |

**جدول 0-3: الوسوم المستخدمة في جداول النطاق**

| الوسم | المعنى |
|---|---|
| `MVP` | مطلوب داخل النسخة الأولى القابلة للعرض. |
| `Future` | مؤجَّل إلى ما بعد الـ MVP، ويُذكر كاقتراح لا كوعد. |
| `Required` | تقنية أو خطوة لا يعمل النظام بدونها. |
| `Optional` | مفيدة لكن يمكن تأجيلها دون كسر النظام. |

## 0.4 اتفاقية القيم المؤقتة (Placeholders)

في بعض المواضع لا نعرف القيمة النهائية: مسار ملف أوزان الـ Model، أو اسم الجامعة، أو اسم عضو الفريق. في هذه الحالات نكتب القيمة على شكل وسم واضح بين علامتي أصغر وأكبر، وليس رقماً أو اسماً مُخترعاً.

**جدول 0-4: أمثلة على القيم المؤقتة وأين تُستبدل**

| الشكل المستخدم | أين تُستبدل |
|---|---|
| `<PPE_MODEL_CHECKPOINT_PATH>` | ملف `.env` ثم `core/config.py` |
| `<FIRE_SMOKE_MODEL_CHECKPOINT_PATH>` | نفس المكان، متغير مستقل لكل Model |
| `<MODEL_VERSION>` | يُسجَّل مع كل كشف في جدول `detections` |
| `<DATASET_NAME>` | فصول البيانات والتقييم عند توثيق مصدر التدريب |
| `<UNIVERSITY_NAME>` | صفحة الغلاف وبطاقة المشروع |
| `<SUPERVISOR_NAME>` | صفحة الغلاف |
| `<TEAM_MEMBER_1_NAME>` .. `<TEAM_MEMBER_5_NAME>` | فصل توزيع العمل |

> **لا تُملأ القيم المؤقتة بأرقام تقديرية**
>
> القيمة المؤقتة التي تُستبدل برقم مُخمَّن أسوأ من تركها كما هي، لأنها تُنشئ ادّعاءً لا يمكن الدفاع عنه أمام لجنة المناقشة. اتركها ظاهرة حتى تقيس القيمة الحقيقية.

## 0.5 اصطلاحات الرسوم التوضيحية والأكواد

كل رسم توضيحي له رقم ووصف أسفله، وكل كود له اسم الملف الذي ينتمي إليه في شريط علوي، حتى يعرف القارئ أين يضع الكود بالضبط.




**شكل 0-1:** بنية كل صندوق كود في هذا الدليل: المسار، ثم الكود، ثم طريقة التجربة

- الأكواد والأوامر تُكتب دائماً من اليسار إلى اليمين حتى داخل النص العربي.
- لا نضع تعليقات عربية داخل الكود، لأن أي محرّر أو أداة سيتعامل معها بشكل غير متوقّع.
- كل قيمة قابلة للتغيير تُقرأ من `Environment Variable` وليس من داخل الكود مباشرة.
- مخرجات التشغيل تُوصف **شكلها** (سطر حالة، رسالة نجاح، ملف ناتج) ولا تُكتب كأرقام دقة أو سرعة.

## 0.6 قواعد الأمانة التوثيقية

نطبّق على هذا الدليل نفس المعيار الذي نطبّقه على الكود: لا نكتب ما لم نتحقق منه.

**لا نفعل**

- لا نكتب دقة أو mAP أو FPS لم تُقس فعلياً.
- لا نقول إن ميزة "مكتملة" قبل أن تعمل وتُختبر.
- لا ننسخ أرقاماً من أوراق بحثية وننسبها لمشروعنا.
- لا نكتب أسعاراً أو وعود GPU مجاني.
- لا ننشر مفاتيح أو كلمات مرور أو بيانات عاملين حقيقيين.

**نفعل بدلاً من ذلك**

- نترك جدول نتائج فارغاً بعنوان (يُملأ بعد التجربة).
- نوضّح الفرق بين `MVP` و`Future`.
- نذكر الحدود والقيود صراحةً بدل إخفائها.
- نراجع تراخيص البيانات والـ Models قبل استخدامها.
- نستخدم قيماً مؤقتة واضحة لأي معلومة ناقصة.

**خلاصة المقدمة**

**ما تعلّمناه**

عرفنا بنية الفصل الواحد، ومعنى كل لون ووسم، وكيف نتعامل مع القيم الناقصة، ولماذا لا نكتب أرقاماً لم نقسها.

**ما يجب تنفيذه**

- [ ] إنشاء ملف docs/GLOSSARY.md وملف docs/SCOPE.md.
- [ ] استبدال قيم الغلاف (<UNIVERSITY_NAME>، <SUPERVISOR_NAME>) قبل الطباعة النهائية.

- تثبيت أسماء الأعضاء الخمسة في فصل توزيع العمل.

### المخرجات المتوقعة

- نسخة مطبوعة أو PDF من هذا الدليل يحمل بيانات المشروع الصحيحة.
- ملف مصطلحات موحّد يستخدمه كل الأعضاء.

### كيف نتحقق من الإنجاز

- لا يوجد في الدليل أي وسم قيمة مؤقتة غير مذكور في جدول 0-4.
- كل عضو يعرف معنى كل وسم لوني دون الرجوع للجدول.

### أخطاء شائعة

- تجاهل أجزاء نهاية الفصل لأنها "شكلية"، وهي في الحقيقة أدوات متابعة الفريق.
- ملء القيم المؤقتة بأرقام تقديرية لتجميل التوثيق.
- خلط الوسم `MVP` بالوسم `Future` في جداول النطاق.

CH01 · OVERVIEW & SCOPE

# 01 نظرة عامة على المشروع ونطاقه

ما الذي نبنيه بالضبط، لمن، وما هي الحدود التي لا نتجاوزها

هذا الفصل هو نقطة الاتفاق الأولى للفريق. قبل أن نكتب سطر كود واحد، يجب أن يكون واضحاً للخمسة أعضاء ما هي المشكلة التي نحلّها، ومن سيستخدم النظام، وما الذي يدخل إليه وما الذي يخرج منه، وأين ينتهي نطاق العمل المتفق عليه. كل الفصول التالية تبني على هذا الفصل.

## 1.1 المشكلة التي يعالجها المشروع

في المصانع ومواقع الإنشاء والمستودعات، تُسجَّل كاميرات المراقبة (CCTV) ساعات طويلة من الفيديو كل يوم. المشكلة ليست في نقص التسجيل، بل في أن **لا أحد يستطيع مشاهدة كل هذه الساعات**. النتيجة أن مخالفات معدات الوقاية الشخصية، أو بدايات الحرائق والدخان، أو اقتراب العاملين من المعدات الخطرة، قد تمر دون أن يلاحظها أحد إلا بعد وقوع الحادث.

الحل التقليدي هو تكليف فرد أمن بمشاهدة الشاشات، وهو حل غير موثوق لأن الانتباه البشري يتراجع سريعاً مع الوقت، ولأن المراجعة تكون لحظية ولا تُنتج سجلاً يمكن الرجوع إليه وتحليله. لذلك يحتاج الموقع إلى طبقة برمجية تُفرز ما يحدث تلقائياً، وتُنتج **سجلاً منظّماً للحوادث** قابلاً للبحث والمراجعة والتحليل.

### ماذا نقصد بـ Incident؟

**Incident (حادث/واقعة سلامة):** سجل واحد في قاعدة البيانات يمثّل حدثاً أمنياً محتملاً استخرجته قواعد النظام من نتائج تحليل الفيديو، مثل "عامل بدون خوذة" أو "دخان ظهر في الإطار" أو "شخص قريب جداً من ماكينة". الـ Incident ليس مجرد كشف لحظي، بل وحدة عمل لها وقت بداية ووقت نهاية ودرجة خطورة وحالة مراجعة.

## 1.2 المستخدمون المستهدفون

النظام موجَّه لثلاث فئات داخل المنشأة الصناعية، ولا نستهدف الجمهور العام ولا نقدّم خدمة مدنية.

**جدول 1-1: المستخدمون المستهدفون وأهم احتياجاتهم**

| المستخدم | ما يحتاجه من النظام | ما يراه في الـ Frontend |
|---|---|---|
| مسؤول السلامة `safety_officer` | مراجعة الوقائع، تأكيدها أو رفضها، إضافة ملاحظة، ومتابعة المخالفات المتكررة | قائمة الـ Incidents، تفاصيل الواقعة مع الدليل، أزرار المراجعة |
| مدير الموقع / الإدارة | مؤشرات عامة عن الالتزام بالسلامة خلال فترة زمنية | لوحة الـ Analytics وتقارير الالتزام القابلة للتصدير |
| مهندس النظام / مدير التقنية `admin` | التأكد من عمل الـ Models وحالة الـ Jobs ومعدّل الفشل | صفحة الحالة والإعدادات، سجل الـ Jobs والأخطاء |

## 1.3 مدخلات النظام ومخرجاته

### المدخلات (Inputs)

- ملفات فيديو مسجَّلة من كاميرات CCTV تُرفع من المتصفح.
- مقطع فيديو قصير جاهز للعرض التوضيحي.
- إعدادات النظام: مسارات الـ Model checkpoints، عتبات الثقة، مناطق الخطر.
- قرارات بشرية: تأكيد أو رفض واقعة، وملاحظات المراجعة.

### المخرجات (Outputs)

- سجل وقائع incidents بحالة ودرجة خطورة.
- صور دليل evidence مقتطعة من الإطار الذي وقعت فيه الواقعة.
- حالة ونتائج الـ processing_jobs مع نسبة التقدّم.
- لوحة تحليلات وتقارير يومية وتصدير CSV.

### تنبيه مهم بخصوص المدخلات

المدخل الأساسي في هذه المرحلة هو **فيديو مسجَّل ومرفوع**، وليس بثاً مباشراً (Live stream). المشروع **برمجي بالكامل** ولا يعتمد على شراء حساسات أو أجهزة أو دمج عتاد كشرط للـ MVP.

## 1.4 دور الـ AI Models المتفق عليها

الـ `AI Model` مسؤوليته محدودة ومحدَّدة: أن يستقبل صورة (Frame) ويُعيد قائمة بالأشياء التي رآها داخل الصورة، مع صندوق إحاطة ودرجة ثقة. الـ Model **لا** يعرف شيئاً عن قاعدة البيانات، ولا عن المستخدمين، ولا عن معنى "مخالفة سلامة"، ولا عن التقارير. هذه مسؤوليات التطبيق حوله.

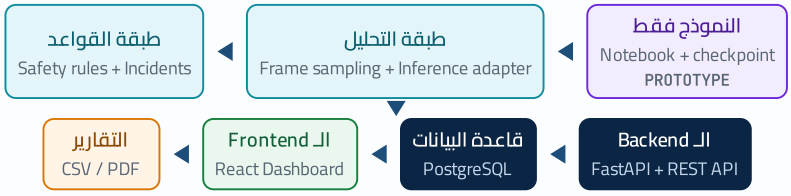
**جدول 1-2: مسؤولية كل مكوّن ذكاء اصطناعي وحدود مسؤوليته**

| المكوّن | المسؤولية | ما هو خارج مسؤوليته |
|---|---|---|
| PPE Detector `MVP` | كشف معدات الوقاية والأشخاص في الإطار وتصنيف حالتها | لا يقرّر أن هذه "مخالفة مسجَّلة" ولا يخزّن شيئاً |
| Person Detector + Tracker `MVP` | كشف الأشخاص وإعطاء كل شخص مُعرّفاً مؤقتاً `track_id` يتبعه بين الإطارات | لا يتعرّف على هوية الشخص الحقيقية ولا يستخدم التعرف على الوجوه |
| Fire / Smoke Detector `MVP` | كشف وجود لهب أو دخان في الإطار | لا يحدّد سبب الحريق ولا يقيّم شدّته الفعلية |
| Machinery / Equipment Detector `MVP` | تحديد مواضع المعدات داخل الإطار لدعم منطق الاقتراب | لا يقيس مسافات حقيقية بالأمتار بدون معايرة كاميرا |

> **قاعدة لا نتنازل عنها**
>
> لا نكتب في أي مكان أن الـ Model وصل إلى دقة معيّنة، أو أن سرعة المعالجة كذا إطار/ثانية، قبل أن نقيس ذلك فعلياً على مجموعة اختبار المشروع. أي رقم لم يُقس يساوي "غير معروف"، وليس رقماً تقديرياً.

## 1.5 الفرق بين الـ AI Model والتطبيق الكامل

أغلب مشاريع التخرّج تتوقف عند "الـ Model يعمل في Notebook". هذا ليس نظاماً. النظام يحتاج طبقات حول الـ Model تُحوّله إلى منتج قابل للاستخدام. الشكل التالي يوضّح هذه الطبقات.




**شكل 1-1:** من نموذج في Notebook إلى نظام كامل – كل مربّع هو مسؤولية يجب تنفيذها واختبارها

## 1.6 مجموعات الميزات الست الأساسية

نطاق المشروع محدَّد بست مجموعات ميزات. لا نضيف ميزات ذكاء اصطناعي جديدة خارجها، لأن أي إضافة تفتح الباب لبيانات جديدة وتدريب جديد واختبار جديد، وهو ما لا يكفي الوقت لتنفيذه بشكل صحيح.

### 1) PPE Detection and Violation Detection

كشف معدات الوقاية الشخصية وتحديد الحالات التي تكون فيها غير مستخدمة، وتحويلها إلى وقائع مخالفة.

### 2) Person Detection and Anonymous Tracking

كشف الأشخاص وتتبّعهم بمُعرّف مؤقت مجهول الهوية لتجميع الملاحظات المتفرّقة في واقعة واحدة.

### 3) Fire and Smoke Detection

كشف ظهور اللهب أو الدخان في الإطار وتسجيله كواقعة عالية الخطورة.

### 4) Machine Proximity Detection

قياس اقتراب الأشخاص من المعدات أو من مناطق خطر مُعرَّفة، وتحويل الاقتراب إلى واقعة.

### 5) Incident Management and Analytics

إدارة الوقائع ومراجعتها وتحليلها إحصائياً من بيانات حقيقية مخزّنة في قاعدة البيانات.

### 6) Safety Compliance Reports

إنتاج تقارير التزام دورية قابلة للتصفية والتصدير لدعم قرارات السلامة.

## 1.7 جدول النطاق التفصيلي

**جدول 1-3: نطاق الميزات — المدخل، المعالجة، المخرج، وهل هي داخل الـ MVP أم لاحقة**

| الميزة | Input | Processing | Output | MVP / Future |
|---|---|---|---|---|

| الميزة | Input | Processing | Output | MVP / Future |
|---|---|---|---|---|
| PPE Detection | إطارات مُنتقاة من الفيديو | PPE Detector ← فلترة بالثقة ← قواعد عدم الاستخدام ← تأكيد زمني | واقعة `ppe_violation` + صورة دليل | `MVP` |
| Person Tracking | إطارات متتالية | Person Detector + Tracker ← مُعرّف مؤقت لكل شخص | سجلات `tracks` + ربط الوقائع | `MVP` |
| Fire / Smoke | إطارات مُنتقاة | Fire/Smoke Detector ← تأكيد على عدة إطارات ← منع التكرار | واقعة `fire_smoke` بخطورة عالية | `MVP` |
| Machine Proximity | إطارات + منطقة خطر مُعرّفة | كشف الشخص والمعدة ← حساب تقاطع الصناديق ← قاعدة الاقتراب | واقعة `machine_proximity` | `MVP` (تقريبي) |
| Incident Management | سجلات الوقائع | تصفية، ترقيم صفحات، مراجعة بشرية، سجل تدقيق | حالة واقعة محدَّثة + ملاحظات مراجعة | `MVP` |
| Analytics | جداول الوقائع | تجميعات SQL على النطاقات الزمنية والأنواع والخطورة | مؤشرات ورسوم بيانية | `MVP` |
| Compliance Reports | نفس البيانات المخزّنة | تجميع يومي/أسبوعي + تصدير | ملف CSV | `MVP` (CSV) |
| PDF Report | بيانات التقارير | توليد ملف PDF منسّق | ملف PDF | `Future` |
| Notifications | وقائع مؤكَّدة | إرسال بريد/إشعار داخلي | إشعار للمستخدم | `Future` |
| Live CCTV Streaming | بث مباشر RTSP | معالجة فورية مستمرة | تنبيه لحظي | `Future` |
| IoT / Sensors | حساسات فيزيائية | دمج إشارات عتاد | قراءات بيئية | `Future` |

## 1.8 الفرق بين Detection و Tracking و Rule و Incident و Analytics

أكثر خطأ يوقع الفرق الطلابية في الارتباك هو خلط هذه المستويات الخمسة. كل مستوى له مدخلات ومخرجات مختلفة، ومسؤول مختلف في الفريق.

**جدول 1-4: مستويات المعالجة الخمسة من الإطار الخام إلى المؤشر التحليلي**

| المستوى | المعنى | المخرج | من ينفّذه |
|---|---|---|---|
| `Detection` | رصد شيء في إطار واحد: صندوق + صنف + ثقة | قائمة كشوفات لإطار واحد | Member 1 (AI) |
| `Tracking` | ربط الكشوفات المتشابهة بين الإطارات لتكوين مسار لشخص واحد | مُعرّف مسار `track_id` مع مدى زمني | Member 1 + Member 2 |
| `Rule Evaluation` | تطبيق قاعدة سلامة على الكشوفات والمسارات والمناطق | قرار: "هذا مخالفة" أو "هذا اقتراب خطر" | Member 2 |
| `Incident` | تجميع المخالفات المتقاربة زمنياً في سجل واحد قابل للمراجعة | صف في جدول `incidents` + دليل | Member 2 + Member 3 |
| `Analytics` | تجميع الوقائع المخزّنة لحساب مؤشرات واتجاهات | أرقام ورسوم في لوحة التحكم | Member 3 + Member 4 |

**مثال يربط المستويات ببعضها**

في الإطار 120 يرى الـ Model شخصاً بثقة 0.88 بدون خوذة. هذا **Detection**. في الإطارات 118 إلى 132 يتكرر نفس الكشف لنفس المسار track_id=7، هذا **Tracking**. عند وصول عدد الإطارات المتتالية إلى الحد المطلوب، تُطلق قاعدة PPE_MISSING_HEAD قراراً بالمخالفة، هذا **Rule Evaluation**. فيُسجَّل صف واحد في incidents من الثانية 4.0 إلى 4.8، هذا **Incident**. وعند حساب عدد مخالفات الخوذة في هذا الأسبوع لهذا الفيديو، هذا **Analytics**.

## 1.9 حدود الدقة في حساب مناطق الخطر

حسابات الاقتراب ومناطق الخطر تعمل على **إحداثيات البكسل** داخل الإطار، وليس على مسافات حقيقية. الإطار صورة مسطّحة ناتجة من إسقاط ثلاثي الأبعاد إلى مستوٍ ثنائي، فالمسافة بالبكسل لا تتناسب خطياً مع المسافة الحقيقية، وتتغيّر حسب موضع الكاميرا وارتفاعها وزاوية ميلها.

**لا نكتب أمتاراً بدون معايرة**

لا يجوز أن نكتب في الواجهة "الشخص يبعد 1.5 متر عن الماكينة" إلا إذا كانت الكاميرا معايَرة (`Camera Calibration`) ومعروف معامل التحويل بين البكسل والمتر في مستوى الأرضية. البديل الصحيح للـ MVP هو التعبير بمقاييس نسبية: نسبة تداخل الصناديق، أو نسبة مسافة مركز الشخص إلى حدود المنطقة من عرض الإطار. ونكتب في الواجهة أن التقدير **تقريبي**.

**جدول 1-5: ما يمكن قوله بأمان وما لا يمكن قوله**

| الحالة | تعبير مقبول في الـ MVP | تعبير مرفوض بدون معايرة |
|---|---|---|
| شخص داخل منطقة خطر مرسومة على الإطار | "داخل منطقة الخطر المُعرَّفة" | "يبعد 2 متر عن الماكينة" |
| تقارب صندوق الشخص مع صندوق المعدة | "تداخل بنسبة 18% من مساحة الصندوق" | "المسافة الفعلية 0.8 متر" |
| سرعة شخص متتبَّع | "إزاحة بالبكسل/ثانية بين إطارين" | "سرعة 1.2 م/ث" |

## 1.10 Planned Architecture مقابل MVP مقابل Future Enhancements

نميّز بصراحة بين ثلاثة مستويات، ونكتب هذه التفرقة في كل فصل حتى لا يتحوّل الوصف إلى ادّعاء بإنجاز.

**جدول 1-6: مستويات الإنجاز الثلاثة**

| المستوى | المعنى | كيف نتعامل معه في هذا الدليل |
|---|---|---|
| `Planned Architecture` | الصورة الكاملة للنظام كما نريده بعد اكتماله، وتشمل مكوّنات قد لا تُنفَّذ الآن | نرسمه ونشرحه بوضوح مع وسم `Future` |
| `MVP` | أصغر نسخة مكتملة تعمل من أول رفع الفيديو إلى تصدير التقرير | هو ما ننفّذه فعلاً ووسم `MVP` في كل فصل |
| `Future Enhancements` | قدرات تُضاف بعد استقرار الـ MVP: PDF، إشعارات، بث مباشر، معايرة كاميرا | نذكرها في آخر الفصل كمقترح لا كوعد |

### لماذا نكتب هذا الفصل أصلاً؟

لأن لجنة المناقشة تسأل دائماً سؤالين: "ما الذي نفّذتموه فعلاً؟" و"ما الذي لم تنفّذوه ولماذا؟". الفريق الذي يجيب بوضوح يكسب ثقة، والفريق الذي يخلط بين المخطَّط والمنفَّذ يفقدها بسرعة.

## خلاصة الفصل 1

### ما تعلّمناه

حدّدنا المشكلة (فيديو كثير ومتابعة بشرية غير موثوقة)، والمستخدمين الثلاثة، والمدخلات والمخرجات. وفصلنا بين مسؤولية الـ Model ومسؤولية التطبيق، وثبّتنا ست مجموعات ميزات لا نتجاوزها، وميّزنا بين Detection و Tracking و Rule و Incident و Analytics، ووضّحنا أن مناطق الخطر تقريبية بدون معايرة.

### ما يجب تنفيذه

- [ ] الموافقة على جدول النطاق (جدول 1-3) وتثبيته في ملف docs/SCOPE.md.
- [ ] تحديد قيمة كل Placeholder يخص الـ Models والبيانات.
- [ ] كتابة تعريفات المصطلحات الخمسة في docs/GLOSSARY.md.
- [ ] تحديد مواصفات فيديو العرض التوضيحي (المدة، الدقة، المحتوى).

### المخرجات المتوقعة

- ملف docs/SCOPE.md يحتوي جدول النطاق النهائي.
- ملف docs/GLOSSARY.md يحتوي المصطلحات وتعريفاتها بالعربية.
- قائمة نهائية بأسماء الـ Classes في كل Model مكتوبة حرفياً.

### كيف نتحقق من الإنجاز

- كل عضو في الفريق يستطيع شرح الفرق بين Detection و Incident في دقيقتين.
- لا توجد ميزة في جدول النطاق غير مصنّفة `MVP` أو `Future`.
- لا يوجد رقم دقة أو سرعة مكتوب بدون مصدر قياس.

### أخطاء شائعة

- اعتبار أن "الـ Model يعمل" يعني أن المشروع انتهى.
- إضافة ميزات ذكاء اصطناعي جديدة أثناء التنفيذ لأنها "تبدو سهلة".
- كتابة مسافات بالأمتار في الواجهة اعتماداً على البكسل.
- استخدام مصطلحات Detection و Incident بالتبادل دون تمييز.

CH02 · SYSTEM ARCHITECTURE

# 02 البنية الكاملة للنظام

من رفع الفيديو حتى التقرير: المكوّنات ومسؤولياتها واتجاه التواصل

يرسم هذا الفصل الصورة الكبيرة: مكوّنات **SafeVision AI**، ومسؤولية كل مكوّن، وكيف تتحدّث مع بعضها، وهو المرجع عند الشك في "من يفعل ماذا".

## 2.1 معنى البنية المعمارية

**System Architecture (البنية المعمارية):** وصف للمكوّنات ومسؤولية كل مكوّن وطريقة تواصلها. اخترنا **Modular Monolith** (كتلة واحدة معيارية) مع فصل واضح بين الـ **Backend** والـ **Frontend** وعملية معالجة مستقلة (السبب في 2.9).

## 2.2 المكوّنات الرئيسية ومسؤولية كل مكوّن

لكل مكوّن مسؤولية واحدة واضحة، والجدول التالي هو المرجع السريع لها:

**جدول 2-1: المكوّنات الرئيسية ومسؤولية كل مكوّن**

| المكوّن | المسؤولية الأساسية | التقنية |
|---|---|---|
| Video Upload Interface | استقبال ملف الفيديو، التحقق من امتداده وحجمه، وإرساله لـ API | React + Axios |
| Frontend | العرض والتفاعل فقط: القوائم، التفاصيل، التحليلات، المراجعة | React + TypeScript + Vite |
| FastAPI Backend | بوابة الـ API: التحقق، التوجيه، طبقات المنطق، الوصول لقاعدة البيانات | FastAPI + Uvicorn |
| Video Processing Service | فك الفيديو إطاراً إطاراً وتجهيزه للاستنتاج | OpenCV + NumPy |
| AI Inference Components | تشغيل الـ checkpoints لإنتاج الكشوفات والتتبّع | PyTorch + Ultralytics + ByteTrack |
| Safety Rules Engine | تحويل الكشوفات الخام إلى وقائع وفق قواعد السلامة | Python |
| Background Job Processing | تنفيذ المعالجة الطويلة خارج دورة الطلب وتحديث التقدّم | DB-backed queue + worker |
| PostgreSQL | حفظ كل البيانات الوصفية: الفيديوهات، الـ Jobs، الوقائع، الدليل | PostgreSQL 16 |
| Evidence / Video Storage | حفظ الملفات الكبيرة (الفيديو، صور الدليل) على القرص | Local filesystem |
| Reporting Module | تجميع الوقائع في تقارير يومية وتصدير CSV | Pandas + FastAPI |
| Optional Notifications | تنبيه المسؤول عند واقعة حرجة (مستقبلي) | Pending |

**تمييز أساسي**

مخرجات الـ **AI Inference Components** هي **كشوفات خام**؛ الواقعة incident لا تُنشأ إلا بعد قواعد السلامة. الفيديو والدليل يبقيان على القرص، وPostgreSQL يحفظ البيانات الوصفية فقط.

## 2.3 الفرق بين الطلب المتزامن والوظيفة غير المتزامنة

أهم فكرة معمارية: تحليل الفيديو يحتاج آلاف الإطارات ولا يمكن أن يحدث داخل طلب HTTP عادي.

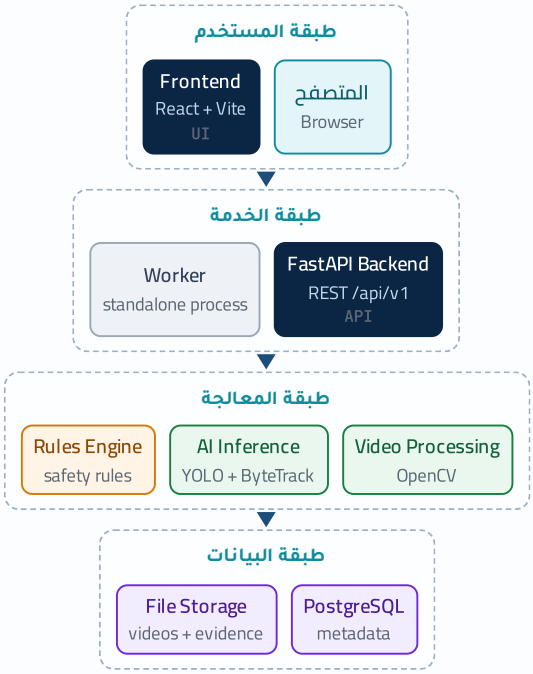
**جدول 2-2: المتزامن مقابل غير المتزامن**

| الوجه | طلب متزامن (Synchronous) | وظيفة غير متزامنة (Asynchronous job) |
|---|---|---|
| المدة | أجزاء من الثانية | من ثوانٍ إلى ساعات |
| الرد | البيانات كاملة داخل الرد | رد فوري بمعرّف المهمة وحالتها |
| المثال | GET /api/v1/videos | POST /api/v1/videos/{video_id}/process |
| رمز النجاح | 200 | Accepted 202 |
| متابعة الحالة | لا حاجة | Polling على GET /api/v1/jobs/{job_id} |
| عند الفشل | خطأ فوري في نفس الطلب | حالة failed مع error_message |

**قاعدة عملية**

أي عملية قد تتجاوز ثانية تُنفَّذ كـ job؛ رفع الملف متزامن وتحليله غير متزامن دائماً.

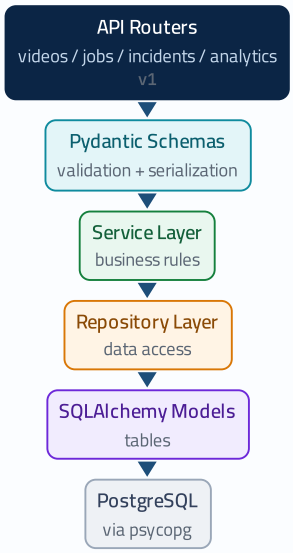
## 2.4 مخطط المعمارية عالي المستوى




**شكل 2-1:** البنية عالية المستوى من المستخدم حتى التخزين

## 2.5 معمارية مكوّنات الـ Backend

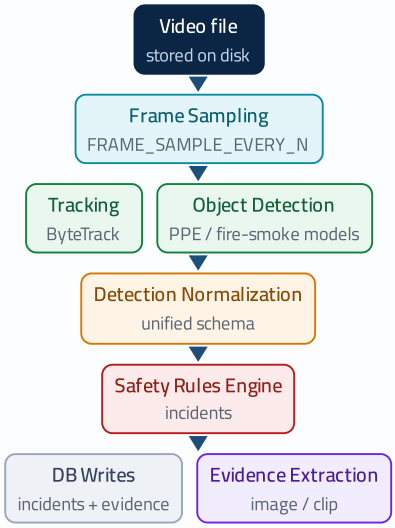
تطبيق واحد يُنشر كوحدة واحدة، لكنه مقسّم داخلياً إلى وحدات معيارية (videos, jobs, incidents, analytics, reports) وطبقات أفقية. القاعدة: الراوتر لا يتحدّث مع قاعدة البيانات مباشرة، والخدمة لا تعرف الـ HTTP.




**شكل 2-2:** طبقات الـ Backend: كل طبقة تعرف التي تحتها فقط

## 2.6 خط معالجة الـ AI

يمرّ الفيديو بمراحل متتالية، وكل مرحلة تنتج مخرجاً يستهلكه ما بعدها؛ وأي خطأ يُسجَّل في الـ job ولا يُسقط النظام.




**شكل 2-3:** خط معالجة الـ AI من الملف إلى الوقائع والدليل

## 2.7 التواصل بين الـ Frontend والـ Backend

كل التواصل عبر REST فوق HTTP بصيغة JSON، والمسارات تبدأ بـ /api/v1؛ الواجهة لا تعرف قاعدة البيانات.

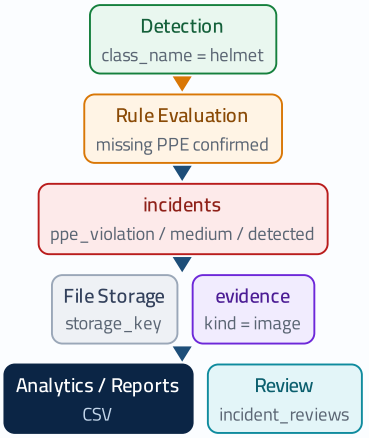
| المستخدم | Frontend | Backend | DB / Worker |
|---|---|---|---|
| يختار ملفاً | `POST /api/v1/videos` | validate + store | INSERT videos |
| يرى الفيديو | Created 201 | VideoOut | commit |
| يطلب المعالجة | `POST /api/v1/videos/{video_id}/process` | create job | INSERT processing_jobs |
| | Accepted 202 | JobOut (queued) | worker picks up |
| يراقب التقدّم | `GET /api/v1/jobs/{job_id}` | read progress | UPDATE job |
| يفتح الوقائع | `GET /api/v1/incidents` | paginate | SELECT incidents |

**شكل 2-4:** تدفّق الرسائل بين المستخدم والواجهة والـ Backend وقاعدة البيانات

متابعة الحالة تتم عبر **Polling**: تسأل الواجهة عن الـ job دورياً حتى تتغير من running إلى completed أو failed. لم نستخدم WebSocket في الـ MVP.

## 2.8 تدفّق بيانات الوقائع والدليل

تُكتب الواقعة في incidents، ثم يُقتطع إطار ويُحفظ كدليل في evidence مع storage_key.




**شكل 2-5:** من الكشف الخام إلى الواقعة والدليل ثم المراجعة والتقرير

مسار الحالة ثابت: confirmed ← pending_review ← detected أو closed ← rejected، وكل تغيير يُسجَّل في incident_reviews.

## 2.9 لماذا Modular Monolith أفضل من Microservices لفريق طلابي

**Modular Monolith:** تطبيق واحد مقسّم داخلياً إلى وحدات. **Microservices:** خدمات منفصلة، لكل منها عملية وقاعدة بيانات وتواصل عبر الشبكة.

**جدول 2-3: مقارنة الاختيار المعماري**

| المعيار | Modular Monolith (اختيارنا) | Microservices |
|---|---|---|
| التشغيل المحلي | أمر واحد يُشغّل الـ Backend كله | عدة خدمات ومنافذ وعمليات |
| التصحيح (Debug) | Stack trace واحد | تتبّع عبر الشبكة وعدة سجلات |
| قاعدة البيانات | قاعدة واحدة وعلاقات حقيقية | قواعد متعدّدة ومزامنة معقّدة |
| الاختبار | اختبار الـ API من مكان واحد | اختبار تكامل بيني (contract testing) |
| مناسب لخمسة أعضاء؟ | نعم | لا، عبء تشغيلي أكبر من قيمة المشروع |

السبب الأهم: نظامنا سلسلة كتابة مترابطة (فيديو ← evidence ← incidents ← detections ← job). في الـ Microservices تصبح معاملات موزّعة معقّدة، أمّا هنا فتبقى معاملة قاعدة بيانات واحدة مضمونة. لا نرفضها بل نؤجّلها، والحدود النظيفة تجعل الانتقال ممكناً لاحقاً.

## 2.10 اتجاه التواصل بين المكوّنات

التواصل في خط واحد ولا يعود للخلف:

- **المتصفح ← Frontend:** تفاعل المستخدم فقط، ولا وصول مباشر لخدمة أخرى.
- **Frontend ← Backend:** طلبات HTTP فقط، بلا معرفة بقاعدة البيانات.
- **Backend ← PostgreSQL / Storage:** قراءة وكتابة عبر طبقة الـ Repository.
- **Worker ← Models / OpenCV:** المعالجة الثقيلة تحدث في الـ worker لا في عملية الـ API.
- **Worker ← PostgreSQL:** تحديث تقدّم الـ job وكتابة الوقائع والدليل.

> **لماذا؟**
> لأن تشغيل الـ Model داخل عملية الـ API يجمّد بقية الطلبات؛ فصل الـ worker يبقي النظام قابلاً للاستخدام.

### خلاصة الفصل 2

**ما تعلّمناه**

رسمنا البنية الكاملة، وحدّدنا مسؤولية كل مكوّن، وفرّقنا بين الطلب المتزامن والوظيفة غير المتزامنة، وشرحنا لماذا Modular Monolith هو الخيار الصحيح لفريق من خمسة أعضاء.

**ما يجب تنفيذه**

- [ ] حفظ الأشكال الخمسة في docs/ARCHITECTURE.md.
- [ ] الاتفاق على أن كل معالجة ثقيلة تُنفَّذ كـ job.
- [ ] تثبيت حدود الوحدات: videos, jobs, incidents, analytics, reports.
- [ ] تأكيد أن الفيديو والدليل لا يُخزَّنان في PostgreSQL.

**المخرجات المتوقعة**

- ملف docs/ARCHITECTURE.md يضم الأشكال 2-1 حتى 2-5.
- جدول المكوّنات (جدول 2-1) معتمد من الفريق.
- توصيف واضح لمسار حالة الـ job وحالة الواقعة.

**كيف نتحقق من الإنجاز**

- كل عضو يستطيع رسم شكل 2-1 من الذاكرة.
- لا يوجد مسار يربط الـ Frontend بقاعدة البيانات مباشرة.
- الفرق بين Detection و Incident و Job واضح في كل وثيقة.

**أخطاء شائعة**

- اعتبار كل وحدة معيارية ميكروسيرفس ونشرها منفصلة.
- تشغيل الـ Model داخل عملية الـ API فتتجمد الطلبات.
- تخزين ملفات الفيديو داخل قاعدة البيانات.
- جعل الواجهة تقرأ مسارات الملفات بنفسها.

CH03 · TECHNOLOGY STACK

# 03 حزمة التقنيات واختيار الأدوات

لماذا اخترنا كل أداة، وما البدائل، وما هو مطلوب في الـ MVP

اختيار التقنية ليس "أيّ شيء يعمل"، بل قرار يجب أن نبرّره. يعرض هذا الفصل كل أداة، ووظيفتها، وسبب اختيارها، وحالتها (MVP أم اختيارية)، وبديلها، حتى يكون كل عضو قادراً على الدفاع عن كل قرار أمام لجنة المناقشة.

## 3.1 معايير اختيار التقنية

طبّقنا خمسة معايير على كل خيار: **الملاءمة للمهمة** (هل الأداة مصمَّمة لهذا العمل؟)، **التوفّر والتوثيق** الرسمي الجيد، **سهولة التعلّم** لعضو مبتدئ، **التكامل** مع بقية الحزمة، و**التشغيل المحلي** بلا تعقيد أو تكلفة.

## 3.2 جدول حزمة التقنيات الكامل

**جدول 3-1: حزمة تقنيات SafeVision AI**

| المجال | الأداة | وظيفتها ولماذا | الحالة | البديل |
|---|---|---|---|---|
| لغة | Python 3.11 | لغة الـ Backend والذكاء الاصطناعي معاً | MVP | Node.js / Java |
| تعلم عميق | PyTorch | محرّك تشغيل شبكات الكشف | MVP | TensorFlow |
| كشف | Ultralytics YOLO | كشف الأجسام والفئات المدربة | MVP | Detectron2 |
| رؤية حاسوبية | OpenCV | قراءة الفيديو والإطارات والكتابة على الصور | MVP | imageio / ffmpeg |
| تتبّع | ByteTrack | تتبّع الأشخاص بين الإطارات (track_id) | MVP | DeepSORT |
| حساب عددي | NumPy | عمليات المصفوفات والإحداثيات | MVP | — |
| تحليل بيانات | Pandas | تجميع الوقائع للتقارير والتصدير | MVP | SQL مباشر |
| إطار ويب | FastAPI + Uvicorn | بناء الـ REST API وتشغيله | MVP | Flask / Django |
| تحقق | Pydantic v2 + pydantic-settings | التحقق من المدخلات وإدارة الإعدادات | MVP | marshmallow |
| ORM | SQLAlchemy 2.0 | تعريف الجداول والتعامل معها بأمان | MVP | Django ORM |
| migrations | Alembic | تتبّع تغييرات مخطط قاعدة البيانات | MVP | SQL يدوي |
| قاعدة بيانات | PostgreSQL 16 | حفظ كل البيانات الوصفية والعلاقات | MVP | MySQL / SQLite |
| مشغّل DB | psycopg (v3) | الاتصال بـ PostgreSQL من Python | MVP | asyncpg |
| مصادقة | JWT (PyJWT) + bcrypt | تسجيل الدخول وتجزئة كلمات المرور | Optional | OAuth / Session |
| مهام خلفية | DB-backed queue + worker | تنفيذ المعالجة الطويلة خارج الطلب | MVP | Celery + Redis |
| تحديث الحالة | Polling | الواجهة تسأل عن حالة الـ job دورياً | MVP | WebSocket |
| واجهة | React 18 + TypeScript 5 + Vite 5 | واجهة مستخدم سريعة وآمنة الأنواع | MVP | Vue / Svelte |
| تصميم | Tailwind CSS 3 | تنسيق الواجهة بفئات جاهزة | MVP | Bootstrap |
| رسوم بيانية | Recharts | رسم مؤشرات الـ Analytics | MVP | Chart.js |
| HTTP client | Axios | إرسال الطلبات للـ Backend | MVP | fetch |
| توجيه | react-router-dom v6 | التنقل بين صفحات الواجهة | MVP | TanStack Router |
| حاويات | Docker + Docker Compose | تشغيل موحّد للتسليم | MVP | تشغيل يدوي |
| CI | GitHub Actions | فحص آلي للكود عند الدفع | Future | — |

## 3.3 لماذا Python للـ Backend وللذكاء الاصطناعي معاً

**Python:** لغة عالية المستوى تُقرأ كالإنجليزية تقريباً. اخترناها لأن عالم الذكاء الاصطناعي مبني حولها (PyTorch وUltralytics وOpenCV كلها مكتبات Python)، فلو اخترنا لغة أخرى للـ Backend لاحتجنا عمليتين بلغتين والتنسيق بينهما. النسخة المستهدفة Python 3.11، ونستخدم venv لعزل مكتبات المشروع عن مكتبات النظام.

## 3.4 حزمة الذكاء الاصطناعي والرؤية الحاسوبية

- **PyTorch:** المحرّك الذي يشغّل أوزان الشبكات، والأكثر استخداماً في المراجع.
- **Ultralytics YOLO:** كشف الأجسام بدالة واحدة، وتوفّر تتبّعاً جاهزاً عبر model.track فلا نكتب منطق الشبكة بأنفسنا.
- **ByteTrack:** تتبّع يمنح كل شخص معرّفاً ثابتاً عبر الإطارات لنحسب مدة المخالفة.
- **OpenCV:** قراءة الفيديو واستخراج الإطارات وتغيير الحجم وكتابة صور الدليل.
- **NumPy:** العمليات العددية وحساب التداخل والمسافات. **Pandas:** تجميع الوقائع للتقارير والتصدير.

## 3.5 حزمة الـ Backend وقاعدة البيانات

- **FastAPI:** يبني الـ API بسرعة ويولّد توثيق Swagger تلقائياً من الأنواع، مع تحقق آلي من المدخلات. **Uvicorn:** الخادم الذي يشغّله بأمر uvicorn app.main:app --reload.
- **Pydantic v2:** يتحقق من شكل البيانات عبر نماذج بأنواع صريحة، و pydantic-settings يقرأ متغيّرات البيئة في كائن إعدادات واحد.
- **SQLAlchemy 2.0:** يربط Python بـ PostgreSQL ويعرّف الجداول كأصناف. **Alembic:** يتتبّع تغييرات المخطط كملفات migration قابلة للتطبيق والتراجع.
- **PostgreSQL 16:** تدعم UUID والأنواع والعناصر الجزئية الفريدة (partial unique index) التي نحتاجها لمنع مهمّتين على نفس الفيديو. اخترناها على SQLite لأن الأخيرة لا تكفي لبيئة متعددة العمليات.
- **psycopg (v3):** المشغّل الذي يربط SQLAlchemy بـ PostgreSQL.
- **JWT + bcrypt:** مصادقة اختيارية خلف AUTH_ENABLED؛ الـ JWT رمز دخول موقّع، وbcrypt تجزئة كلمات المرور فلا تُخزَّن كنص صريح.

## 3.6 حزمة الـ Frontend وأدوات التطوير

- **React 18:** بناء الواجهة من مكوّنات قابلة لإعادة الاستخدام. **TypeScript 5:** أنواع صريحة تكشف الأخطاء قبل التشغيل. **Vite 5:** تشغيل وبناء سريع بتحديث فوري.
- **Tailwind CSS 3** للتنسيق، **Recharts** للمخططات، **Axios** لطلبات HTTP، **react-router-dom v6** للتنقل.
- **Git + GitHub** لتتبّع الإصدارات والتعاون عبر فروع وPull Requests، و **VS Code** كمحرّر مع إضافات Python وTailwind وESLint.
- **Node.js + npm** لبيئة الـ Frontend، و**venv** لبيئة الـ Backend.
- **Docker** لحزم التطبيق، و**Docker Compose** لتشغيل عدة حاويات بأمر واحد عبر **GitHub Actions**. docker-compose.yml مؤجَّلة لما بعد الـ MVP.

## 3.7 ما لا نستخدمه في الـ MVP ولماذا

قرار "ما لا نستخدمه" لا يقل أهمية؛ كل تقنية إضافية تستهلك وقت الفريق وتعقّد التشغيل.

**جدول 3-2: تقنيات مستثناة من الـ MVP وتكلفتها**

| التقنية | التكلفة الحقيقية لو استخدمناها | البديل الذي اخترناه |
|---|---|---|
| Kubernetes | عناقيد وملفات YAML وتشغيل سحابي معقّد لا يحتاجه مشروع تخرّج | Docker Compose محلياً |
| Kafka | وسيط رسائل ثقيل يحتاج تشغيلاً ومراقبة وحجمنا لا يبرّره | جدول processing_jobs كطابور |
| Redis | خدمة إضافية تُدار وتُراقب لحالة الطابور | حالة الـ job في PostgreSQL |
| Celery | إطار مهام يعتمد على وسيط خارجي ويصعّب التصحيح | worker بسيط يقرأ من قاعدة البيانات |
| Microservices | شبكة وعدة قواعد بيانات ومعاملات موزّعة على خمسة أعضاء | Modular Monolith |
| RTSP Live | بث مباشر يحتاج تعاملاً لحظياً وحالات فشل متعدّدة | رفع ملفات فيديو مسجَّلة |

> **تحذير**
>
> لا نضيف أي تقنية من هذا الجدول "لأنها مشهورة"؛ أي إضافة تتطلب موافقة الفريق وتبريراً مكتوباً.

## 3.8 ملاحظة عن أرقام الإصدارات

الإصدارات المذكورة نقطة بداية لا وعد نهائي. القاعدة: تحقّق دائماً من أحدث إصدار مستقر من المصادر الرسمية (PyPI وnpm والتوثيق الرسمي) قبل التثبيت في ملفات المتطلبات، وعند التعارض نختار الإصدار الذي يتوافق معه أكبر عدد من المكتبات الأساسية (PyTorch وUltralytics أولاً).

## خلاصة الفصل 3

### ما تعلّمناه

عرفنا كل أداة: وظيفتها وسبب اختيارها وحالتها وبديلها. وثبّتنا أن Python تخدم الـ Backend والذكاء الاصطناعي معاً، وأن FastAPI وPostgreSQL وReact أعمدة الحزمة، وأن مجموعة تقنيات شهيرة مرفوضة في الـ MVP.

### ما يجب تنفيذه

- [ ] تثبيت الإصدارات في backend/requirements.txt وfrontend/package.json.
- [ ] التحقق من أحدث إصدار مستقر لكل مكتبة أساسية من المصدر الرسمي.
- [ ] تأجيل كل تقنيات جدول 3-2 حتى إشعار مكتوب.

### المخرجات المتوقعة

- ملف backend/requirements.txt بإصدارات مثبّتة.
- ملف frontend/package.json بإصدارات مثبّتة.
- وثيقة docs/STACK.md تضم جدول 3-1 وتبرير كل اختيار.

### كيف نتحقق من الإنجاز

- كل عضو يشرح لماذا اخترنا FastAPI لا Flask، وPostgreSQL لا SQLite.
- لا مكتبة في المشروع بدون مبرّر مكتوب.
- لا رقم إصدار غير موثّق من مصدر رسمي.

### أخطاء شائعة

- إضافة Redis أو Celery بلا حاجة فعلية.
- تثبيت إصدارات قديمة من درس قديم دون تحقق.
- تشغيل الـ Model داخل عملية الـ API مباشرة.

CH04 · REPOSITORY LAYOUT

# 04 هيكل المستودع وتنظيم المجلدات

أين يوضع كل ملف، وما لا يجب أن يدخل Git أبداً

نثبّت هنا شجرة المجلدات كاملة، ووظيفة كل مجلد، وما يُرفع إلى GitHub وما لا يُرفع أبداً، مع ملفَّي .gitignore و.env.example جاهزين للنسخ.

## 4.1 شجرة المجلدات الكاملة

هذه الشجرة هي المرجع الرسمي؛ كل ملف جديد يجب أن يجد مكانه فيها، ولا ننشئ مجلدات عشوائية.

repository tree — text

```
safevision-ai/
  backend/                  # FastAPI application (package name: app)
    app/
      main.py               # entry point, creates the FastAPI app
      api/v1/
        router.py           # aggregates all v1 routers
        endpoints/          # health.py, videos.py, jobs.py, incidents.py ...
      core/                 # config.py, logging.py, security.py
      db/                   # session.py, base.py
      models/               # SQLAlchemy models
      schemas/              # Pydantic schemas
      repositories/         # data access layer
      services/             # business logic layer
      ai/                   # models loading + inference wrappers
      workers/              # job runner process
      storage/              # local file storage helpers
      reports/              # reporting + CSV export
      utils/                # shared helpers
    migrations/             # Alembic migrations
    tests/                  # backend tests
    requirements.txt
  frontend/                 # React + TypeScript + Vite
    src/
    public/
    package.json
    vite.config.ts
  models/                   # model checkpoints (NOT in git)
  data/
    raw/
    processed/
    sample/
  storage/                  # uploads | evidence | reports (NOT in git)
    uploads/
    evidence/
    reports/
  docs/                     # architecture, API, glossary
  scripts/                  # helper scripts (db init, demo)
  docker-compose.yml
  .env.example
  .gitignore
  README.md
```

## 4.2 وظيفة كل مجلد مهم

### جدول 4-1: وظيفة المجلدات الأساسية

| المجلد | وظيفته | يُرفع إلى Git؟ |
|---|---|---|
| backend/app | كل كود الـ API بطبقاته (routers, services, repositories, models, schemas) | نعم |
| backend/migrations | ملفات Alembic التي تتبّع تغييرات قاعدة البيانات | نعم |
| backend/tests | اختبارات الـ Backend | نعم |
| frontend/src | مكوّنات React وصفحاتها وطبقة الاتصال بالـ API | نعم |
| models | أوزان الـ Models المدرَّبة (ملفات كبيرة بالميجابايت) | لا |
| data | البيانات الخام والمعالجة وعيّنات الاختبار | العيّنات الصغيرة فقط |
| storage | الفيديوهات المرفوعة وصور الدليل والتقارير المولَّدة | لا |

| المجلد | وظيفته | يُرفع إلى Git؟ |
|---|---|---|
| `docs` | وثائق المعمارية وعقد الـ API والمصطلحات | نعم |
| `scripts` | سكربتات مساعدة مثل تهيئة قاعدة البيانات وتشغيل العرض التوضيحي | نعم |

### 4.3 ما لا يجب أن يدخل Git أبداً ولماذا

**Git:** نظام تتبّع الإصدارات، وهو مصمَّم للنصوص الصغيرة لا للملفات الضخمة، لذلك ثلاثة أصناف لا نرفعها:

- **الملفات الكبيرة (Binaries):** الفيديوهات وأوزان الـ Models؛ رفعها يُنتفخ حجم المستودع ويجعل كل git clone بطيئاً.
- **البيانات المولَّدة (Generated):** صور الدليل والتقارير؛ مخرجات تشغيل يمكن إعادة توليدها.
- **الأسرار (Secrets):** كلمة مرور قاعدة البيانات وSECRET_KEY ومفاتيح S3؛ تسريبها خطر أمني حقيقي.

> **خطر**
>
> رفع ملف .env أو أي مفتاح حقيقي خطأ لا يُصلَح بمجرد الحذف لأن التاريخ يحتفظ به. نرفع .env.example فقط وننشئ .env محلياً.

### 4.4 ملف .gitignore كامل

يوضع في جذر المستودع ويخبر Git بالملفات التي يتجاهلها، مرتّبة حسب النوع.

**.gitignore** — gitignore

```
# ---- Python ----
__pycache__/
*.py[cod]
*.egg-info/
.venv/
venv/
env/
.pytest_cache/
.mypy_cache/
.ruff_cache/
htmlcov/
.coverage

# ---- Environment & secrets ----
.env
.env.*
!.env.example
*.pem
*.key

# ---- Node / frontend ----
node_modules/
dist/
dist-ssr/
.vite/
npm-debug.log*
yarn-error.log*

# ---- Model checkpoints (large binaries) ----
models/
*.pt
*.pth
*.onnx
*.weights

# ---- Local storage & generated data ----
storage/uploads/
storage/evidence/
storage/reports/
data/raw/
data/processed/
*.mp4
*.avi
*.mov
*.mkv

# ---- Reports & exports ----
*.csv
*.pdf
!data/sample/**

# ---- Editor / OS ----
```

```
.vscode/
.idea/
.DS_Store
Thumbs.db
*.log

# ---- Docker / build ----
*.pid
docker-compose.override.yml
```

> **ملاحظة**
>
> السطر !.env.example يسمح بالقالب فقط، و!data/sample/** يسمح برفع عيّنات صغيرة للاختبار.

### 4.5 ملف .env.example بقيم Placeholder فقط

قالب يُرفع إلى GitHub ويُنسخ على كل جهاز إلى .env. قيمه Placeholder بالشكل <NAME>، وأسماء المتغيّرات مطابقة لـ SPEC.md.

.env.example — ini

```
# ---- Application ----
APP_NAME=SafeVision AI
APP_ENV=development
APP_DEBUG=true
API_V1_PREFIX=/api/v1
AUTH_ENABLED=false
SECRET_KEY=<CHANGE_ME_LONG_RANDOM_SECRET>
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
CORS_ORIGINS=http://localhost:5173

# ---- PostgreSQL ----
POSTGRES_USER=<DB_USER>
POSTGRES_PASSWORD=<DB_PASSWORD>
POSTGRES_DB=safevision
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
DATABASE_URL=postgresql+psycopg://<DB_USER>:<DB_PASSWORD>@localhost:5432/safevision

# ---- Storage ----
STORAGE_BACKEND=local
LOCAL_STORAGE_ROOT=./storage
UPLOAD_DIR=./storage/uploads
EVIDENCE_DIR=./storage/evidence
REPORT_DIR=./storage/reports
S3_ENDPOINT_URL=<S3_ENDPOINT_URL>
S3_BUCKET=<S3_BUCKET>
S3_ACCESS_KEY_ID=<S3_ACCESS_KEY_ID>
S3_SECRET_ACCESS_KEY=<S3_SECRET_ACCESS_KEY>
S3_REGION=<S3_REGION>

# ---- Upload limits ----
MAX_UPLOAD_MB=500
ALLOWED_VIDEO_EXTENSIONS=mp4,avi,mov,mkv
UPLOAD_CHUNK_MB=4

# ---- AI models ----
MODEL_DIR=./models
PPE_MODEL_PATH=<PPE_MODEL_CHECKPOINT_PATH>
FIRE_SMOKE_MODEL_PATH=<FIRE_SMOKE_MODEL_CHECKPOINT_PATH>
PERSON_TRACKER_PATH=<PERSON_TRACKER_CHECKPOINT_PATH>
MODEL_DEVICE=cpu
DETECTION_CONF_THRESHOLD=0.35
NMS_IOU_THRESHOLD=0.45
INFERENCE_IMG_SIZE=640
PERSIST_ALL_DETECTIONS=false
FRAME_SAMPLE_EVERY_N=5
MAX_FRAMES_PER_JOB=0

# ---- Safety rules ----
INCIDENT_CONFIRM_FRAMES=3
INCIDENT_COOLDOWN_SECONDS=10
PPE_CONF_THRESHOLD=0.40
FIRE_CONF_THRESHOLD=0.45
SMOKE_CONF_THRESHOLD=0.40
PROXIMITY_MIN_OVERLAP=0.10
DANGER_ZONE_CONFIG=<DANGER_ZONE_CONFIG_JSON>

# ---- Worker ----
```

```
WORKER_POLL_INTERVAL_SECONDS=3
WORKER_ID=worker-1
WORKER_CONCURRENCY=1
JOB_MAX_RETRIES=2

# ---- Frontend (Vite) ----
VITE_API_BASE_URL=http://localhost:8000/api/v1
VITE_ENABLE_AUTH=false
```

**لماذا قيم Placeholder؟**

لأن القالب يوثّق **أسماء** المتغيّرات لا قيمها؛ كل عضو يضع قيمه الخاصة، مثل <PPE_MODEL_CHECKPOINT_PATH> الذي يحدّده فريق الذكاء الاصطناعي.

## 4.6 اتفاقيات تسمية الملفات والفروع

اتفاق التسمية يمنع الالتباس ويسهّل البحث. نلتزم بالقواعد التالية دون استثناء:

**جدول 4-2: اتفاقيات التسمية**

| النوع | القاعدة | أمثلة |
|---|---|---|
| ملفات Python | حروف صغيرة مع شرطة سفلية snake_case | video_service.py, processing_job.py |
| أصناف Python | PascalCase | VideoRepository, VideoOut |
| مكوّنات React | PascalCase بامتداد .tsx | VideoList.tsx, IncidentCard.tsx |
| جداول قاعدة البيانات | جمع بحروف صغيرة snake_case | videos, processing_jobs, incidents |
| الفروع | type/short-topic | feat/video-upload, fix/cors-headers |
| رسائل الـ commit | نوع + وصف قصير | feat: add job status endpoint |

**نصيحة**

الفرع الرئيسي main يبقى دائماً قابلاً للتشغيل. لا يُدمج أي فرع إلا عبر Pull Request يراجعه عضو آخر على الأقل.

### خلاصة الفصل 4

**ما تعلّمناه**

ثبّتنا شجرة المجلدات الرسمية، ووظيفة كل مجلد، وأصناف الملفات الثلاثة التي لا تدخل Git (الملفات الكبيرة، والمخرجات المولّدة، والأسرار)، وكتبنا .gitignore و .env.example كاملين، واتفقنا على قواعد التسمية والفروع.

**ما يجب تنفيذه**

- [ ] إنشاء المستودع وشجرة المجلدات كما في القسم 4.1.
- [ ] إضافة .gitignore و.env.example قبل أول commit.
- [ ] نسخ .env.example إلى .env على كل جهاز وملء القيم.
- [ ] الاتفاق على حماية الفرع main من الدفع المباشر.

**المخرجات المتوقعة**

- مستودع GitHub نظيف يحتوي الكود فقط بلا ملفات كبيرة.
- ملف README.md يشرح خطوات التشغيل.
- قالب بيئة موحّد لكل الأعضاء.

**كيف نتحقق من الإنجاز**

- الأمر git status لا يُظهر أي ملف في storage/ أو models/.
- حجم المستودع بعد git clone صغير (ميجابايتات قليلة).
- لا يوجد ملف .env أو مفتاح حقيقي في تاريخ المستودع.

**أخطاء شائعة**

- رفع ملف .env الحقيقي بالخطأ.

- رفع أوزان الـ Models فيصبح الاستنساخ بطيئاً جداً.
- إنشاء مجلدات بأسماء عشوائية خارج الشجرة المعتمدة.
- الدفع المباشر إلى main دون مراجعة.

CH05 · ENVIRONMENT SETUP

# 05 تجهيز بيئة التطوير

من جهاز فارغ إلى Backend و Frontend يعملان ويتحدثان معاً

هذا الفصل عملي بالكامل. الهدف أن يصل كل عضو من جهاز فارغ إلى نظام يعمل: الـ Backend يرد على الطلبات، والـ Frontend يعرض البيانات، والاثنان يتحدثان معاً. نُقدّم أوامر كاملة قابلة للنسخ، بنسخة **Windows PowerShell** أولاً ثم بدائل **Linux / macOS**.

## 5.1 مفاهيم أساسية قبل البدء

- **Terminal:** نافذة نصية نكتب فيها الأوامر. في Windows نستخدم PowerShell، وفي Linux/macOS نستخدم Bash.
- **Working Directory:** المجلد الحالي الذي ينفّذ فيه الطرفية الأوامر. الأوامر النسبية تُحسب منه، لذا يجب التأكد من مكاننا قبل التنفيذ.
- **Port:** رقم يحدّد الخدمة على الجهاز. الـ Backend يعمل على 8000، والـ Frontend على 5173، وPostgreSQL على 5432. لا يمكن لخدمتين استخدام نفس المنفذ.
- **Environment Variable:** متغيّر يُخزَّن خارج الكود ليحمل الإعدادات والأسرار. نضعها في ملف .env ولا نرفعه إلى Git.

## 5.2 الأدوات المطلوبة والتحقق منها

**جدول 5-1: الأدوات وأمر التحقق من التثبيت**

| الأداة | الإصدار المستهدف | أمر التحقق |
|---|---|---|
| Python | 3.11 | `python --version` |
| Git | أحدث إصدار مستقر | `git --version` |
| Node.js + npm | Node 20 LTS أو أحدث | `node --version && npm --version` |
| PostgreSQL | 16 (أو Docker) | `psql --version` |
| Docker Desktop | أحدث إصدار | `docker --version` |
| VS Code | أحدث إصدار | يفتح من قائمة البرامج |

**تحذير**

إذا لم يظهر رقم إصدار بعد الأمر، فالأداة غير مثبّتة أو غير مضافة إلى PATH. أعد تثبيتها مع خيار "Add to PATH" قبل المتابعة.

## 5.3 الخطوات الاثنا عشر خطوة بخطوة

### الخطوة 1: إنشاء المستودع محلياً

terminal — powershell

```
$ git clone https://github.com/<YOUR_ORG>/safevision-ai.git
$ cd safevision-ai
```

### الخطوة 2: إنشاء البيئة الافتراضية

ننشئ venv في جذر المشروع لعزل مكتبات الـ Backend عن النظام.

terminal — powershell / bash

```
# Windows PowerShell
$ python -m venv .venv

# Linux / macOS
$ python3 -m venv .venv
```

### الخطوة 3: تنشيط البيئة الافتراضية

powershell / bash — terminal

```
# Windows PowerShell
$ .venv\Scripts\Activate.ps1
# If script execution is blocked, run this once for the current session:
$ Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

# Linux / macOS
$ source .venv/bin/activate
```

### الخطوة 4: تثبيت متطلبات الـ Backend

powershell / bash — terminal

```
$ cd backend
$ pip install --upgrade pip
$ pip install -r requirements.txt
```

### الخطوة 5: إنشاء تطبيق React بـ Vite و TypeScript

إن كان مجلد frontend فارغاً أنشئ فيه مشروع Vite بقالب TypeScript.

powershell / bash — terminal

```
$ cd ../frontend
$ npm create vite@latest . -- --template react-ts
$ npm install
```

### الخطوة 6: تثبيت متطلبات الـ Frontend

powershell / bash — terminal

```
$ npm install axios react-router-dom recharts
$ npm install -D tailwindcss postcss autoprefixer
$ npx tailwindcss init -p
```

### الخطوة 7: إعداد ملف .env

ننسخ القالب ثم نملأ القيم الحقيقية محلياً (كلمة مرور قاعدة البيانات، مسارات الـ checkpoints).

powershell / bash — terminal

```
# Windows PowerShell
$ Copy-Item .env.example .env

# Linux / macOS
$ cp .env.example .env
```

### الخطوة 8: تشغيل PostgreSQL

الطريقة الموصى بها عبر Docker Compose، والبديل تشغيل حاوية مباشرة.

powershell / bash — terminal

```
# Preferred: start the database service from docker-compose.yml
$ docker compose up -d db

# Alternative: run a single PostgreSQL 16 container
docker run --name safevision-db -d -p 5432:5432 ^
  -e POSTGRES_USER=<DB_USER> -e POSTGRES_PASSWORD=<DB_PASSWORD> ^
  -e POSTGRES_DB=safevision postgres:16
```

### الخطوة 9: تشغيل الـ Backend

powershell / bash — terminal

```
$ cd backend
$ uvicorn app.main:app --reload --port 8000
```

### الخطوة 10: تشغيل الـ Frontend

نفتح طرفية ثانية ونترك الأولى تعمل.

```powershell / bash
$ cd frontend
$ npm run dev
```

### الخطوة 11: فتح توثيق Swagger

افتح في المتصفح http://localhost:8000/docs لتظهر كل الـ Endpoints وتجربتها مباشرة.

### الخطوة 12: التأكد من التواصل

نتأكد أن الـ Backend يرد، وأن الـ Frontend يصل إليه عبر VITE_API_BASE_URL.

```powershell / bash
$ curl http://localhost:8000/api/v1/health
# expected: {"status":"ok"}
```

## 5.4 أخطاء الإعداد الشائعة وحلولها

**جدول 5-2: أخطاء شائعة عند الإعداد**

| الخطأ | السبب المحتمل | الحل |
|---|---|---|
| `python: command not found` | Python غير مثبّت أو غير مضاف لـ PATH | أعد التثبيت مع خيار Add to PATH |
| تنشيط venv مرفوض في PowerShell | سياسة تنفيذ السكربتات | `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` |
| `ModuleNotFoundError` | نسيان تنشيط venv أو تثبيت المتطلبات | نشّط .venv ثم `pip install -r requirements.txt` |
| فشل الاتصال بـ PostgreSQL | الحاوية لا تعمل أو المنفذ مشغول | تحقق بـ `docker ps` وتأكد من POSTGRES_PORT |
| خطأ CORS في المتصفح | الأصل غير مسموح به | أضف عنوان الـ Frontend إلى CORS_ORIGINS |
| المنفذ 8000 أو 5173 مشغول | خدمة أخرى تستخدم المنفذ | أغلق العملية أو غيّر رقم المنفذ |
| الـ Frontend لا يرى البيانات | VITE_API_BASE_URL خاطئ | تأكد أنه `http://localhost:8000/api/v1` ثم أعد تشغيل Vite |

### خلاصة الفصل 5

#### ما تعلّمناه

فهمنا الطرفية ومجلد العمل والمنافذ ومتغيّرات البيئة، وتحقّقنا من الأدوات الست، ونفّذنا الخطوات الاثنتي عشرة من الاستنساخ حتى تأكيد التواصل، وعرفنا حلول أكثر أخطاء الإعداد شيوعاً.

#### ما يجب تنفيذه

- تثبيت الأدوات الست والتحقق من إصداراتها.
- إنشاء .venv وتنشيطه وتثبيت المتطلبات.
- نسخ .env.example إلى .env وملء القيم.
- تشغيل PostgreSQL ثم الـ Backend ثم الـ Frontend.
- التأكد من رد /api/v1/health بلا أخطاء CORS.

#### المخرجات المتوقعة

- جهاز كل عضو يشغّل النظام كاملاً محلياً.
- Swagger يعمل على http://localhost:8000/docs.
- الـ Frontend يعرض بيانات قادمة من الـ Backend فعلاً.

#### كيف نتحقق من الإنجاز

- الأمر curl http://localhost:8000/api/v1/health يعيد status ok.
- لا يوجد خطأ CORS في وحدة تحكّم المتصفح.
- المنافذ 8000 و5173 و5432 مفتوحة وتعمل معاً.

### أخطاء شائعة

- تثبيت المكتبات في بيئة النظام بدل .venv.
- نسيان تشغيل قاعدة البيانات قبل الـ Backend.
- تعديل .env.example بقيم حقيقية بدل .env.
- إغلاق طرفية الـ Backend أثناء العمل على الـ Frontend.

# 06 معمارية الـ Backend بـ FastAPI

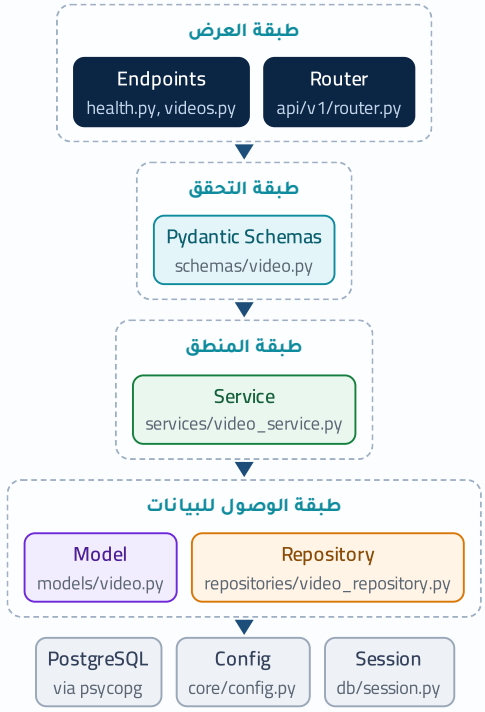
طبقات واضحة: Route ثم Schema ثم Model ثم Repository ثم Service

نضع طبقات واضحة تفصل بين استقبال الطلب ومنطق العمل والوصول لقاعدة البيانات، ونعرض ملفات مترابطة قابلة للتشغيل بأسماء متطابقة في كل مكان.

## 6.1 الطبقات ولماذا نفصلها

كل طبقة تعرف الطبقة التي تحتها فقط: الراوتر يستقبل الطلب، والـ **Service** يحمل منطق العمل، والـ **Repository** يتحدّث مع قاعدة البيانات، والـ **Model** يعرّف الجدول. هذا يجعل الكود قابلاً للاختبار وللعمل المتوازي.

## 6.2 مخطط معمارية الطبقات




**شكل 6-1:** معمارية طبقات الـ Backend وملفاتها الفعلية

## 6.3 نقطة الدخول والـ Routers

main.py نقطة الدخول: ينشئ تطبيق FastAPI، يضيف CORS، ويُدرج الراوتر الرئيسي تحت /api/v1.

python backend/app/main.py

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.api.v1.router import api_router
from app.core.config import settings

app = FastAPI(title=settings.APP_NAME, version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origin_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(api_router, prefix=settings.API_V1_PREFIX)


@app.get("/")
def root() → dict:
    return {"app": settings.APP_NAME, "docs": "/docs"}
```

الراوتر الرئيسي يجمع راوترات v1:

python backend/app/api/v1/router.py

```python
from fastapi import APIRouter

from app.api.v1.endpoints import health, videos

api_router = APIRouter()
api_router.include_router(health.router)
api_router.include_router(videos.router)
```

### 6.4 إدارة الإعدادات بـ pydantic-settings

كل الإعدادات تُقرأ من متغيّرات البيئة عبر كائن واحد بأسماء مطابقة لـ SPEC.md وقيم افتراضية محلية.

python backend/app/core/config.py

```python
from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    APP_NAME: str = "SafeVision AI"
    APP_ENV: str = "development"
    APP_DEBUG: bool = True
    API_V1_PREFIX: str = "/api/v1"
    AUTH_ENABLED: bool = False
    SECRET_KEY: str = "change-me"
    JWT_ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
    CORS_ORIGINS: str = "http://localhost:5173"

    POSTGRES_USER: str = "postgres"
    POSTGRES_PASSWORD: str = "postgres"
    POSTGRES_DB: str = "safevision"
    POSTGRES_HOST: str = "localhost"
    POSTGRES_PORT: int = 5432
    DATABASE_URL: str = "postgresql+psycopg://postgres:postgres@localhost:5432/safevision"

    STORAGE_BACKEND: str = "local"
    LOCAL_STORAGE_ROOT: str = "./storage"
    UPLOAD_DIR: str = "./storage/uploads"
    EVIDENCE_DIR: str = "./storage/evidence"
    REPORT_DIR: str = "./storage/reports"

    MAX_UPLOAD_MB: int = 500
    ALLOWED_VIDEO_EXTENSIONS: str = "mp4,avi,mov,mkv"
    UPLOAD_CHUNK_MB: int = 4

    @property
```

```
    def cors_origin_list(self) -> list[str]:
        return [o.strip() for o in self.CORS_ORIGINS.split(",") if o.strip()]

    @property
    def allowed_extension_set(self) -> set[str]:
        return {e.strip().lower() for e in self.ALLOWED_VIDEO_EXTENSIONS.split(",") if e.strip()}


@lru_cache
def get_settings() -> Settings:
    return Settings()


settings = get_settings()
```

## 6.5 إدارة الـ session

db/session.py ينشئ المحرّك ومصنع الجلسات، ويوفّر get_db التي تُعطي كل طلب جلسة ثم تُغلقها تلقائياً.

backend/app/db/session.py — python

```python
from collections.abc import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker

from app.core.config import settings

engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True, future=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, future=True)


def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

## 6.6 الـ Model: تعريف جدول videos

الـ **Model** صنف SQLAlchemy يمثّل جدولاً؛ كل حقل يطابق عموداً بنوع صريح عبر Mapped، وهذا ما يمكّن Alembic من توليد الـ migrations.

backend/app/models/video.py — python

```python
import uuid
from datetime import datetime

from sqlalchemy import BigInteger, DateTime, Integer, Numeric, String, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class Base(DeclarativeBase):
    pass


class Video(Base):
    __tablename__ = "videos"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    original_filename: Mapped[str] = mapped_column(String(255), nullable=False)
    stored_filename: Mapped[str] = mapped_column(String(255), nullable=False)
    storage_key: Mapped[str] = mapped_column(String(512), unique=True, nullable=False)
    content_type: Mapped[str] = mapped_column(String(100), nullable=False)
    size_bytes: Mapped[int] = mapped_column(BigInteger, nullable=False)
    duration_seconds: Mapped[float | None] = mapped_column(Numeric(10, 3), nullable=True)
    fps: Mapped[float | None] = mapped_column(Numeric(8, 3), nullable=True)
    width: Mapped[int | None] = mapped_column(Integer, nullable=True)
    height: Mapped[int | None] = mapped_column(Integer, nullable=True)
    frame_count: Mapped[int | None] = mapped_column(Integer, nullable=True)
    status: Mapped[str] = mapped_column(String(20), nullable=False, default="uploaded")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now()
    )
```

**ملاحظة**

لتبسيط الفصل عرّفنا Base هنا؛ في المشروع الكامل يُنقل إلى app/db/base.py.

## 6.7 الـ Schema: عقد الدخل والخرج

الـ **Schema** صنف Pydantic يحدّد شكل البيانات الداخلة والخارجة: VideoOut للردّ وVideoCreate للإدخال. الـ JobOut مضمّن هنا لأن VideoOut.latest_job يشير إليه (وفي المشروع الكامل يوضع في schemas/job.py).

backend/app/schemas/video.py

```python
import uuid
from datetime import datetime

from pydantic import BaseModel, ConfigDict


class VideoBase(BaseModel):
    original_filename: str
    content_type: str
    size_bytes: int


class VideoCreate(VideoBase):
    stored_filename: str
    storage_key: str


class JobOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: uuid.UUID
    video_id: uuid.UUID
    status: str
    progress: float
    frames_total: int | None = None
    frames_processed: int = 0
    sample_every_n_frames: int = 5
    error_message: str | None = None
    created_at: datetime


class VideoOut(VideoBase):
    model_config = ConfigDict(from_attributes=True)

    id: uuid.UUID
    stored_filename: str
    storage_key: str
    duration_seconds: float | None = None
    fps: float | None = None
    width: int | None = None
    height: int | None = None
    frame_count: int | None = None
    status: str
    created_at: datetime
    updated_at: datetime
    latest_job: JobOut | None = None
```

## 6.8 طبقة الـ Repository

الـ **Repository** هو المكان الوحيد الذي يكتب استعلامات قاعدة البيانات؛ الـ Service يستدعيه ولا يكتب SQL بنفسه.

backend/app/repositories/video_repository.py

```python
import uuid

from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.models.video import Video


class VideoRepository:
    def __init__(self, db: Session) -> None:
        self.db = db

    def add(self, video: Video) -> Video:
```

```
        self.db.add(video)
        self.db.commit()
        self.db.refresh(video)
        return video

    def get(self, video_id: uuid.UUID) → Video | None:
        return self.db.get(Video, video_id)

    def list(self, offset: int = 0, limit: int = 20) → list[Video]:
        stmt = select(Video).order_by(Video.created_at.desc()).offset(offset).limit(limit)
        return list(self.db.scalars(stmt).all())

    def count(self) → int:
        return self.db.scalar(select(func.count()).select_from(Video)) or 0

    def delete(self, video: Video) → None:
        self.db.delete(video)
        self.db.commit()
```

## 6.9 طبقة الـ Service

الـ **Service** يحمل منطق العمل: يتحقق من الامتداد، يبني كائن الـ Model، ويستدعي الـ Repository؛ لا يعرف الـ HTTP ولا الاستعلامات.

python **backend/app/services/video_service.py**

```python
import uuid

from sqlalchemy.orm import Session

from app.core.config import settings
from app.models.video import Video
from app.repositories.video_repository import VideoRepository
from app.schemas.video import VideoCreate


class VideoService:
    def __init__(self, db: Session) → None:
        self.repository = VideoRepository(db)

    def is_extension_allowed(self, filename: str) → bool:
        ext = filename.rsplit(".", 1)[-1].lower() if "." in filename else ""
        return ext in settings.allowed_extension_set

    def create_video(self, payload: VideoCreate) → Video:
        video = Video(
            original_filename=payload.original_filename,
            stored_filename=payload.stored_filename,
            storage_key=payload.storage_key,
            content_type=payload.content_type,
            size_bytes=payload.size_bytes,
            status="uploaded",
        )
        return self.repository.add(video)

    def get_video(self, video_id: uuid.UUID) → Video | None:
        return self.repository.get(video_id)

    def list_videos(self, page: int = 1, page_size: int = 20) → list[Video]:
        offset = (page - 1) * page_size
        return self.repository.list(offset=offset, limit=page_size)
```

## 6.10 الـ Endpoints

كل ملف في api/v1/endpoints/ يعرّف راوتراً صغيراً يستقبل الطلب، يتحقق من الصحة، يستدعي الـ Service، ويرمي أخطاء HTTP عند الحاجة.

python backend/app/api/v1/endpoints/health.py

```python
from fastapi import APIRouter, Depends
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.db.session import get_db

router = APIRouter(tags=["health"])


@router.get("/health")
def health(db: Session = Depends(get_db)) → dict:
    db_ok = True
    try:
        db.execute(text("SELECT 1"))
    except Exception:
        db_ok = False
    return {"status": "ok" if db_ok else "degraded", "database": db_ok}
```

python backend/app/api/v1/endpoints/videos.py

```python
import uuid

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.schemas.video import VideoOut
from app.services.video_service import VideoService

router = APIRouter(prefix="/videos", tags=["videos"])


@router.get("", response_model=list[VideoOut])
def list_videos(page: int = 1, page_size: int = 20, db: Session = Depends(get_db)):
    service = VideoService(db)
    return service.list_videos(page=page, page_size=page_size)


@router.get("/{video_id}", response_model=VideoOut)
def get_video(video_id: uuid.UUID, db: Session = Depends(get_db)):
    service = VideoService(db)
    video = service.get_video(video_id)
    if video is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Video not found")
    return video
```

المسارات الناتجة: /api/v1/health و api/v1/videos/ و api/v1/videos/{video_id}/.

## 6.11 Dependency Injection والأخطاء و Logging و CORS

- **Dependency Injection:** تمرير التبعيات عبر Depends كما في get_db، فيحصل كل طلب على جلسة مستقلة تُغلق تلقائياً.
- **معالجة الأخطاء:** HTTPException بحالات واضحة (404، 400، 409)، ومعالج عام للأخطاء غير المتوقّعة دون كشف التفاصيل الحسّاسة.
- **Logging:** سجل موحّد يطبع المسار والحالة وزمن التنفيذ، ومستواه من APP_DEBUG.
- **CORS:** نسمح بأصل الـ Frontend فقط عبر CORS_ORIGINS، وهي أشهر مشكلة عند بداية الربط.

## 6.12 التحقق من الملفات والمصادقة وإصدار الـ API

- **التحقق من الملف المرفوع:** الامتداد عبر allowed_extension_set والحجم عبر MAX_UPLOAD_MB، مع 413 للملف الأكبر و422 للنوع غير المسموح.
- **Authentication / Authorization:** خلف علم AUTH_ENABLED. عند تفعيله نتحقق من الـ JWT عبر PyJWT ونضبط الصلاحيات حسب user_role. في وضع العرض التوضيحي يكون AUTH_ENABLED=false.
- **API versioning:** كل المسارات تحت /api/v1 عبر API_V1_PREFIX، فإضافة v2 لاحقاً لا تكسر العملاء.

## 6.13 الفرق بين الطبقات وترتيب التنفيذ

**جدول 6-1: مسؤولية كل طبقة**

| الطبقة | مسؤوليتها | ما لا تفعله |
|---|---|---|

| الطبقة | مسؤوليتها | ما لا تفعله |
|---|---|---|
| Route | استقبال الطلب، التحقق من الصحة، إعادة الرد | لا تكتب منطق عمل ولا SQL |
| Schema | تحديد شكل البيانات الداخلة والخارجة | لا تتحدّث مع قاعدة البيانات |
| Model | تعريف الجدول والأعمدة والعلاقات | لا تحتوي منطق عمل |
| Repository | كل استعلامات قاعدة البيانات | لا تقرّر قواعد السلامة |
| Service | منطق العمل وقواعد القرار | لا تعرف HTTP ولا SQL |
| Worker | تشغيل المعالجة الطويلة خارج الطلب | لا تخدم طلبات HTTP |

ترتيب التنفيذ عند بناء وحدة جديدة: **Model** ثم **Schema** ثم **Repository** ثم **Service** ثم **Endpoint**، وأخيراً الـ **Worker** عند الحاجة.

## 6.14 متطلبات الـ Backend

المكتبات الأساسية؛ نثبّت إصداراتها النهائية بعد التحقق من المصادر الرسمية (الفصل الثالث).

backend/requirements.txt — text

```
fastapi
uvicorn[standard]
pydantic
pydantic-settings
sqlalchemy
alembic
psycopg[binary]
python-multipart
pyjwt
bcrypt
opencv-python
numpy
pandas
torch
ultralytics
```

**تشغيل سريع**

نشغّل الـ Backend بأمر uvicorn app.main:app --reload، ثم نفتح http://localhost:8000/docs للتأكد من ظهور الـ endpoints.

### خلاصة الفصل 6

**ما تعلّمناه**

بنينا الـ Backend بطبقات واضحة مترابطة: main.py router.pyg endpointsg للعرض، config.pyg session.pyg للبنية، models/video.pyg schemas/video.pyg للبيانات، video_repository.pyg video_service.pyg للمنطق.

**ما يجب تنفيذه**

- إنشاء الملفات بنفس الأسماء والاستيرادات أعلاه.
- تشغيل uvicorn app.main:app --reload بلا أخطاء استيراد.
- التأكد من رد /api/v1/health بحالة سليمة.
- إضافة باقي الوحدات (jobs, incidents, analytics, reports) بنفس النمط.

**المخرجات المتوقعة**

- Backend يعمل Swaggerg يعرض كل المسارات.
- طبقات منفصلة بلا استيرادات دائرية.
- ملف requirements.txt قابل للتثبيت على أي جهاز.

**كيف نتحقق من الإنجاز**

- لا يستورد أي endpoint قاعدة البيانات خارج الـ Repository.
- لا استعلامات SQL داخل الـ endpoints.
- كل ملف يستورد أسماء موجودة فعلاً، ولا أخطاء ImportError.

### أخطاء شائعة

- كتابة منطق العمل داخل الـ endpoint بدل الـ Service.
- نسيان from_attributes=True فيردّ الـ Schema خطأ تحويل.
- إرجاع كائن SQLAlchemy بدون response_model.
- نسيان إغلاق الجلسة، أو مشاركتها بين الطلبات.
- إغفال CORS فتظهر أخطاء في المتصفح فقط.

CH07 · API CONTRACT

# 07 تصميم الـ API وتوثيق الـ Endpoints

عقد واحد يتفق عليه الـ Backend والـ Frontend قبل كتابة الكود

عقد الـ API (**API contract**) هو الاتفاق الملزم بين الـ Backend والـ Frontend: أسماء المسارات، وأشكال الطلب والرد، ورموز الحالة، وأشكال الخطأ. تثبيته أولاً يسمح ببناء الطرفين بالتوازي، وهو المرجع الحرفي لكل نداء في الفصول التالية.

## 7.1 لماذا نكتب العقد أولاً

البناء التسلسلي (Backend ثم Frontend) يضيّع أسابيع؛ البديل أن نتفق على الـ Endpoints والأشكال مكتوبةً فيبدأ الطرفان معاً. القاعدة الحاكمة: **أي تغيير في العقد يُعلَن ويُوثَّق قبل تنفيذه**، ولا يُغيَّر اسم حقل بصمت. كل المسارات تبدأ بـ /api/v1 على http://localhost:8000، وكل الأجسام **JSON** إلا الرفع.

## 7.2 جدول الـ Endpoints الكامل

الجدول التالي هو المصدر الوحيد للحقيقة. الرمز yes* يعني أن المصادقة تُفرض فقط عند AUTH_ENABLED=true؛ في وضع العرض (false) يعمل النظام بمستخدم افتراضي دوره safety_officer.

**جدول 7-1: عقد الـ REST API الكامل (البادئة /api/v1)**

| Method | Endpoint | Purpose | Request | Response | Auth | Errors |
|---|---|---|---|---|---|---|
| GET | /health | liveness + component status | — | 200 | no | 500 |
| POST | /videos | upload a video (multipart/form-data) | file | VideoOut 201 | *yes | 400 413 415 422 409 |
| GET | /videos | paginated video list | page, filters | envelope 200 | *yes | 422 |
| GET | /videos/{video_id} | one video + latest job | path id | VideoOut 200 | *yes | 404 |
| DELETE | /videos/{video_id} | delete video + files (safe delete) | path id | 204 | *yes | 404 409 |
| POST | /videos/{video_id}/process | create a processing job | path id | JobOut 202 | *yes | 404 409 |
| GET | /jobs/{job_id} | job status + progress | path id | JobOut 200 | *yes | 404 |
| GET | /jobs | list jobs (filter by status/video) | page, filters | envelope 200 | *yes | 422 |
| GET | /incidents | paginated + filtered incident list | page, filters | envelope 200 | *yes | 422 |
| GET | /incidents/{incident_id} | incident detail | path id | IncidentOut 200 | *yes | 404 |
| PATCH | /incidents/{incident_id} | review/status change | IncidentPatch | IncidentOut 200 | *yes | 404 409 422 |
| GET | /incidents/{incident_id}/evidence | evidence list | path id | 200 []EvidenceOut | *yes | 404 |
| GET | /evidence/{evidence_id}/file | stream evidence image/clip | path id | binary 200 | *yes | 404 |
| GET | /analytics/overview | dashboard KPIs | — | 200 AnalyticsOverview | *yes | 422 |
| GET | /analytics/timeseries | incidents per day | date range | 200 []TimeseriesPoint | *yes | 422 |
| GET | /reports/daily | daily summary rows | date range | rows 200 | *yes | 422 |
| GET | /reports/export | CSV export (text/csv) | filters | CSV 200 | *yes | 422 |

| Method | Endpoint | Purpose | Request | Response | Auth | Errors |
|---|---|---|---|---|---|---|
| POST | /auth/login | obtain JWT | email, password | token 200 | no | 401 422 |
| GET | /auth/me | current user | Bearer token | UserOut 200 | yes | 401 |

## 7.3 رموز الحالة (Status Codes)

نستخدم رموز HTTP القياسية ولا نعيد 200 عند الفشل:

**جدول 7-2: رموز الحالة المستخدمة ومعانيها**

| Code | المعنى | متى يظهر في نظامنا |
|---|---|---|
| 200 | OK | قراءة ناجحة، أو PATCH نجح وأعاد الواقعة المحدّثة |
| 201 | Created | بعد رفع فيديو ناجح، نعيد سجل VideoOut |
| 202 | Accepted | إنشاء processing job؛ الطلب قُبل والمعالجة غير متزامنة |
| 204 | No Content | حذف فيديو ناجح؛ لا نعيد جسماً |
| 400 | Bad Request | طلب مشوّه أو ملف فارغ |
| 401 | Unauthorized | توكن مفقود أو منتهٍ عندما AUTH_ENABLED=true |
| 403 | Forbidden | المستخدم موجود لكن دوره لا يسمح بالإجراء |
| 404 | Not Found | معرّف video/job/incident غير موجود |
| 409 | Conflict | يوجد job نشط لنفس الفيديو، أو انتقال حالة غير مسموح |
| 413 | Payload Too Large | حجم الفيديو يتجاوز MAX_UPLOAD_MB |
| 415 | Unsupported Media Type | امتداد أو نوع محتوى غير مسموح |
| 422 | Validation Error | فشل تحقق Pydantic في جسم أو معاملات الاستعلام |
| 500 | Server Error | خطأ غير متوقع؛ يُسجَّل في الـ logs دون كشف تفاصيل داخلية |

## 7.4 أمثلة JSON حقيقية

هذه الأشكال مطابقة حرفياً لـ SPEC.md وتُستنسخ في أنواع TypeScript بنفس الأسماء. رد الرفع/القراءة VideoOut:

GET /api/v1/videos/{video_id} — 200 OK

```json
{
  "id": "3f1a9c2e-0b7d-4e51-9a44-77d2c1b0e8f3",
  "original_filename": "site_a_cam1.mp4",
  "stored_filename": "8f3c21a0b4d94f2ea1.mp4",
  "content_type": "video/mp4",
  "size_bytes": 184320000,
  "duration_seconds": 300.5,
  "fps": 25.0,
  "width": 1920,
  "height": 1080,
  "frame_count": 7512,
  "status": "processing",
  "created_at": "2026-01-12T08:30:00Z",
  "updated_at": "2026-01-12T08:31:10Z",
  "latest_job": {
    "id": "c7b2e4d1-9a3f-4c88-b0e1-5d9f2a6c7b10",
    "video_id": "3f1a9c2e-0b7d-4e51-9a44-77d2c1b0e8f3",
    "status": "running",
    "progress": 42.5,
    "frames_total": 7512,
    "frames_processed": 3193,
    "sample_every_n_frames": 5,
    "started_at": "2026-01-12T08:30:20Z",
    "finished_at": null,
    "error_message": null,
    "created_at": "2026-01-12T08:30:15Z"
  }
}
```

IncidentOut يحمل مصفوفة evidence، وكل عنصر فيه file_url جاهز للعرض:

GET /api/v1/incidents/{incident_id} — 200 OK

```json
```

```
{
  "id": "9d4c1b70-2e6a-4f13-8c55-1a2b3c4d5e6f",
  "video_id": "3f1a9c2e-0b7d-4e51-9a44-77d2c1b0e8f3",
  "job_id": "c7b2e4d1-9a3f-4c88-b0e1-5d9f2a6c7b10",
  "incident_type": "ppe_violation",
  "severity": "high",
  "status": "pending_review",
  "started_at_seconds": 61.2,
  "ended_at_seconds": 66.8,
  "frame_number": 1530,
  "track_id": 7,
  "confidence": 0.91,
  "class_name": "person_no_helmet",
  "zone_name": null,
  "summary": "Worker without helmet detected for 5.6s",
  "detection_count": 12,
  "created_at": "2026-01-12T08:35:00Z",
  "updated_at": "2026-01-12T08:35:00Z",
  "evidence": [
    {
      "id": "a1b2c3d4-e5f6-4a7b-8c9d-0e1f2a3b4c5d",
      "incident_id": "9d4c1b70-2e6a-4f13-8c55-1a2b3c4d5e6f",
      "kind": "image",
      "storage_key": "evidence/9d4c1b70/frame_1530.jpg",
      "content_type": "image/jpeg",
      "size_bytes": 284512,
      "frame_number": 1530,
      "timestamp_seconds": 61.2,
      "file_url": "/api/v1/evidence/a1b2c3d4-e5f6-4a7b-8c9d-0e1f2a3b4c5d/file"
    }
  ]
}
```

الردود الإحصائية تُبنى من صفوف مخزَّنة، وكائنات التوزيع مفاتيحها قيم الـ enum:

json — GET /api/v1/analytics/overview — 200 OK

```
{
  "total_videos": 14,
  "processed_videos": 11,
  "total_incidents": 37,
  "incidents_by_type": { "ppe_violation": 24, "fire_smoke": 3, "machine_proximity": 10 },
  "incidents_by_severity": { "low": 9, "medium": 15, "high": 11, "critical": 2 },
  "incidents_by_status": { "detected": 5, "pending_review": 12, "confirmed": 14, "rejected": 4, "closed": 2 },
  "pending_review": 12,
  "jobs_running": 1,
  "jobs_failed": 2,
  "last_incident_at": "2026-01-12T08:35:00Z"
}
```

## 7.5 Pagination و Filtering و Sorting و Date-range

كل مسارات القوائم تعيد غلاف الصفحات نفسه (`pagination envelope`)؛ الحد الأقصى لـ page_size هو 100.

json — GET /api/v1/incidents?page=2&page_size=20&incident_type=ppe_violation&sort=-created_at — 200 OK

```
{
  "items": [ { "id": "9d4c1b70-...", "incident_type": "ppe_violation", "severity": "high" } ],
  "total": 24,
  "page": 2,
  "page_size": 20,
  "pages": 2
}
```

**جدول 7-3: معاملات الاستعلام المشتركة لكل مسارات القوائم**

| المعامل | النوع | المعنى |
|---|---|---|
| `page` | int, ≥1 | رقم الصفحة، يبدأ من 1 |
| `page_size` | int, 1..100 | عدد العناصر في الصفحة، الافتراضي 20 |
| `sort` | string | حقل الترتيب؛ إشارة - تعني تنازلياً مثل `-created_at` |
| `date_from / date_to` | YYYY-MM-DD | نطاق زمني مغلق على `created_at` |
| `incident_type` | enum | فلترة بنوع الواقعة |
| `severity` | enum | فلترة بدرجة الخطورة |

| المعامل | النوع | المعنى |
|---|---|---|
| status | enum | فلترة بحالة المراجعة |
| video_id | UUID | وقائع فيديو واحد بعينه |

نكتب معاملات الترقيم مرة واحدة في مخطط **Pydantic** قابل لإعادة الاستخدام:

backend/app/schemas/common.py — python

```python
from typing import Generic, TypeVar

from pydantic import BaseModel, Field

T = TypeVar("T")


class PageParams(BaseModel):
    page: int = Field(default=1, ge=1)
    page_size: int = Field(default=20, ge=1, le=100)
    sort: str = "-created_at"


class Paginated(BaseModel, Generic[T]):
    items: list[T]
    total: int = 0
    page: int = 1
    page_size: int = 20
    pages: int = 0
```

## 7.6 إبقاء العقد مستقراً أثناء العمل المتوازي

نحمي العقد بخمس أدوات:

- **ملف `OpenAPI`:** يولّده FastAPI تلقائياً على /openapi.json؛ نصدّره إلى docs/openapi.json بعد كل تعديل مقصود.
- **الإصدارات (`versioning`):** أي تغيير كاسر يذهب إلى /api/v2، ولا نحذف حقلاً من v1 إلا بعد إعلان.
- **خادم وهمي (`mock server`):** يبني الـ Frontend على استجابات جاهزة من الـ OpenAPI قبل جهوزية الـ Backend.
- **أنواع مشتركة:** ملف frontend/src/types/api.ts (الفصل 14) هو النسخة اليدوية المطابقة للـ VideoOut وغيره.
- **مراجعة العقد:** أي PATCH على المسار يمرّ على مسؤول الـ Backend ومسؤول الـ Frontend معاً قبل الدمج.

> **نصيحة عملية**
>
> جرّب الـ Endpoints يدوياً على /`Swagger UI` (docs) قبل كتابة كود الواجهة.

مثال من الطرفية:

curl — create a processing job and poll it — bash

```bash
$ curl -X POST http://localhost:8000/api/v1/videos/<video_id>/process
# HTTP/1.1 202 Accepted

$ curl http://localhost:8000/api/v1/jobs/<job_id>
# HTTP/1.1 200 OK  {"status":"running","progress":42.5}
```

## 7.7 أخطاء الـ API الموحّدة

شكل خطأ واحد لكل الـ endpoints: رمز آلي ورسالة صديقة، فلا تفحص الواجهة نص الخطأ.

Error response shape — 404 / 409 / 413 — json

```json
{
  "detail": "Video not found",
  "code": "video_not_found"
}
```

أخطاء 422 تأتي من **Pydantic** بـ detail كمصفوفة لكل حقل مرفوض، ونحوّل loc إلى رسالة عربية في الواجهة.

**خلاصة الفصل 7**

### ما تعلّمناه

ثبّتنا جدول الـ Endpoints ورموز الحالة، وأشكال كائنات الرد وغلاف الصفحات، وأدوات إبقاء العقد مستقراً أثناء البناء المتوازي.

### ما يجب تنفيذه

- تثبيت جدول 7-1 في docs/API.md كمرجع ملزم للفريقين.
- تصدير openapi.json إلى docs/ بعد كل تغيير.
- كتابة مخطط Paginated[T] في الـ Backend ونظيره في الـ Frontend.
- توحيد شكل الخطأ عبر exception handler واحد.

### المخرجات المتوقعة

- ملف docs/API.md يطابق SPEC حرفياً.
- Swagger يعمل على /docs ويغطي كل المسارات.
- أمثلة JSON قابلة للنسخ في الفصول التالية.

### كيف نتحقق من الإنجاز

- كل مسار في الجدول له رمز نجاح ورموز خطأ محدّدة.
- لا يوجد endpoint يعيد 200 عند الفشل.
- أنواع الـ Frontend تطابق أسماء الحقول دون اختلاف حرف واحد.

### أخطاء شائعة

- تغيير اسم حقل بصمت فيكسر شاشة جاهزة.
- إعادة أشكال خطأ مختلفة من كل مسار.
- الخلط بين 202 (قبول غير متزامن) و201 (إنشاء مورد).
- تجاهل حد page_size الأقصى فيثقل الخادم.

CH08 · DATABASE DESIGN

# 08 تصميم قاعدة البيانات مع PostgreSQL

الجداول والعلاقات والفهارس والمهاجرات: مخطط واحد يوافق عليه الفريق كله

قاعدة البيانات هي الذاكرة طويلة المدى للنظام. في هذا الفصل نُثبّت مخططاً واحداً (schema) ونكتب أول migration تنشئه في PostgreSQL، فيبني أي عضو القاعدة بأمر واحد.

## 8.1 مبادئ التصميم التي نلتزم بها

ستة مبادئ تحكم كل قرار تصميمي في هذا الفصل.

**جدول 8-1: مبادئ تصميم قاعدة البيانات**

| المبدأ | المعنى العملي |
|---|---|
| المفتاح الأساسي Primary Key | لكل جدول عمود id فريد لا يتغيّر. الجداول "الكبيرة" (مثل videos و incidents) تستخدم UUID لأنها تظهر في مسارات الـ API، وجداول الأحداث الكثيفة (tracks، detections) تستخدم BIGSERIAL لأن كتابتها سريعة وترقيمها المتزايد لا يُعرض للمستخدم. |
| المفتاح الأجنبي Foreign Key | كل علاقة بين جدولين تُعبَّر عنها بمفتاح أجنبي صريح مع قاعدة حذف واضحة، حتى يمنع PostgreSQL البيانات المعلّقة (orphan rows). |
| الفهرس Index | لا نضيف فهرساً "للتأمين"، بل لكل استعلام نعرف أننا سنستخدمه فعلاً في الفلترة أو الترتيب. |
| الطوابع الزمنية Timestamps | كل جدول مهم يحمل created_at، والجداول القابلة للتعديل تحمل أيضاً updated_at. النوع دائماً TIMESTAMPTZ مع المنطقة الزمنية. |
| القيد Constraint | القيم المحدودة (الحالات، الأنواع، الأدوار) تُخزَّن كـ enum أصلي، فلا يمكن إدخال حالة غير معروفة حتى لو أخطأ الكود. |
| لا تخزين بايناري | الفيديو والصور لا تُخزَّن داخل PostgreSQL؛ نخزّن البيانات الوصفية (metadata) والمفتاح storage_key فقط. |

## 8.2 الـ Enums: القيم المسموح بها فقط

ثمانية أنواع تعدادية (enum) بدل النصوص الحرة، فلا يمكن كتابة حالة غير معروفة. القيم التالية حرفية ومطابقة لعقد الـ API في الفصل 7.

**جدول 8-2: أنواع الـ Enums وقيمها**

| الـ Enum | القيم المسموح بها |
|---|---|
| video_status | uploaded \| processing \| processed \| failed |
| job_status | queued \| running \| completed \| failed \| cancelled |
| incident_type | ppe_violation \| fire_smoke \| machine_proximity |
| incident_severity | low \| medium \| high \| critical |
| incident_status | detected \| pending_review \| confirmed \| rejected \| closed |
| evidence_kind | image \| clip \| metadata |
| review_action | confirm \| reject \| close \| comment |
| user_role | admin \| safety_officer \| viewer |

python backend/app/models/enums.py

```python
# backend/app/models/enums.py
import enum


class VideoStatus(str, enum.Enum):
    uploaded = "uploaded"
    processing = "processing"
    processed = "processed"
    failed = "failed"


class JobStatus(str, enum.Enum):
    queued = "queued"
    running = "running"
    completed = "completed"
    failed = "failed"
    cancelled = "cancelled"


class IncidentType(str, enum.Enum):
    ppe_violation = "ppe_violation"
    fire_smoke = "fire_smoke"
    machine_proximity = "machine_proximity"


class IncidentSeverity(str, enum.Enum):
    low = "low"
    medium = "medium"
    high = "high"
    critical = "critical"


class IncidentStatus(str, enum.Enum):
    detected = "detected"
    pending_review = "pending_review"
    confirmed = "confirmed"
    rejected = "rejected"
    closed = "closed"


class EvidenceKind(str, enum.Enum):
    image = "image"
    clip = "clip"
    metadata = "metadata"


class ReviewAction(str, enum.Enum):
    confirm = "confirm"
    reject = "reject"
    close = "close"
    comment = "comment"


class UserRole(str, enum.Enum):
    admin = "admin"
    safety_officer = "safety_officer"
    viewer = "viewer"
```

## 8.3 الجداول الأساسية في الـ MVP

جداول المرحلة الأولى: العمود والنوع والمفتاح وقابلية NULL والغرض، بأسماء حرفية.

### جدول users

يُنشأ فقط عند AUTH_ENABLED=true؛ وإلا يتصرّف النظام كأن safety_officer افتراضي مسجَّل دخوله. كلمة المرور تُخزَّن كتجزئة **bcrypt**.

**جدول 8-3: أعمدة users**

| العمود | النوع | المفتاح | NULL | الغرض |
|---|---|---|---|---|
| id | UUID | PK | لا | المعرّف الأساسي |
| email | VARCHAR(255) | unique | لا | اسم الدخول |
| full_name | VARCHAR(150) | | لا | الاسم المعروض |
| hashed_password | VARCHAR(255) | | لا | تجزئة bcrypt |
| role | user_role | | لا | الافتراضي safety_officer |

| العمود | النوع | المفتاح | NULL | الغرض |
|---|---|---|---|---|
| is_active | BOOLEAN | | لا | الافتراضي true |
| created_at / updated_at | TIMESTAMPTZ | | لا | طوابع التدقيق |

### جدول videos MVP

صف واحد لكل فيديو مرفوع؛ الملف نفسه ليس هنا بل مفتاحه storage_key (الفصل 9). أعمدة duration_seconds و fps و frame_count تُملأ بعد التحقق فتُقبل NULL عند الرفع.

**جدول 8-4: أعمدة videos**

| العمود | النوع | المفتاح | NULL | الغرض |
|---|---|---|---|---|
| id | UUID | PK | لا | المعرّف الأساسي |
| camera_id | UUID | FK → cameras.id | نعم | كاميرا المصدر (اختياري في الـ MVP) |
| original_filename | VARCHAR(255) | | لا | الاسم كما رفعه المستخدم |
| stored_filename | VARCHAR(255) | | لا | اسم آمن وفريد على القرص |
| storage_key | VARCHAR(512) | unique | لا | المسار النسبي / مفتاح الكائن |
| content_type | VARCHAR(100) | | لا | مثل video/mp4 |
| size_bytes | BIGINT | | لا | حجم الملف |
| duration_seconds | NUMERIC(10,3) | | نعم | يُملأ بعد التحقق |
| fps | NUMERIC(8,3) | | نعم | معدل الإطارات |
| width / height | INTEGER | | نعم | أبعاد الإطار |
| frame_count | INTEGER | | نعم | عدد الإطارات |
| checksum_sha256 | CHAR(64) | index | نعم | كشف الملفات المكرّرة |
| status | video_status | | لا | الافتراضي uploaded |
| uploaded_by | UUID | FK → users.id | نعم | رافع الملف |
| created_at / updated_at | TIMESTAMPTZ | | لا | طوابع زمنية |

### جدول processing_jobs MVP

صف لكل تشغيل لخط المعالجة، تُتابَع فيه الحالة والتقدّم. الفهرس الفريد الجزئي يمنع وظيفتين نشطتين لنفس الفيديو.

**جدول 8-5: أعمدة processing_jobs**

| العمود | النوع | المفتاح | NULL | الغرض |
|---|---|---|---|---|
| id | UUID | PK | لا | المعرّف الأساسي |
| video_id | UUID | FK → videos.id | لا | ON DELETE CASCADE |
| status | job_status | | لا | الافتراضي queued |
| progress | NUMERIC(5,2) | | لا | الافتراضي 0 |
| frames_total | INTEGER | | نعم | إجمالي الإطارات |
| frames_processed | INTEGER | | لا | الافتراضي 0 |
| sample_every_n_frames | INTEGER | | لا | الافتراضي 5 |
| model_bundle_version | VARCHAR(80) | | نعم | إصدار حزمة الـ Models المستخدمة |
| error_message | TEXT | | نعم | سبب الفشل عند الحاجة |
| worker_id | VARCHAR(80) | | نعم | العامل الذي نفّذ الوظيفة |
| started_at / finished_at | TIMESTAMPTZ | | نعم | زمن البداية والنهاية |
| created_at | TIMESTAMPTZ | | لا | زمن الإنشاء |

> **الفهرس الفريد الجزئي**
>
> القيد UNIQUE (video_id) WHERE status IN ('queued','running') هو خط الدفاع الأول ضد المعالجة المزدوجة (تفصيلاً في الفصل 11).

### جدول detections MVP

كل كشف خام: الصندوق والثقة والإطار والزمن واسم الـ Model وإصداره. أسرع الجداول نمواً، فنتعامل معه بحذر (القسم 8.7).

**جدول 8-6: أعمدة detections**

| العمود | النوع | المفتاح | الغرض |
|---|---|---|---|
| id | BIGSERIAL | PK | معرّف متزايد سريع الكتابة |
| job_id | UUID | FK → processing_jobs.id | ON DELETE CASCADE |
| video_id | UUID | FK → videos.id | الرجوع السريع للفيديو |
| frame_number | INTEGER | | رقم الإطار (لا يقبل NULL) |
| timestamp_seconds | NUMERIC(10,3) | | الزمن داخل الفيديو بالثواني |
| class_name | VARCHAR(80) | | اسم الصنف بعد التوحيد |
| confidence | NUMERIC(5,4) | | درجة الثقة بين 0 و 1 |
| bbox_x1 / bbox_y1 / bbox_x2 / bbox_y2 | INTEGER | | إحداثيات الصندوق بالبكسل |
| track_id | INTEGER | | معرّف التتبّع إن وُجد |
| model_name / model_version | VARCHAR | | مصدر الكشف وإصداره |
| created_at | TIMESTAMPTZ | | زمن الكتابة |

الفهارس: (job_id, frame_number) و (class_name) و (video_id).

### جدول tracks ▫▫ ▫▫▫▫

ملخّص لكل مسار تتبّع (track) بأول وآخر إطار وعدد الظهور، بدل تخزين كل إطار. القيد UNIQUE (job_id, track_id) يمنع التكرار؛ التتبّع عبر ByteTrack (الفصل 10).

### جدول incidents MVP

الجدول الذي يقرأه مسؤول السلامة: واقعة بنوع وخطورة وحالة، وزمن بداية ونهاية. الافتراضي detected و medium (الفصلان 12 و 13).

**جدول 8-7: أعمدة incidents**

| العمود | النوع | المفتاح | الغرض |
|---|---|---|---|
| id | UUID | PK | المعرّف الأساسي |
| video_id | UUID | FK → videos.id | لا يقبل NULL |
| job_id | UUID | FK → processing_jobs.id | الوظيفة المولّدة |
| incident_type | incident_type | | لا يقبل NULL |
| severity | incident_severity | | الافتراضي medium |
| status | incident_status | | الافتراضي detected |
| started_at_seconds | NUMERIC(10,3) | | بداية الواقعة |
| ended_at_seconds | NUMERIC(10,3) | | نهاية الواقعة |
| frame_number | INTEGER | | الإطار المرجعي |
| track_id | INTEGER | | الشخص أو الكائن المعني |
| confidence | NUMERIC(5,4) | | أعلى ثقة مسجّلة |
| class_name | VARCHAR(80) | | الصنف المرتبط |
| zone_name | VARCHAR(120) | | اسم منطقة الخطر |
| summary | TEXT | | وصف نصي مختصر |
| detection_count | INTEGER | | عدد الكشوفات، الافتراضي 1 |
| created_at / updated_at | TIMESTAMPTZ | | طوابع زمنية |

الفهارس: (video_id) و (incident_type) و (severity) و (status) و (created_at) — تخدم فلاتر شاشة الـ Incidents.

### جدول evidence MVP

دليل الواقعة: صورة أو مقطع أو بيانات وصفية. الملف في طبقة التخزين، وهنا storage_key ونوع المحتوى والحجم.

**جدول 8-8: أعمدة evidence**

| العمود | النوع | الغرض |
|---|---|---|
| id | UUID PK | المعرّف الأساسي |

| العمود | النوع | الغرض |
|---|---|---|
| incident_id | UUID FK → incidents.id | ON DELETE CASCADE |
| kind | evidence_kind | image / clip / metadata |
| storage_key | VARCHAR(512) | مفتاح الوصول للملف |
| content_type | VARCHAR(100) | مثل image/jpeg |
| size_bytes | BIGINT | حجم الملف |
| frame_number | INTEGER | إطار الاقتطاع |
| timestamp_seconds | NUMERIC(10,3) | زمن الاقتطاع |
| created_at | TIMESTAMPTZ | زمن الإنشاء |

### جدول incident_reviews MVP

سجل تدقيق (audit log) لا يُعدَّل ولا يُحذَف: كل إجراء يُلحق صفاً بالحالة السابقة والجديدة والملاحظة (الفصل 13).

**جدول 8-9: أعمدة incident_reviews**

| العمود | النوع | الغرض |
|---|---|---|
| id | UUID PK | المعرّف الأساسي |
| incident_id | UUID FK → incidents.id | ON DELETE CASCADE |
| reviewer_id | UUID FK → users.id | قابل لأن يكون NULL في وضع العرض |
| action | review_action | confirm / reject / close / comment |
| previous_status / new_status | incident_status | قبل وبعد الإجراء |
| note | TEXT | ملاحظة المراجع |
| created_at | TIMESTAMPTZ | زمن الإجراء |

## 8.4 الجداول المؤجَّلة

ثلاثة جداول مخطَّطة خارج الـ MVP، نثبّت أسماءها حتى لا تختلط لاحقاً.

**cameras** Future

id, name, location_label, description, created_at. تربط الفيديو بكاميرا؛ videos.camera_id يبقى NULL في الـ MVP.

**model_runs** Future

job_id, model_name, model_version, device, weights_path, classes_json, duration_ms. تسجّل تفاصيل كل تشغيل Model للتشخيص.

**notifications** Future

user_id, incident_id, channel, status — تنبيهات خارجية بعد استقرار الـ MVP.

## 8.5 مخطط العلاقات (ER Diagram)

كل الجداول وعلاقاتها من نوع 1..N، و videos جذر تتفرّع منه بقية الكيانات.




**شكل 8-1:** مخطط الكيانات والعلاقات (ER) – الجداول المرمّزة بالرمادي مؤجَّلة عن الـ MVP

### جدول 8-10: ملخّص العلاقات

| العلاقة | المعنى |
|---|---|
| `videos 1..N processing_jobs` | الفيديو الواحد يمكن معالجته أكثر من مرة |
| `processing_jobs 1..N detections` | كل وظيفة تُنتج كشوفات كثيرة |
| `processing_jobs 1..N tracks` | كل وظيفة تُنتج مسارات تتبّع |
| `videos 1..N incidents` | الفيديو الواحد قد يحمل عدة وقائع |
| `processing_jobs 1..N incidents` | الوظيفة الواحدة تولّد وقائع |
| `incidents 1..N evidence` | الواقعة الواحدة قد يكون لها أكثر من دليل |
| `incidents 1..N incident_reviews` | سجل كامل لإجراءات المراجعة |
| `model_runs N..1 processing_jobs` | تشغيلات متعدّدة تنتمي لوظيفة واحدة |

## 8.6 الفهارس ودعم الفلترة والترتيب

كل فلتر شرط WHERE؛ نربط كل فهرس باستعلام حقيقي لتفادي المسح الكامل للجدول.

### جدول 8-11: الفهارس ومبرّر كل واحد

| الفهرس | الاستعلام الذي يخدمه |
|---|---|
| `videos(checksum_sha256)` | كشف الفيديو المكرّر قبل حفظ الملف |
| `videos(status)` | عدّ الفيديوهات قيد المعالجة أو الفاشلة في لوحة التحكم |
| `detections(job_id, frame_number)` | قراءة كشوفات وظيفة مرتّبةً بالإطار |
| `detections(class_name)` | تحليل توزيع الأصناف |
| `incidents(status)` | قائمة الوقائع المعلّقة للمراجعة `pending_review` |
| `incidents(incident_type)` | فلترة بنوع الواقعة في الـ Analytics |
| `incidents(severity)` | فلترة بالخطورة |
| `incidents(created_at)` | الترتيب الزمني الافتراضي `-created_at` ونطاقات التاريخ |
| `incidents(video_id)` | جلب كل وقائع فيديو معيّن |

الفلترة المتعدّدة تُبنى كسلاسل شروط، مع ترقيم صفحات (**pagination**) دائماً عبر LIMIT و OFFSET.

## 8.7 نمو جدول detections وخيار PERSIST_ALL_DETECTIONS

فيديو عشر دقائق = 15000 إطار؛ تسجيل كل كشف يُنشئ ملايين الصفوف بلا فائدة، فالمهمّ الوقائع لا كل كشف.

- **الافتراضي:** PERSIST_ALL_DETECTIONS=false، فلا نكتب إلا الكشوفات المهمّة أو عيّنات منها.
- **عند التفعيل:** PERSIST_ALL_DETECTIONS=true للتطوير والتشخيص فقط.
- **الربط:** قرار الكتابة داخل الـ worker (الفصل 11)، لا داخل الـ Model.

> **تنبيه عملي**
>
> ابدأ بالقيمة الافتراضية، وارفعها مؤقتاً فقط عند تحقيقك في مشكلة كشف، وتجنّبها على جهاز ضعيف أو قرص ممتلئ.

## 8.8 لماذا لا نخزّن الفيديو داخل PostgreSQL

لماذا لا نضع الفيديو في حقل BYTEA؟ لأن حجمه يضخّم النسخ الاحتياطي والاستعلام. نحفظ **وصفاً صغيراً** فقط، ويبقى الملف في نظام ملفات أو تخزين كائنات مستقل.

**جدول 8-12: أين يوضع كل شيء**

| البيانات | مكانها | السبب |
|---|---|---|
| البيانات الوصفية (original_filename، size_bytes، status) | PostgreSQL | صغيرة، قابلة للفلترة والفهرسة |
| ملف الفيديو نفسه | طبقة التخزين /storage | كبير جداً، لا يحتاج استعلاماً |
| صور ومقاطع الدليل | طبقة التخزين /storage | تُخزَّن مرة وتُقرأ عند العرض |
| المفتاح storage_key | PostgreSQL | يربط السجل بالملف دون تخزين الملف |

## 8.9 Alembic والمهاجرات الأولى

**Alembic** أداة المهاجرات (`migrations`) لـ **SQLAlchemy**: تحوّل تعريفات الـ Models إلى تغييرات فعلية، فيعيد أي عضو بناء المخطط بأمر واحد.

backend/app/models/video.py — python

```python
# backend/app/models/video.py (abbreviated)
import uuid
from sqlalchemy import BigInteger, DateTime, Enum, ForeignKey, Integer, Numeric, String, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.db.base import Base
from app.models.enums import VideoStatus


class Video(Base):
    __tablename__ = "videos"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    camera_id: Mapped[uuid.UUID | None] = mapped_column(ForeignKey("cameras.id"))
    original_filename: Mapped[str] = mapped_column(String(255), nullable=False)
    stored_filename: Mapped[str] = mapped_column(String(255), nullable=False)
    storage_key: Mapped[str] = mapped_column(String(512), unique=True)
    content_type: Mapped[str] = mapped_column(String(100))
    size_bytes: Mapped[int] = mapped_column(BigInteger)
    duration_seconds: Mapped[float | None] = mapped_column(Numeric(10, 3))
    fps: Mapped[float | None] = mapped_column(Numeric(8, 3))
    frame_count: Mapped[int | None] = mapped_column(Integer)
    checksum_sha256: Mapped[str | None] = mapped_column(String(64), index=True)
    status: Mapped[VideoStatus] = mapped_column(
        Enum(VideoStatus, name="video_status"), default=VideoStatus.uploaded)
    uploaded_by: Mapped[uuid.UUID | None] = mapped_column(ForeignKey("users.id"))
    created_at: Mapped[None] = mapped_column(DateTime(timezone=True), server_default=func.now())
    updated_at: Mapped[None] = mapped_column(DateTime(timezone=True), server_default=func.now(),
onupdate=func.now())

    jobs = relationship("ProcessingJob", back_populates="video", cascade="all, delete-orphan")
    incidents = relationship("Incident", back_populates="video")
```

backend/alembic.ini (excerpt) — ini

```ini
[alembic]
script_location = migrations
prepend_sys_path = .
# sqlalchemy.url is set in migrations/env.py from settings.DATABASE_URL
```

```bash
$ cd backend
$ alembic init migrations
$ alembic revision --autogenerate -m "initial schema"
$ alembic upgrade head
```

terminal

بعد الأمر الأخير تطابق القاعدة المخطط. --autogenerate يولّد المهاجرة بمقارنة الـ Models بالوضع الحالي، ومراجعتها يدوياً إلزامية.

## 8.10 الخصوصية: لا نخزّن هويات حقيقية للعاملين

لا نحتاج معرفة **من** هو العامل، فلا نخزّن أسماء ولا صوراً تعريفية ولا أرقام هوية، بل track_id مؤقّتاً وصور دليل للمراجعة.

> **ملاحظة للجنة المناقشة**
>
> النظام يحفظ أدلة بصرية للحدث لا هويات أشخاص، ويساعد المراجعة البشرية لا التعقّب الفردي.

### خلاصة الفصل 8

**ما تعلّمناه**

ثبّتنا مخطط قاعدة البيانات كاملاً: ثمانية enums، وجداول الـ MVP الثمانية، وثلاثة جداول مؤجَّلة، وعلاقات 1..N، والفهارس المرتبطة بالفلترة، وسبب عدم تخزين الفيديو داخل **PostgreSQL**، وكيف نُدير كل ذلك بـ **Alembic**.

**ما يجب تنفيذه**

- [ ] كتابة app/models/enums.py بكل القيم الثمانية حرفياً.
- [ ] كتابة ملفات الـ Models لجميع جداول الـ MVP.
- [ ] تهيئة migrations/ وتوليد أول مهاجرة ثم مراجعتها يدوياً.
- [ ] إضافة الفهرس الفريد الجزئي على processing_jobs(video_id).
- [ ] إبقاء PERSIST_ALL_DETECTIONS=false افتراضياً.

**المخرجات المتوقعة**

- قاعدة بيانات محلية تُبنى بأمر alembic upgrade head.
- ملف مهاجرة أولى يحتوي كل الجداول والفهارس.
- مخطط ER محدَّث في docs/ مطابق للتنفيذ.

**كيف نتحقق من الإنجاز**

- أي عضو يستطيع بناء القاعدة من الصفر بأمر واحد دون خطوات يدوية.
- محاولة إدخال حالة خارج الـ enum تفشل.
- إنشاء وظيفتين نشطتين لنفس الفيديو يفشل بسبب الفهرس الفريد الجزئي.

**أخطاء شائعة**

- تخزين الفيديو أو صور الدليل داخل القاعدة.
- تغيير أسماء الأعمدة أو قيم الـ enum لتسهيل الكتابة.
- تعديل ملفات الـ Models دون توليد مهاجرة، فيختلف المخطط بين الأجهزة.
- تفعيل تخزين كل الكشوفات على جهاز ضعيف فيمتلئ القرص.

# 09 رفع الفيديو وتخزين الملفات

كيف يستقبل النظام الفيديو ويتحقّق منه ويحفظه بأمان بعيداً عن قاعدة البيانات

هذا الفصل يغطّي أول خطوة في النظام: وصول ملف الفيديو إلى الخادم. الهدف أن نقبل الملفات الصحيحة فقط، ونمنع التكرار، ونحفظ الملف بمفتاح (`storage key`) آمن، مع بقاء بياناته الوصفية في `PostgreSQL` والملف نفسه في طبقة تخزين مستقلة.

## 9.1 الصيغ المدعومة وحدود الحجم

تُقيّد الرفع بقائمة امتدادات مسموح بها وحد أقصى للحجم، ونقرأ هذه القيم من متغيّرات البيئة لا من الكود، حتى يمكن تغييرها دون إعادة بناء المشروع.

**جدول 9-1: قيود الرفع ومتغيّراتها**

| القيد | المتغيّر | القيمة الافتراضية المقترحة |
|---|---|---|
| الامتدادات المسموح بها | ALLOWED_VIDEO_EXTENSIONS | mp4, mov, avi, mkv |
| الحد الأقصى للحجم (ميغابايت) | MAX_UPLOAD_MB | يُحدَّد حسب الفيديو المتاح |
| حجم قطعة القراءة (ميغابايت) | UPLOAD_CHUNK_MB | 4 |
| مجلّد الفيديوهات المرفوعة | UPLOAD_DIR | storage/uploads |
| مجلّد أدلة الوقائع | EVIDENCE_DIR | storage/evidence |

## 9.2 التحقق من الملف في ثلاث طبقات

لا نثق باسم الملف وحده؛ نتحقّق في ثلاث مراحل متتالية:

1. **الامتداد:** يجب أن يكون ضمن ALLOWED_VIDEO_EXTENSIONS.
2. **البصمة الرقمية للمحتوى (`magic bytes`):** نقرأ أول بايتات الملف ونقارنها بتوقيع صيغ الفيديو المعروفة، فيُرفض ملف تنفيذي مُسمّى .mp4.
3. **فحص المحتوى:** نفتح الملف بـ `OpenCV` لقراءة عدد الإطارات والأبعاد والمدة، وإن فشل الفتح نعتبره فيديو تالفاً ونرفضه.

> **لا تحفظ قبل التحقق**
>
> الخطأ الشائع هو كتابة الملف على القرص ثم التحقق منه. الأفضل تجميع القطع في الذاكرة أثناء الفحص، ثم حفظ الملف مرة واحدة فقط بعد اجتياز كل الفحوص.

## 9.3 أسماء الملفات الآمنة والمسارات والبيانات الوصفية

اسم الملف الأصلي لا يُستخدم أبداً على القرص. نولّد اسماً فريداً (stored_filename) من معرّف UUID مع الامتداد، ونبني المفتاح storage_key مثل uploads/<uuid>.mp4. هذا يمنع اجتياز المسار (`path traversal`) ويمنع تعارض الأسماء. الاسم الأصلي يبقى في original_filename للعرض فقط.

**جدول 9-2: الفرق بين السجل في القاعدة والملف على القرص**

| العنصر | المكان | مثال |
|---|---|---|
| السجل الوصفي | videos في PostgreSQL | id, size_bytes, status |
| اسم العرض | original_filename | site_a_cam1.mp4 |
| الاسم على القرص | stored_filename | 8f3c...a1.mp4 |
| المفتاح | storage_key | uploads/8f3c...a1.mp4 |

## 9.4 منع التكرار بـ checksum

أثناء القراءة نحسب sha256 للمحتوى ونخزّنه في checksum_sha256. قبل الحفظ نسأل القاعدة: هل يوجد فيديو بنفس البصمة؟ إن وُجد، نعيد السجل الموجود بدل تخزين نسخة مكرّرة، فنوفّر مساحة القرص ونمنع معالجة نفس الملف مرّتين. عمود البصمة مفهرس لهذا الاستعلام تحديداً.

## 9.5 طبقة Storage abstraction

نفصل الكود عن مكان التخزين الفعلي عبر واجهة واحدة StorageBackend، ولها تطبيقان: محلي للتطوير وتخزين كائنات للإنتاج. الـ endpoint يتعامل مع الواجهة فقط، ولا يعرف إن كان الملف على القرص أم في السحابة.




**شكل 9-1:** طبقة التخزين تفصل الـ endpoint عن مكان الملف

الفائدة العملية أن الـ endpoint والـ worker لا يحتاجان تعديلاً عند تغيير مكان التخزين؛ يكفي تبديل التطبيق المختار حسب قيمة STORAGE_BACKEND.

**backend/app/storage/base.py** — python

```python
# backend/app/storage/base.py
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import BinaryIO


@dataclass
class StoredObject:
    storage_key: str
    size_bytes: int
    content_type: str


class StorageBackend(ABC):
    """Common interface for local and object storage."""

    @abstractmethod
    def save(self, key: str, data: bytes, content_type: str) -> StoredObject: ...

    @abstractmethod
    def open_stream(self, key: str) -> BinaryIO: ...

    @abstractmethod
    def exists(self, key: str) -> bool: ...

    @abstractmethod
    def delete(self, key: str) -> None: ...

    @abstractmethod
    def url_for(self, key: str) -> str: ...
```

**backend/app/storage/local.py** — python

```python
# backend/app/storage/local.py
from pathlib import Path
from typing import BinaryIO
from app.core.config import settings
from app.storage.base import StorageBackend, StoredObject


class LocalStorage(StorageBackend):
    """Filesystem storage used for local development."""

    def __init__(self, root: str | None = None) -> None:
        self.root = Path(root or settings.LOCAL_STORAGE_ROOT).resolve()
        self.root.mkdir(parents=True, exist_ok=True)

    def _path(self, key: str) -> Path:
        target = (self.root / key.lstrip("/")).resolve()
        if not str(target).startswith(str(self.root)):
            raise ValueError("storage key escapes the storage root")
        return target

    def save(self, key, data, content_type) -> StoredObject:
        path = self._path(key)
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_bytes(data)
        return StoredObject(key, len(data), content_type)

    def open_stream(self, key) -> BinaryIO:
        return open(self._path(key), "rb")
```

```python
    def exists(self, key) -> bool:
        return self._path(key).is_file()

    def delete(self, key) -> None:
        self._path(key).unlink(missing_ok=True)

    def url_for(self, key) -> str:
        return f"/api/v1/evidence/file?key={key}"
```

## 9.6 نقطة الرفع POST /api/v1/videos

هذه هي النقطة التي يرفع إليها الـ Frontend الملف بصيغة multipart/form-data. تقرأ الملف على قطع، تتحقق من الحد الأقصى والحجم، تحسب البصمة، ثم تفوّض الحفظ والتحقق العميق إلى VideoService. الرد بحالة 201 يحمل كائن VideoOut.

python — backend/app/api/v1/endpoints/videos.py

```python
# backend/app/api/v1/endpoints/videos.py (upload excerpt)
import hashlib
from fastapi import APIRouter, Depends, File, HTTPException, UploadFile, status
from app.core.config import settings
from app.dependencies import get_storage, get_video_service
from app.schemas.video import VideoOut

router = APIRouter(prefix="/videos", tags=["videos"])


@router.post("", response_model=VideoOut, status_code=status.HTTP_201_CREATED)
async def upload_video(file: UploadFile = File(...), storage=Depends(get_storage),
                       service=Depends(get_video_service)) -> VideoOut:
    ext = (file.filename or "").rsplit(".", 1)[-1].lower()
    if ext not in settings.ALLOWED_VIDEO_EXTENSIONS:
        raise HTTPException(status.HTTP_415_UNSUPPORTED_MEDIA_TYPE, "extension not allowed")

    digest, total, chunks = hashlib.sha256(), 0, []
    limit = settings.MAX_UPLOAD_MB * 1024 * 1024
    while chunk := await file.read(settings.UPLOAD_CHUNK_MB * 1024 * 1024):
        total += len(chunk)
        if total > limit:
            raise HTTPException(status.HTTP_413_REQUEST_ENTITY_TOO_LARGE, "file too large")
        digest.update(chunk)
        chunks.append(chunk)

    payload = b"".join(chunks)
    if not service.looks_like_video(payload[:64]):
        raise HTTPException(status.HTTP_400_BAD_REQUEST, "not a valid video file")

    video = await service.create_from_upload(
        original_filename=file.filename,
        content_type=file.content_type or "application/octet-stream",
        payload=payload, checksum=digest.hexdigest(), storage=storage)
    return VideoOut.model_validate(video)
```

**ماذا يفعل VideoService؟**

looks_like_video يفحص البصمة الرقمية، create_from_upload يفتح الملف بـ **OpenCV** لقراءة المدة وعدد الإطارات، ثم يحفظه ويُنشئ صف videos بالحالة uploaded.

## 9.7 الوصول الآمن والتنظيف والاحتفاظ

- **الوصول للملفات:** لا نكشف مسارات القرص مباشرةً. الـ Frontend يطلب GET /api/v1/evidence/{evidence_id}/file، والخادم يبثّ الملف (**streaming**) بعد التحقق من الصلاحية.
- **تنظيف الرفع المتروك:** أي ملف كُتب ثم فشل إنشاء السجل يُحذف فوراً لتفادي الملفات اليتيمة.
- **سياسة الاحتفاظ:** عند حذف فيديو (DELETE /api/v1/videos/{video_id}) نحذف سجله وملفاته وأدلته، مع إبقاء ما يفرضه الموقع من مدة احتفاظ قانونية.
- **الأدلة:** عند إنشاء واقعة نقتطع صورة من الإطار ونحفظها في EVIDENCE_DIR ونربطها بجدول evidence (الفصل 12).

عند توليد الدليل نقتطع صورة JPEG من الإطار الذي وقعت فيه الواقعة عبر **OpenCV** بنوع محتوى image/jpeg. المقطع القصير (**clip**) اختياري في الـ MVP ويُضاف بنفس آلية التخزين لاحقاً. المهم أن الصورة والمقطع يُخزَّنان كملفّين مستقلّين، ولا يُقحمان داخل صف قاعدة البيانات.

## 9.8 مقارنة خيارات التخزين

**جدول 9-3: خيارات التخزين ومتى نستخدم كل خيار**

| الخيار | المتغيّر STORAGE_BACKEND | الاستخدام |
|---|---|---|
| مجلد محلي | `local` | التطوير والعرض التوضيحي على جهاز واحد، لا يحتاج شبكة أو حساب سحابي |
| تخزين متوافق مع S3 | `s3` | الإنتاج: يُعدّ بمتغيّرات S3_ENDPOINT_URL و S3_BUCKET و المفاتيح |
| تخزين سحابي مُدار | يُبنى فوق واجهة S3 | خيار مستقبلي؛ نفس الواجهة بلا تغيير في الكود |

**نصيحة تنفيذ**

ابدأ دائماً بـ local حتى يعمل الرفع والمعالجة من البداية للنهاية، ثم انتقل إلى s3 بتغيير متغيّر واحد فقط، وهذا هو مكسب الطبقة المجرّدة. بهذا يصبح الانتقال من التطوير إلى الإنتاج خطوة إعداد لا إعادة كتابة.

### خلاصة الفصل 9

**ما تعلّمناه**

كيف نستقبل الفيديو ونتحقّق منه في ثلاث طبقات، ونولّد مفتاح تخزين آمناً، ونمنع التكرار بالبصمة، ونفصل الكود عن مكان التخزين عبر واجهة StorageBackend.

**ما يجب تنفيذه**

- [ ] كتابة app/storage/base.py و local.py.
- [ ] ضبط MAX_UPLOAD_MB و ALLOWED_VIDEO_EXTENSIONS و UPLOAD_CHUNK_MB.
- [ ] تنفيذ POST /api/v1/videos مع التحقق والبصمة.
- [ ] تنفيذ نقطة البثّ الآمن للأدلة.

**المخرجات المتوقعة**

- ملف فيديو محفوظ بمفتاح آمن وصف videos بحالة uploaded.
- رفض واضح للملفات الكبيرة أو التالفة أو المكرّرة.

**كيف نتحقق من الإنجاز**

- رفع نفس الملف مرتين لا يُنشئ نسختين على القرص.
- ملف نصّي بامتداد .mp4 يُرفض.
- الوصول المباشر لمسار القرص غير ممكن، والقراءة تتم عبر الـ API فقط.

**أخطاء شائعة**

- استخدام اسم الملف الأصلي على القرص.
- حفظ الملف قبل التحقق فيترك ملفات يتيمة.
- كشف مسار التخزين للـ Frontend.
- وضع بيانات اعتماد التخزين داخل الكود بدل متغيّرات البيئة.

CH10 · AI MODEL INTEGRATION

# 10 دمج الـ AI Models المتفق عليها

تحميل الـ checkpoints وتحويل مخرجاتها إلى صيغة داخلية موحّدة

لا نصمّم Models جديدة ولا نعيد التدريب؛ المهمة هي **الدمج**: تحميل كل `checkpoint` وتحويل مخرجاته إلى صيغة كشف واحدة تربط الفصول 11 و 12 و 13.

## 10.1 مبدأ أساسي: لا نعيد تصميم الـ Models

الـ Models الثلاثة (PPE، الحرائق والدخان، تتبّع الأشخاص) اختيرت سابقاً؛ دورنا أن نغلّفها بمحوّل (`adapter`) يوحّد مخرجاتها. تغيير المعمارية أو إعادة التدريب خارج النطاق.

**قاعدة صارمة**

لا نغيّر أوزاناً ولا نعيد تدريباً. إن لم يعمل الـ Model نسجّل الخطأ ونعالجه، ولا نستبدله في منتصف المشروع.

## 10.2 مسارات الـ checkpoints وكيف نستبدل الـ Placeholders

كل مسار من متغيّر بيئة وقيمته الافتراضية `placeholder`؛ نستبدله بمسار حقيقي بمجرد تسليم الملف، دون تعديل كود.

**جدول 10-1: الـ Placeholders وكيف تُستبدل**

| الـ Placeholder | المتغيّر | كيف نستبدله |
|---|---|---|
| `<PPE_MODEL_CHECKPOINT_PATH>` | `PPE_MODEL_PATH` | نضع مسار ملف أوزان الـ PPE داخل `MODEL_DIR` |
| `<FIRE_SMOKE_MODEL_CHECKPOINT_PATH>` | `FIRE_SMOKE_MODEL_PATH` | مسار أوزان الحرائق والدخان |
| `<PERSON_TRACKER_CHECKPOINT_PATH>` | `PERSON_TRACKER_PATH` | مسار أوزان تتبّع الأشخاص |
| `<MODEL_VERSION>` | ثابت داخل المحوّل | يُستبدل برقم إصدار الـ Model بعد التدريب، ويُسجَّل في كل كشف |

## 10.3 اختيار الجهاز: CPU أم GPU

المتغيّر MODEL_DEVICE يحدّد cpu أو cuda: نطوّر على cpu للجميع، ونستخدم cuda عند توفّر كرت لتقليل الزمن. الكود لا يتغيّر.

**ذاكرة الـ GPU**

إذا نفدت ذاكرة كرت الرسوم، نُصغّر INFERENCE_IMG_SIZE أو نعود إلى cpu. لا نُشغّل عدّة عمليات inference ثقيلة في نفس الوقت على نفس الجهاز.

## 10.4 إعدادات الـ Inference

كل إعدادات التشغيل تأتي من core/config.py، فلا توجد أرقام مكتوبة داخل الكود.

backend/app/core/config.py

```python
# backend/app/core/config.py (model settings excerpt)
from pydantic_settings import BaseSettings


class Settings(BaseSettings):
    MODEL_DIR: str = "models"
    PPE_MODEL_PATH: str = "<PPE_MODEL_CHECKPOINT_PATH>"
    FIRE_SMOKE_MODEL_PATH: str = "<FIRE_SMOKE_MODEL_CHECKPOINT_PATH>"
    PERSON_TRACKER_PATH: str = "<PERSON_TRACKER_CHECKPOINT_PATH>"
    MODEL_DEVICE: str = "cpu"
    DETECTION_CONF_THRESHOLD: float = 0.25
    NMS_IOU_THRESHOLD: float = 0.45
    INFERENCE_IMG_SIZE: int = 640
    PERSIST_ALL_DETECTIONS: bool = False
    PPE_CONF_THRESHOLD: float = 0.40
    FIRE_CONF_THRESHOLD: float = 0.40
    SMOKE_CONF_THRESHOLD: float = 0.40


settings = Settings()
```

INFERENCE_IMG_SIZE حجم الإطار قبل الـ Model، وNMS_IOU_THRESHOLD يحذف الصناديق المتداخلة (**Non-Maximum Suppression**)، وDETECTION_CONF_THRESHOLD حدّ الثقة العام. ولأن كل Model مختلف، توجد عتبات أدق لكل نوع (PPE_CONF_THRESHOLD، FIRE_CONF_THRESHOLD، SMOKE_CONF_THRESHOLD).

## 10.5 خريطة أسماء الأصناف

كل **checkpoint** يعيد أسماء أصناف خاصة به، نخريطها (**class map**) إلى أسماء داخلية واحدة ليتعامل معها الفصل 12. الصنف غير المعروف نُبقيه باسمه الأصلي.

## 10.6 الصيغة الموحّدة للكشف

العقد الداخلي: كل محوّل يعيد قائمة Detection بنفس حقول المواصفة (الفصل 4)، والواجهة DetectionModel تفرض load و predict.

backend/app/ai/base.py

```python
# backend/app/ai/base.py
from dataclasses import dataclass
from typing import Protocol
import numpy as np


@dataclass
class Detection:
    """Normalized detection shared by every model adapter."""
    class_name: str
    confidence: float
    bbox: tuple[int, int, int, int]  # x1, y1, x2, y2
    frame_number: int
    timestamp_seconds: float
    model_name: str
    model_version: str
    track_id: int | None = None


class DetectionModel(Protocol):
    name: str
    version: str

    def load(self) -> None: ...

    def predict(self, frame: np.ndarray, frame_number: int,
                timestamp_seconds: float) -> list[Detection]: ...
```

## 10.7 الخدمات الأربع

نبني أربع خدمات لكل منها مسؤولية واحدة واضحة:

- PPEDetectionService — كشف معدات الوقاية (ملف app/ai/ppe_detector.py).
- FireSmokeDetectionService — كشف الحرائق والدخان (app/ai/fire_smoke_detector.py).
- PersonTrackingService — تتبّع الأشخاص بـ **ByteTrack** عبر model.track (app/ai/person_tracker.py).
- SafetyRuleEngine — طبقة القواعد التي تُستخدم مخرجات الخدمات الثلاث (الفصل 12، ملف app/ai/rules.py).

## 10.8 محوّل الـ PPE كنموذج للتطبيق

هذا الملف يوضّح النمط الذي تتبعه بقية الخدمات: تحميل الـ checkpoint مرة واحدة، ثم ترجمة كل صندوق إلى Detection بأسماء موحّدة.

backend/app/ai/ppe_detector.py

```python
# backend/app/ai/ppe_detector.py
from ultralytics import YOLO
from app.ai.base import Detection, DetectionModel
from app.core.config import settings

# Map raw checkpoint labels to agreed internal class names.
PPE_CLASS_MAP = {
    "helmet": "helmet",
    "no_helmet": "no_helmet",
    "vest": "vest",
    "no_vest": "no_vest",
    "person": "person",
}
```

```python
class PPEDetectionService(DetectionModel):
    """Adapter for the agreed PPE checkpoint (loaded, not redesigned)."""

    name = "ppe_detector"
    version = "<MODEL_VERSION>"  # replace with the real trained version

    def __init__(self) -> None:
        self._model: YOLO | None = None

    def load(self) -> None:
        self._model = YOLO(settings.PPE_MODEL_PATH)
        self._model.to(settings.MODEL_DEVICE)

    def predict(self, frame, frame_number, timestamp_seconds) -> list[Detection]:
        if self._model is None:
            raise RuntimeError("PPEDetectionService.load() was not called")
        results = self._model.predict(
            frame, imgsz=settings.INFERENCE_IMG_SIZE,
            conf=settings.PPE_CONF_THRESHOLD, iou=settings.NMS_IOU_THRESHOLD,
            device=settings.MODEL_DEVICE, verbose=False)
        detections: list[Detection] = []
        for box in results[0].boxes:
            raw = self._model.names[int(box.cls.item())]
            detections.append(Detection(
                class_name=PPE_CLASS_MAP.get(raw, raw),
                confidence=float(box.conf.item()),
                bbox=tuple(int(v) for v in box.xyxy[0].tolist()),
                frame_number=frame_number, timestamp_seconds=timestamp_seconds,
                model_name=self.name, model_version=self.version))
        return detections
```

### 10.9 سجل الـ Models model_registry

لتفادي تحميل ثقيل متكرّر، يضمن السجل نسخة واحدة من كل خدمة عبر lru_cache.

python backend/app/ai/model_registry.py

```python
# backend/app/ai/model_registry.py
from functools import lru_cache
from app.ai.base import DetectionModel
from app.ai.ppe_detector import PPEDetectionService
from app.ai.fire_smoke_detector import FireSmokeDetectionService
from app.ai.person_tracker import PersonTrackingService


@lru_cache(maxsize=1)
def get_ppe_detector() -> DetectionModel:
    model = PPEDetectionService()
    model.load()
    return model


@lru_cache(maxsize=1)
def get_fire_smoke_detector() -> DetectionModel:
    model = FireSmokeDetectionService()
    model.load()
    return model


@lru_cache(maxsize=1)
def get_person_tracker() -> DetectionModel:
    model = PersonTrackingService()
    model.load()
    return model
```

### 10.10 المحوّل التجريبي demo_adapter ▯▯▯ ▯▯▯▯▯▯▯▯

لاختبار خط المعالجة قبل توفّر الـ checkpoints نكتب محوّلاً وهمياً يعيد صناديق ثابتة دون تشغيل أي Model. هذا المحوّل **وهمي صراحةً** ولا يُستخدم في العرض النهائي ولا في أي قياس دقة.

python backend/app/ai/demo_adapter.py

```python
# backend/app/ai/demo_adapter.py
# WARNING: this adapter is FAKE. It runs no model and returns fixed boxes.
# Use it ONLY to test the pipeline when real checkpoints are missing.
from app.ai.base import Detection, DetectionModel
```

```python
class DemoDetectionService(DetectionModel):
    """Deterministic fake detections for pipeline testing only."""

    name = "demo_adapter"
    version = "demo"

    def load(self) -> None:
        return None

    def predict(self, frame, frame_number, timestamp_seconds) -> list[Detection]:
        height, width = frame.shape[:2]
        # One fixed person box in the centre of the frame.
        return [Detection(
            class_name="person", confidence=0.50,
            bbox=(width // 4, height // 4, width // 2, height // 2),
            frame_number=frame_number, timestamp_seconds=timestamp_seconds,
            model_name=self.name, model_version=self.version, track_id=1)]
```

**خطر الخلط**

لا تترك demo_adapter يعمل خارج وضع الاختبار؛ يجب أن يكون تبديله صريحاً عبر الإعدادات وأن يظهر في السجل أن الـ Model وهمي.

## 10.11 الأداء والموارد ومعالجة الأخطاء

- **الدفعات:** نجمع عدّة إطارات في دفعة (`batch`) لتسريع الـ inference، ونعود للإطار المفرد عند ضيق الذاكرة.
- **التسخين `warm-up`:** تشغيل الـ Model على إطار وهمي عند التحميل لتفادي بطء أول إطار.
- **الموارد:** الـ checkpoint يُحمَّل مرة واحدة عبر السجل ويُعاد استخدامه.
- **الأخطاء:** كل predict داخل try/except؛ خطأ إطار لا يُسقط الوظيفة.

## 10.12 حالات الفشل الشائعة وكيف نعالجها

**جدول 10-2: حالات الفشل وعلاجها**

| الحالة | التشخيص والعلاج |
|---|---|
| أوزان مفقودة | الملف في `PPE_MODEL_PATH` غير موجود؛ تتحقّق من `MODEL_DIR` ونمنع بدء الـ worker برسالة واضحة |
| صيغة checkpoint غير متوافقة | الـ checkpoint ليس من نفس إصدار مكتبة التشغيل؛ نعيد التصدير من الملف الأصلي ولا نعيد التدريب |
| لا يوجد GPU | نضبط `MODEL_DEVICE=cpu` ويستمر النظام بزمن أطول |
| نفاد ذاكرة GPU | نُصغّر `INFERENCE_IMG_SIZE` أو نقلّل حجم الدفعة أو نعود إلى `cpu` |
| صنف غير معروف | نُبقيه باسمه الأصلي في السجل ونراجعه يدوياً قبل إضافته للخريطة |

## 10.13 تسجيل الإصدار وإعادة الإنتاج، والدقة

كل كشف يحمل model_name و model_version، وإصدار الحزمة في processing_jobs.model_bundle_version، فنُفسّر نتائج أي وظيفة لاحقاً. الدقة تُقاس على مجموعة اختبار مستقلة ولا تُفترض؛ الجدول يبقى فارغاً حتى تنتهي التجربة.

**جدول 10-3: نتائج تقييم الـ Models (يُملأ بعد التجربة)**

| الـ Model | مجموعة الاختبار | النتيجة |
|---|---|---|
| `ppe_detector` | `<DATASET_NAME>` | (يُملأ بعد التجربة) |
| `fire_smoke` | `<DATASET_NAME>` | (يُملأ بعد التجربة) |
| `person_tracker` | `<DATASET_NAME>` | (يُملأ بعد التجربة) |

**لا ندمج بيانات تدريب مختلفة بلا تحقّق**

قبل دمج أوزان من مصادر مختلفة، تحقّق من خريطة الأصناف وتنسيق الـ annotation وتقسيم (`splits`) البيانات، وإلا كانت النتائج مضلّلة.

**خلاصة الفصل 10**

**ما تعلّمناه**

كيف نغلّف الـ Models المتفق عليها بمحوّلات موحّدة، وكيف نحوّل مخرجاتها إلى كائن Detection واحد، وكيف نُدير التحميل

والإعدادات والأخطاء.

### ما يجب تنفيذه

- [ ] كتابة app/ai/base.py بالـ Detection و DetectionModel.
- [ ] كتابة المحوّلات الثلاثة وسجل الـ Models.
- [ ] استبدال كل `placeholder` بمسار حقيقي عند توفّر الـ checkpoints.
- [ ] إبقاء demo_adapter للاختبار فقط.

### المخرجات المتوقعة

- خدمة تُعيد قائمة Detection موحّدة من أي Model.
- الـ checkpoint يُحمَّل مرة واحدة لكل عملية.

### كيف نتحقق من الإنجاز

- تشغيل demo_adapter يُنتج كشوفات ويُكمل خط المعالجة دون Models حقيقية.
- كل كشف يحمل model_name و model_version.
- لا يوجد رقم دقة في التوثيق بلا قياس.

### أخطاء شائعة

- إعادة تدريب أو استبدال Model في منتصف التكامل.
- تحميل الـ checkpoint في كل استدعاء بدل مرة واحدة.
- استخدام demo_adapter في العرض النهائي.
- كتابة مسارات الأوزان داخل الكود بدل متغيّرات البيئة.

CH11 · VIDEO PIPELINE

# 11 خط معالجة الفيديو والـ Background Jobs

لماذا لا نعالج فيديو طويلاً داخل طلب HTTP واحد، وكيف نبني worker مستقلاً

معالجة فيديو دقائق قد تتجاوز مهلة أي طلب HTTP. لذلك نفصل الطلب عن التنفيذ: الـ API ينشئ وظيفة (`job`) ويعيد فوراً، وعملية مستقلة (`worker`) تنفّذها وتُحدّث التقدّم.

## 11.1 لماذا الطلب المتزامن خطأ

لو عالجنا الفيديو داخل POST /api/v1/videos/{video_id}/process مباشرةً، لانتظر المتصفح دقائق ثم قد يقطعه انقطاع الشبكة فتضيع النتيجة، ولتجمّد خادم `Uvicorn` عند عدّة فيديوهات. الحل: تنفيذ منفصل وواجهة تستعلم عن الحالة.

## 11.2 تصميم خط الـ Jobs في ست عشرة خطوة

1. وصول طلب POST /api/v1/videos/{video_id}/process.
2. التحقق من وجود الفيديو وأن حالته تسمح بالمعالجة.
3. إنشاء صف في processing_jobs بحالة queued.
4. إرجاع 202 مع كائن JobOut فوراً.
5. الـ worker يستعلم دورياً عن وظيفة queued.
6. حجز الوظيفة (`claim`) وتحويلها إلى running.
7. فتح ملف الفيديو وقراءة عدد الإطارات والمدة و fps.
8. تحميل الـ Models مرة واحدة عبر السجل (الفصل 10).
9. المرور على الإطارات مع `frame sampling`.
10. تشغيل الـ inference على الإطارات المختارة فقط.
11. تطبيع المخرجات إلى كائنات Detection.
12. تقييم قواعد السلامة (SafetyRuleEngine).
13. توليد الوقائع وحفظ صور الدليل.
14. تحديث progress و frames_processed دورياً.
15. إغلاق الوظيفة completed وتحديث حالة الفيديو إلى processed.
16. عند الفشل: failed مع error_message واضح.

## 11.3 حالات الـ Job

القيم الخمس في المواصفة: queued ثم running ثم completed أو failed، cancelledg قبل البدء.

## 11.4 لماذا لا نستخدم BackgroundTasks كنظام jobs

آلية BackgroundTasks في `FastAPI` تنفّذ عملاً بعد الردّ، لكنها **ليست نظام وظائف موثوقاً**:

- **تعيش داخل عملية الخادم:** إعادة تشغيل الـ API أو تعطّله تضيّع المهمة بلا أثر.
- **لا صف دائم:** لا سجل يمكن الاستعلام عنه لمعرفة الحالة أو التقدّم.
- **لا إعادة محاولة:** لا `retries` ولا استئناف بعد الفشل.
- **تنافس على الموارد:** تعمل داخل عملية الـ API فتنافس الطلبات على الـ CPU/GPU.
- **لا قابلية للتوسّع:** لا يمكن تشغيل عدّة عمّال على أجهزة مختلفة.

**القاعدة**

نستخدم BackgroundTasks لمهام صغيرة غير حرجة فقط (كتابة سجل)، والمعالجة الثقيلة في الـ worker المستقل.

## 11.5 الـ Worker المستقل وقائمة الوظائف المبنية على قاعدة البيانات

نستخدم **طابور وظائف مدعوماً بقاعدة البيانات** (`DB-backed queue`): الوظائف صفوف في processing_jobs والـ worker عملية منفصلة. الحجز بـ SELECT ... FOR UPDATE SKIP LOCKED يمنع عاملين من أخذ نفس الوظيفة، فنحقّق الموثوقية بلا `Redis` أو `Celery` (المستبعدين من الـ MVP).

## 11.6 `Frame sampling` كمعامل قابل للضبط

نحلّل إطاراً كل FRAME_SAMPLE_EVERY_N (الافتراضي 5): يوفّر وقتاً لكنه قد يُفوّت حدثاً قصيراً، فنضبط القيمة بعد التحقق من استرجاع الأحداث (`event recall`) ومدة الفيديو وسرعة العتاد، بحدّ أعلى عبر MAX_FRAMES_PER_JOB.

> **كيف نختار N؟**
>
> ابدأ بـ 5 على فيديو قصير، وقِس الزمن، ثم كبّر الرقم إن كان طويلاً أو صغّره إن فُقدت أحداث.

## 11.7 منع المعالجة المزدوجة ونفاد الموارد

- **قاعدة البيانات:** الفهرس الفريد الجزئي يمنع وظيفتين نشطتين لنفس الفيديو.
- **الحجز:** SKIP LOCKED يمنع عاملين من أخذ نفس الوظيفة.
- **الموارد:** الـ checkpoint يُحمَّل مرة واحدة، وWORKER_CONCURRENCY يحدّ الوظائف المتزامنة فنمنع عدّة `inference` ثقيلة معاً.
- **الفشل:** إعادة المحاولة أو failed حسب JOB_MAX_RETRIES، مع تنظيف الدليل الناقص.

## 11.8 نسبة التقدّم ورسائل الخطأ

progress نسبة بين frames_processed و frames_total، نحدّثه دورياً (لا كل إطار). الواجهة تستعلم عبر GET /api/v1/jobs/{job_id}، وعند الفشل نكتب سبباً واضحاً في error_message.

## 11.9 مخطط تسلسل معالجة الفيديو

| Frontend | API | PostgreSQL | Worker | AI Models |
|---|---|---|---|---|
| Frontend | POST /api/v1/videos/{video_id}/process | | | |
| API | INSERT processing_jobs (status=queued) | | | |
| API | 202 Accepted + JobOut | | | |
| Worker | SELECT ... FOR UPDATE SKIP LOCKED | | | |
| Worker | UPDATE status=running, worker_id | | | |
| Worker | open video file from storage | | | |
| AI Models | predict(frame) for sampled frames | | | |
| Worker | INSERT detections / incidents, save evidence | | | |
| Worker | UPDATE progress, frames_processed | | | |
| Worker | UPDATE status=completed; videos.status=processed | | | |
| Frontend | GET /api/v1/jobs/{job_id} (polling) | | | |
| API | 200 OK + JobOut | | | |

**شكل 11-1:** مخطط تسلسل معالجة الفيديو – الطلب متزامن سريع، والتنفيذ غير متزامن في الـ worker

## 11.10 التنفيذ العملي

نبدأ بخدمة الوظائف التي تُنشئ الوظيفة وتحجز التالية بأمان.

backend/app/services/job_service.py

```python
# backend/app/services/job_service.py
import uuid
from sqlalchemy import select
from sqlalchemy.orm import Session
from app.core.config import settings
from app.models.enums import JobStatus, VideoStatus
from app.models.processing_job import ProcessingJob
```

```python
from app.models.video import Video


class JobService:
    """Create and claim processing jobs (DB-backed queue)."""

    def __init__(self, db: Session) -> None:
        self.db = db

    def create_for_video(self, video_id: uuid.UUID) -> ProcessingJob:
        video = self.db.get(Video, video_id)
        if video is None:
            raise ValueError("video not found")
        job = ProcessingJob(video_id=video.id, status=JobStatus.queued,
                            sample_every_n_frames=settings.FRAME_SAMPLE_EVERY_N)
        video.status = VideoStatus.processing
        self.db.add(job)
        self.db.commit()
        self.db.refresh(job)
        return job

    def claim_next(self, worker_id: str) -> ProcessingJob | None:
        stmt = (select(ProcessingJob)
                .where(ProcessingJob.status == JobStatus.queued)
                .order_by(ProcessingJob.created_at)
                .with_for_update(skip_locked=True)
                .limit(1))
        job = self.db.execute(stmt).scalar_one_or_none()
        if job is None:
            return None
        job.status = JobStatus.running
        job.worker_id = worker_id
        self.db.commit()
        return job
```

python **backend/app/workers/processor.py**

```python
# backend/app/workers/processor.py
from pathlib import Path
import cv2
from sqlalchemy.orm import Session
from app.ai.model_registry import (
    get_ppe_detector, get_fire_smoke_detector, get_person_tracker)
from app.ai.rules import SafetyRuleEngine
from app.core.config import settings
from app.models.enums import JobStatus, VideoStatus
from app.services.incident_service import IncidentService


class VideoProcessor:
    """Runs the pipeline for a single job."""

    def __init__(self, db: Session) -> None:
        self.db = db
        self.rules = SafetyRuleEngine()

    def process(self, job) -> None:
        path = str(Path(settings.LOCAL_STORAGE_ROOT) / job.video.storage_key)
        capture = cv2.VideoCapture(path)
        frame_number = 0
        while True:
            ok, frame = capture.read()
            if not ok:
                break
            if frame_number % job.sample_every_n_frames == 0:
                timestamp = frame_number / (job.video.fps or 25.0)
                detections = self._infer(frame, frame_number, timestamp)
                for incident in self.rules.evaluate(detections):
                    IncidentService(self.db).persist(job, incident, frame)
                self._progress(job, frame_number)
            frame_number += 1
        capture.release()
        job.status = JobStatus.completed
        job.video.status = VideoStatus.processed
        self.db.commit()

    def _infer(self, frame, frame_number, timestamp) -> list:
        out = []
        out += get_ppe_detector().predict(frame, frame_number, timestamp)
        out += get_fire_smoke_detector().predict(frame, frame_number, timestamp)
        out += get_person_tracker().predict(frame, frame_number, timestamp)
```

```
        return out

    def _progress(self, job, frame_number: int) -> None:
        if job.frames_total:
            job.frames_processed = frame_number
            job.progress = round(100 * frame_number / job.frames_total, 2)
            self.db.commit()
```

python backend/app/workers/run_worker.py

```python
# backend/app/workers/run_worker.py
import time
import uuid
from app.core.config import settings
from app.db.session import SessionLocal
from app.models.enums import JobStatus
from app.services.job_service import JobService
from app.workers.processor import VideoProcessor

WORKER_ID = settings.WORKER_ID or f"worker-{uuid.uuid4().hex[:8]}"


def run_forever() -> None:
    while True:
        with SessionLocal() as db:
            job = JobService(db).claim_next(WORKER_ID)
            if job is None:
                time.sleep(settings.WORKER_POLL_INTERVAL_SECONDS)
                continue
            try:
                VideoProcessor(db).process(job)
            except Exception as exc:  # one job must not kill the worker
                job.status = JobStatus.failed
                job.error_message = str(exc)
                db.commit()


if __name__ == "__main__":
    run_forever()
```

python backend/app/core/config.py (pipeline excerpt)

```python
FRAME_SAMPLE_EVERY_N: int = 5
MAX_FRAMES_PER_JOB: int = 90000
WORKER_POLL_INTERVAL_SECONDS: int = 3
WORKER_ID: str = ""
WORKER_CONCURRENCY: int = 1
JOB_MAX_RETRIES: int = 1
```

**كيف نُشغّل الـ worker؟**

في نافذة طرفية منفصلة: python -m app.workers.run_worker، ويمكن تشغيل أكثر من نسخة على أجهزة مختلفة لأن الحجز عبر قاعدة البيانات آمن.

## 11.11 متى نحتاج Task Queue فعلاً؟

طابور قاعدة البيانات كافٍ لفريق طلابي وعدد محدود من العمّال على جهاز واحد. نحتاج **Redis** أو **Celery** فقط لعمّال على عدّة أجهزة، أو لجدولة مؤقتة وأولويات ومهام متكرّرة — وكلها خارج الـ MVP.

وإلغاء الوظيفة cancelled متاح فقط قبل بدء التنفيذ؛ فبعد الالتقاط نترك الـ worker يُنهي الإطار الحالي بأمان.

### خلاصة الفصل 11

**ما تعلّمناه**

لماذا نفصل التنفيذ عن الطلب، وكيف نبني طابور وظائف على قاعدة البيانات، ولماذا BackgroundTasks غير كافٍ، وكيف نمنع المعالجة المزدوجة ونضبط **frame sampling**.

**ما يجب تنفيذه**

- [ ] كتابة job_service.py و processor.py و run_worker.py.
- [ ] تنفيذ POST /api/v1/videos/{video_id}/process و GET /api/v1/jobs/{job_id}.

- ضبط FRAME_SAMPLE_EVERY_N و WORKER_POLL_INTERVAL_SECONDS.

**المخرجات المتوقعة**

- وظيفة تُنشأ وتُلتقط وتُكمل مع تحديث التقدّم.
- خطأ في وظيفة لا يوقف الـ worker ولا بقية الوظائف.

**كيف نتحقق من الإنجاز**

- إيقاف الـ API أثناء تشغيل وظيفة لا يضيّعها لأنها في قاعدة البيانات.
- محاولة إنشاء وظيفتين لنفس الفيديو تُرفض.
- إيقاف الـ worker ثم تشغيله يستأنف التقاط الوظائف المعلّقة.

**أخطاء شائعة**

- الاعتماد على BackgroundTasks لمعالجة ثقيلة.
- تحديث التقدّم في كل إطار فيُبطئ المعالجة.
- تشغيل عدّة عمّال ثقيلة على نفس الجهاز فتتضارب الموارد.
- عدم التقاط الأخطاء فيتوقّف الـ worker عند أول فيديو تالف.

# 12 قواعد السلامة وتوليد الوقائع

كيف يتحوّل الكشف الخام إلى واقعة لها معنى وقرار

الكشف وحده ليس واقعة. كشف no_helmet في إطار واحد قد يكون خطأً في الـ Model أو لحظة عابرة. دور SafetyRuleEngine أن يقرّر متى يصبح هذا الكشف واقعة تستحق المراجعة، ومنع التكرار، وتحديد الخطورة والدليل.

## 12.1 من الكشف الخام إلى الواقعة

نتتبّع رحلة واحدة: الـ Model يُخرج كشفاً خاماً ← يُطبَّع إلى Detection (الفصل 10) ← تُقيّمه القواعد مقابل عتبات وتأكيد زمني ← يُنتَج IncidentCandidate ← يُحفظ كواقعة مع دليل. هذا التسلسل يفصل "ماذا رأى الـ Model" عن "ما الذي يستحق أن يُسجَّل"، وهو قرار قواعد لا قرار شبكة عصبية.

## 12.2 المصطلحات التي يجب ألا تختلط

**جدول 12-1: الفرق بين المصطلحات السبعة**

| المصطلح | المعنى |
|---|---|
| `Detection` | صندوق واحد بثقة واسم صنف في إطار واحد |
| `Track` | سلسلة كشوفات لنفس الكائن عبر إطارات متتابعة بمعرّف ثابت `track_id` |
| `Rule violation` | تحقّق شرط القاعدة: كشف تجاوز العتبة واستمرّ بما يكفي |
| `Incident` | سجل محفوظ في جدول `incidents` له نوع وخطورة وحالة |
| `Evidence` | صورة أو مقطع مقتطع من الإطار يوثّق الواقعة |
| `Review` | قرار بشري (تأكيد/رفض) يُسجَّل في `incident_reviews` |
| `Notification` | تنبيه خارجي مستقبلي، خارج الـ MVP |

## 12.3 عتبات الثقة والتأكيد الزمني

لكل نوع عتبته الخاصة (PPE_CONF_THRESHOLD و FIRE_CONF_THRESHOLD و SMOKE_CONF_THRESHOLD). ولا يكفي تجاوز العتبة مرة واحدة: يجب أن يتكرّر الكشف لعدد إطارات متتالية (INCIDENT_CONFIRM_FRAMES) قبل توليد الواقعة. هذا "التأكيد الزمني" يقلّل الإنذارات الكاذبة الناتجة عن إطار شاذ واحد.

## 12.4 منع التكرار وفترات التهدئة

بعد توليد واقعة لنفس المسار (track_id) والنوع، نمنع توليد واقعة مكرّرة خلال فترة INCIDENT_COOLDOWN_SECONDS. المفتاح المستخدم في التتبّع الداخلي هو زوج (النوع، track_id)، فيصبح لكل عامل مسار واحد بدل عشرات الوقائع المتطابقة.

## 12.5 الربط بالـ Tracking ووقت البداية والنهاية

نربط الواقعة بالمسار عبر track_id، فيكون لدينا بداية (started_at_seconds) عند أول كشف مؤكَّد، ونهاية (ended_at_seconds) عند آخر كشف مؤكَّد للمسار نفسه. هذا يعطي الواقعة مدة واضحة، ويسمح بعرض المقطع الصحيح للمراجع.

## 12.6 قواعد معدات الوقاية (PPE)

إذا ظهر صنف no_helmet أو no_vest بثقة تتجاوز عتبة الـ PPE واستمرّ لعدد كافٍ من الإطارات، نولّد واقعة من نوع ppe_violation. نربطها بمسار الشخص إن وُجد، وإلا نكتفي بالصندوق والإطار.

## 12.7 قواعد الحرائق والدخان

صُنفا fire و smoke يُولّدان واقعة fire_smoke. لأن هذه الأحداث أخطر وأسرع، نستخدم عتبة مستقلة لكل صنف، ونعتبر ظهورهما بأعلى خطورة مقارنةً بمخالفات الـ PPE.

## 12.8 قواعد الاقتراب من المعدات: تقريب بالبكسل لا مسافة حقيقية

نقارن صندوق الشخص بصندوق المعدة أو بمنطقة خطر معرّفة (DANGER_ZONE_CONFIG) ونحسب نسبة تداخل (PROXIMITY_MIN_OVERLAP). إذا تجاوزت النسبة الحدّ، نولّد واقعة machine_proximity.

> **لا نتحدّث بالأمتار**
>
> هذا القياس **تقريبي بالبكسل** وليس مسافة حقيقية. بدون معايرة كاميرا (`calibration`) وبدون معرفة ارتفاع الكاميرا وزاوية المنظور، لا يمكن تحويل البكسل إلى متر. لذلك لا نكتب أبداً "المسافة 0.8 متر"، بل "تداخل بنسبة 18% من مساحة الصندوق". المعايرة تحسين مستقبلي لا جزء من الـ MVP.

## 12.9 تصنيف الخطورة واختيار الدليل

**جدول 12-2: اقتراح تصنيف الخطورة (قابل للضبط من الإعدادات)**

| النوع | الخطورة المقترحة | ملاحظة |
|---|---|---|
| `fire_smoke` | `high` | يُرفع إلى `critical` عند استمرار الحدث |
| `ppe_violation` | `medium` | قد يُرفع عند تكراره لنفس المسار |
| `machine_proximity` | `medium` | يُرفع عند تداخل كبير جداً |

الدليل هو صورة الإطار الذي تأكّدت فيه الواقعة، وتُحفظ بنوع image. نختار الإطار صاحب أعلى ثقة ضمن نافذة التأكيد، لا الإطار الأول فقط، ليكون الدليل أوضح للمراجع.

## 12.10 مثال عملي: واقعة مُولّدة من عدّة إطارات

ليكن INCIDENT_CONFIRM_FRAMES=3 و INCIDENT_COOLDOWN_SECONDS=30، ولنتبع شخصاً بمسار track_id=7 بلا خوذة. الجدول يوضّح قرار القاعدة في كل إطار.

**جدول 12-3: تتبّع القرار عبر الإطارات**

| الإطار | الزمن (ث) | الصنف | الثقة | قرار القاعدة |
|---|---|---|---|---|
| 100 | 4.0 | no_helmet | 0.62 | العدّاد 1 من 3 — لا واقعة بعد |
| 105 | 4.2 | no_helmet | 0.71 | العدّاد 2 من 3 — لا واقعة بعد |
| 110 | 4.4 | no_helmet | 0.80 | العدّاد 3 من 3 — توليد واقعة ppe_violation، بداية 4.0 ونهاية 4.4، الدليل إطار 110 |
| 115 | 4.6 | no_helmet | 0.83 | داخل فترة التهدئة — لا واقعة جديدة |
| 320 | 12.8 | no_helmet | 0.78 | العدّاد أُعيد؛ إن استمرّ 3 إطارات بعد انتهاء التهدئة تُولّد واقعة ثانية |

بهذا المنطق: الإطار الواحد لا يكفي، والواقعة لها بداية ونهاية ودليل واضح، والتكرار مُقيَّد بفترة تهدئة.

## 12.11 لماذا لا يتحوّل إطار واحد غير مؤكَّد إلى مخالفة خطيرة مؤكَّدة

لأن خطأ إطار واحد وارد في أي Model. لو اعتبرنا كل كشف واقعة مؤكَّدة، لغرق مسؤول السلامة في إنذارات كاذبة وفقد الثقة في النظام. لذلك نفصل بين ثلاث درجات: كشف خام (قد يكون خطأً)، وواقعة detected (تحتاج تأكيداً زمنياً)، وواقعة confirmed (بعد مراجعة بشرية فقط، الفصل 13). النظام لا يعلن "مخالفة خطيرة مؤكَّدة" من تلقاء نفسه أبداً.

## 12.12 التنفيذ: SafetyRuleEngine

backend/app/ai/rules.py

```python
# backend/app/ai/rules.py
from dataclasses import dataclass
from app.ai.base import Detection
from app.core.config import settings


@dataclass
class IncidentCandidate:
    incident_type: str
    severity: str
    class_name: str
    confidence: float
    frame_number: int
    started_at_seconds: float
    ended_at_seconds: float | None
```

```
    track_id: int | None
    zone_name: str | None
    detection_count: int
    summary: str


def overlap_ratio(a, b) -> float:
    """Fraction of box b covered by box a (pixel-based, approximate)."""
    ax1, ay1, ax2, ay2 = a
    bx1, by1, bx2, by2 = b
    ix1, iy1 = max(ax1, bx1), max(ay1, by1)
    ix2, iy2 = min(ax2, bx2), min(ay2, by2)
    if ix2 <= ix1 or iy2 <= iy1:
        return 0.0
    inter = (ix2 - ix1) * (iy2 - iy1)
    area_b = (bx2 - bx1) * (by2 - by1)
    return inter / area_b if area_b else 0.0


class SafetyRuleEngine:
    """Turns raw detections into incident candidates."""

    def __init__(self) -> None:
        self._hits: dict[tuple, int] = {}
        self._last: dict[tuple, float] = {}

    def evaluate(self, detections: list[Detection]) -> list[IncidentCandidate]:
        out: list[IncidentCandidate] = []
        for det in detections:
            rule = self._classify(det.class_name)
            if rule is None or det.confidence < self._threshold(det.class_name):
                continue
            key = (rule, det.track_id)
            self._hits[key] = self._hits.get(key, 0) + 1
            if self._hits[key] < settings.INCIDENT_CONFIRM_FRAMES:
                continue  # not enough evidence yet
            if det.timestamp_seconds - self._last.get(key, -1e9) < settings.INCIDENT_COOLDOWN_SECONDS:
                continue  # cooldown to avoid duplicates
            self._last[key] = det.timestamp_seconds
            hits = self._hits[key]
            self._hits[key] = 0
            out.append(self._build(det, rule, hits))
        return out

    def _classify(self, name: str) -> str | None:
        if name in ("no_helmet", "no_vest"):
            return "ppe_violation"
        if name in ("fire", "smoke"):
            return "fire_smoke"
        return None

    def _threshold(self, name: str) -> float:
        if name in ("no_helmet", "no_vest"):
            return settings.PPE_CONF_THRESHOLD
        if name == "fire":
            return settings.FIRE_CONF_THRESHOLD
        if name == "smoke":
            return settings.SMOKE_CONF_THRESHOLD
        return settings.DETECTION_CONF_THRESHOLD

    def _build(self, det, rule, hits) -> IncidentCandidate:
        severity = "high" if rule == "fire_smoke" else "medium"
        return IncidentCandidate(
            incident_type=rule, severity=severity, class_name=det.class_name,
            confidence=det.confidence, frame_number=det.frame_number,
            started_at_seconds=det.timestamp_seconds, ended_at_seconds=det.timestamp_seconds,
            track_id=det.track_id, zone_name=None, detection_count=hits,
            summary=f"{rule} detected as {det.class_name}")
```

python **backend/app/services/incident_service.py**

```
# backend/app/services/incident_service.py (excerpt)
from sqlalchemy.orm import Session
from app.ai.rules import IncidentCandidate
from app.models.enums import (
    EvidenceKind, IncidentSeverity, IncidentStatus, IncidentType)
from app.models.evidence import Evidence
from app.models.incident import Incident
from app.services.storage_service import save_evidence_image
```

```python
class IncidentService:
    """Persists incident candidates and their evidence."""

    def __init__(self, db: Session) -> None:
        self.db = db

    def persist(self, job, candidate: IncidentCandidate, frame) -> Incident:
        incident = Incident(
            video_id=job.video_id, job_id=job.id,
            incident_type=IncidentType(candidate.incident_type),
            severity=IncidentSeverity(candidate.severity),
            status=IncidentStatus.detected,
            started_at_seconds=candidate.started_at_seconds,
            ended_at_seconds=candidate.ended_at_seconds,
            frame_number=candidate.frame_number, track_id=candidate.track_id,
            confidence=candidate.confidence, class_name=candidate.class_name,
            zone_name=candidate.zone_name, summary=candidate.summary,
            detection_count=candidate.detection_count)
        self.db.add(incident)
        self.db.flush()  # obtain incident.id for the evidence row
        key = save_evidence_image(frame, incident.id, candidate.frame_number)
        self.db.add(Evidence(
            incident_id=incident.id, kind=EvidenceKind.image, storage_key=key,
            content_type="image/jpeg", frame_number=candidate.frame_number,
            timestamp_seconds=candidate.started_at_seconds))
        self.db.commit()
        return incident
```

## خلاصة الفصل 12

### ما تعلّمناه

كيف تتحوّل الكشوفات الخام إلى وقائع عبر عتبات وتأكيد زمني ومنع تكرار، وكيف نصنّف الخطورة ونختار الدليل، ولماذا الاقتراب تقريب بالبكسل لا مسافة بالأمتار.

### ما يجب تنفيذه

- كتابة app/ai/rules.py بقواعد PPE والحرائق والاقتراب.
- كتابة incident_service.py لحفظ الواقعة والدليل.
- ضبط INCIDENT_CONFIRM_FRAMES و INCIDENT_COOLDOWN_SECONDS و PROXIMITY_MIN_OVERLAP.

### المخرجات المتوقعة

- واقعة بحالة detected لها بداية ونهاية ودليل صورة.
- لا وقائع مكرّرة لنفس المسار داخل فترة التهدئة.

### كيف نتحقق من الإنجاز

- كشف واحد في إطار واحد لا يُولّد واقعة.
- ثلاثة كشوفات متتالية تُولّد واقعة واحدة.
- لا يوجد أي نصّ يذكر مسافة بالأمتار.

### أخطاء شائعة

- توليد واقعة من كل كشف فيُغرق النظام بالتكرار.
- إعلان حالة confirmed آلياً بدل تركها للمراجعة.
- تحويل نسبة التداخل إلى مسافة بالأمتار دون معايرة.
- استخدام عتبة عامة واحدة لكل الأنواع.

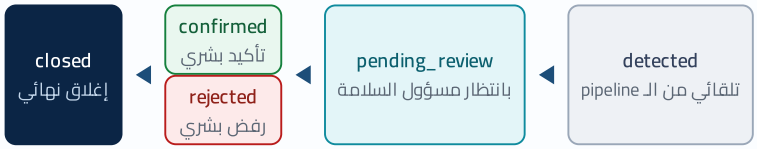
# 13 دورة حياة الواقعة والمراجعة البشرية

detected ثم pending_review ثم confirmed أو rejected ثم closed

الواقعة لا تنتهي عند كشفها. لها دورة حياة واضحة تنقلها من حالة تلقائية إلى قرار بشري موثّق. هذا الفصل يوضّح هذه الدورة، وما يفعله مسؤول السلامة، ولماذا نسجّل كل إجراء.

## 13.1 دورة حياة الواقعة

خمس حالات فقط، وكل انتقال مسموح مذكور صراحةً. الـ pipeline ينشئ الواقعة بحالة detected، ثم تُصبح pending_review في طابور المراجعة، ثم يقرّر المسؤول فتنتقل إلى confirmed أو rejected، وأخيراً إلى closed.




**شكل 13-1:** دورة حياة الواقعة – الانتقالات المسموحة فقط

**جدول 13-1: الانتقالات المسموحة وسبب رفض غيرها**

| الانتقال | الإجراء المُسجَّل | القاعدة |
|---|---|---|
| `detected → pending_review` | — | تلقائي عند دخول الواقعة طابور المراجعة |
| `pending_review → confirmed` | `confirm` | مراجعة بشرية بعد فحص الدليل |
| `pending_review → rejected` | `reject` | مراجعة بشرية: إنذار كاذب |
| `confirmed → closed` | `close` | انتهاء الإجراء المطلوب |
| `rejected → closed` | `close` | إغلاق بعد الرفض |

**انتقال غير مسموح**

لا يمكن الانتقال من detected مباشرةً إلى confirmed، ولا إعادة فتح واقعة closed. أي محاولة كذلك تُرفض بحالة 409.

## 13.2 ماذا يفعل مسؤول السلامة

- يفتح قائمة الوقائع GET /api/v1/incidents ويفلترها بالنوع والخطورة والحالة والتاريخ.
- يفتح الواقعة {GET /api/v1/incidents/{incident_id فيرى الدليل والزمن (started_at_seconds) والفيديو المصدر.
- يشاهد صورة الدليل GET /api/v1/evidence/{evidence_id}/file.
- يؤكّد أو يرفض أو يضيف ملاحظة عبر {PATCH /api/v1/incidents/{incident_id.
- يبحث تاريخياً عن المخالفات المتكرّرة لنفس الموقع أو المسار.

في وضع العرض التوضيحي (AUTH_ENABLED=false) يتصرّف النظام كأن مستخدم safety_officer افتراضي هو المراجع، ويكون reviewer_id قابلاً لأن يكون NULL. عند تفعيل المصادقة يُسجَّل معرّف المستخدم الفعلي.

## 13.3 لماذا نسجّل كل إجراء (Auditability)

كل تغيير حالة يُلحق صفاً جديداً في incident_reviews يحمل الفاعل والحالة السابقة والجديدة والملاحظة. هذا يعطي: تتبّعاً كاملاً (من فعل ماذا ومتى)، ومسؤولية واضحة، وإمكانية إعادة بناء تاريخ الواقعة. لا نعدّل الصفوف القديمة ولا نحذفها أبداً.

وبالاستعلام عن incident_reviews نعرف من أكّد كل واقعة ومتى، وهو ما تحتاجه أي لجنة تدقيق لاحقاً. أما البحث التاريخي في جدول incidents (بالنوع والخطورة والحالة ونطاق التاريخ) فيعتمد على الفهارس المذكورة في الفصل 8.

## 13.4 مثال كامل: طلب المراجعة وتحديث القاعدة وسلوك الواجهة

يؤكّد مسؤول السلامة واقعة. الطلب التالي يغيّر الحالة ويُلحق صف مراجعة.

http request.json

```
PATCH /api/v1/incidents/3f2a1c09-7b44-4e0a-9c11-2d6f0a8b1c9d HTTP/1.1
Content-Type: application/json

{
  "status": "confirmed",
  "note": "confirmed after reviewing the evidence image",
  "reviewer_id": "8b1d4e77-2c10-4f88-9a03-51c9d2e6f042"
}
```

json response.json

```
{
  "id": "3f2a1c09-7b44-4e0a-9c11-2d6f0a8b1c9d",
  "video_id": "a1b2c3d4-0000-4000-8000-000000000001",
  "job_id": "d4e5f6a7-0000-4000-8000-000000000002",
  "incident_type": "ppe_violation",
  "severity": "medium",
  "status": "confirmed",
  "started_at_seconds": 4.0,
  "ended_at_seconds": 4.4,
  "frame_number": 110,
  "track_id": 7,
  "confidence": 0.8,
  "class_name": "no_helmet",
  "zone_name": null,
  "summary": "ppe_violation detected as no_helmet",
  "detection_count": 3,
  "created_at": "2026-01-01T10:00:00Z",
  "updated_at": "2026-01-01T10:05:00Z",
  "evidence": [
    {
      "id": "9c0e1a55-0000-4000-8000-000000000003",
      "incident_id": "3f2a1c09-7b44-4e0a-9c11-2d6f0a8b1c9d",
      "kind": "image",
      "storage_key": "evidence/3f2a1c09/frame_110.jpg",
      "content_type": "image/jpeg",
      "size_bytes": 42133,
      "frame_number": 110,
      "timestamp_seconds": 4.4,
      "file_url": "/api/v1/evidence/9c0e1a55/file"
    }
  ]
}
```

على الـ Frontend: بعد الردّ الناجح تُحدَّث حالة الواقعة في الجدول إلى confirmed، ويظهر صف المراجعة الجديد في سجلّها، ويُزاح العنصر من قائمة pending_review.

python backend/app/api/v1/endpoints/incidents.py

```python
# backend/app/api/v1/endpoints/incidents.py (PATCH excerpt)
import uuid
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from app.db.session import get_db
from app.models.enums import IncidentStatus, ReviewAction
from app.models.incident import Incident
from app.models.incident_review import IncidentReview
from app.schemas.incident import IncidentOut, IncidentPatch

router = APIRouter(prefix="/incidents", tags=["incidents"])

# target status -> (required current status, audit action)
TRANSITIONS = {
    IncidentStatus.confirmed: (IncidentStatus.pending_review, ReviewAction.confirm),
    IncidentStatus.rejected: (IncidentStatus.pending_review, ReviewAction.reject),
    IncidentStatus.closed: (IncidentStatus.confirmed, ReviewAction.close),
}


@router.patch("/{incident_id}", response_model=IncidentOut)
def patch_incident(incident_id: uuid.UUID, payload: IncidentPatch,
                   db: Session = Depends(get_db)) -> IncidentOut:
    incident = db.get(Incident, incident_id)
    if incident is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "incident not found")

    new_status = IncidentStatus(payload.status) if payload.status else incident.status
```

```
    if new_status ≠ incident.status and new_status not in TRANSITIONS:
        raise HTTPException(status.HTTP_409_CONFLICT, "status cannot be set directly")
    if new_status ≠ incident.status:
        required, action = TRANSITIONS[new_status]
        if incident.status ≠ required:
            raise HTTPException(status.HTTP_409_CONFLICT, "invalid status transition")
        db.add(IncidentReview(
            incident_id=incident.id, reviewer_id=payload.reviewer_id, action=action,
            previous_status=incident.status, new_status=new_status, note=payload.note))
        incident.status = new_status
    elif payload.note:
        db.add(IncidentReview(
            incident_id=incident.id, reviewer_id=payload.reviewer_id,
            action=ReviewAction.comment, previous_status=incident.status,
            new_status=incident.status, note=payload.note))

    db.commit()
    db.refresh(incident)
    return IncidentOut.model_validate(incident)
```

python backend/app/models/incident_review.py

```python
# backend/app/models/incident_review.py
import uuid
from datetime import datetime
from sqlalchemy import DateTime, Enum, ForeignKey, Text, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.db.base import Base
from app.models.enums import IncidentStatus, ReviewAction


class IncidentReview(Base):
    __tablename__ = "incident_reviews"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    incident_id: Mapped[uuid.UUID] = mapped_column(
        ForeignKey("incidents.id", ondelete="CASCADE"), nullable=False)
    reviewer_id: Mapped[uuid.UUID | None] = mapped_column(ForeignKey("users.id"))
    action: Mapped[ReviewAction] = mapped_column(
        Enum(ReviewAction, name="review_action"), nullable=False)
    previous_status: Mapped[IncidentStatus | None] = mapped_column(
        Enum(IncidentStatus, name="incident_status"))
    new_status: Mapped[IncidentStatus | None] = mapped_column(
        Enum(IncidentStatus, name="incident_status"))
    note: Mapped[str | None] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

    incident = relationship("Incident", back_populates="reviews")
```

## 13.5 الكشف الآلي يساعد المراجعة ولا يحلّ محل الحكم المهني

**حدّ أساسي في النظام**

النظام أداة فرز ومساعدة، وليس بديلاً عن مسؤول السلامة. الكشف الآلي قد يُخطئ (إنذار كاذب) وقد يُفوّت (حدث مفقود)، والقرار النهائي دائماً بشري. لذلك لا تنتقل أي واقعة إلى confirmed تلقائياً، ولا يُبنى أي إجراء تأديبي على مخرجات الـ Model وحدها. الواجهة تعرض دائماً عبارة واضحة بأن النتيجة كشف آلي يحتاج مراجعة.

عملياً، تعرض الواجهة شارة "كشف آلي" بجانب كل واقعة، وتطلب من المراجع فحص الدليل قبل أي قرار. هذا التصميم يحمي الفريق من الادّعاء بأن النظام يحكم على العاملين بدل أن يساعدهم.

### خلاصة الفصل 13

**ما تعلّمناه**

دورة حياة الواقعة بانتقالاتها الخمس، وما يفعله مسؤول السلامة، ولماذا نسجّل كل إجراء في incident_reviews، وأن الكشف الآلي يدعم المراجعة ولا يستبدلها.

**ما يجب تنفيذه**

- كتابة incident_review.py وجدول incident_reviews.
- تنفيذ PATCH /api/v1/incidents/{incident_id} مع التحقق من الانتقالات.

- إظهار الدليل والزمن والفيديو المصدر في صفحة الواقعة.

**المخرجات المتوقعة**

- واقعة قابلة للتأكيد أو الرفض مع صف مراجعة لكل إجراء.
- سجل تدقيق كامل قابل للعرض.

**كيف نتحقق من الإنجاز**

- تأكيد واقعة يُلحق صفاً بالإجراء confirm والحالة الجديدة confirmed.
- محاولة انتقال غير مسموح تُرجع 409.
- لا توجد واقعة تُصبح confirmed بدون صف مراجعة.

**أخطاء شائعة**

- تأكيد الوقائع آلياً من الـ pipeline.
- تعديل أو حذف صفوف المراجعة القديمة.
- السماح بانتقالات حالة غير معرّفة.
- تقديم الكشف الآلي كحكم نهائي بدل دعم القرار البشري.

CH14 · FRONTEND ARCHITECTURE

# 14 معمارية الـ Frontend بـ React و TypeScript

بنية قابلة للصيانة، هوية بصرية صناعية، وتجربة استخدام واضحة

هذا الفصل يضع هيكل الواجهة قبل كتابة مكوّناتها: تقسيم الكود، وتنظيم المسارات، ومطابقة أنواع **TypeScript** مع عقد الـ API حرفياً، وتوضيح الشاشات العشر قبل تنفيذها.

## 14.1 بنية تطبيق React

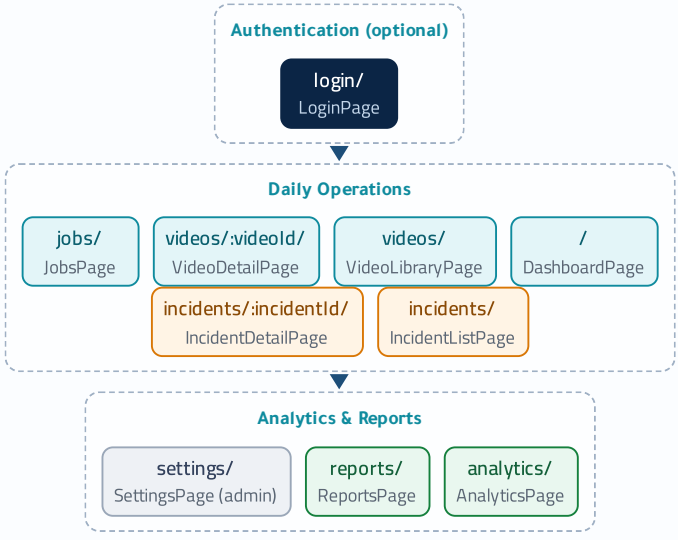
نقسّم src حسب **الميزة (feature)** لا حسب نوع الملف فقط، فيصبح كل جزء مسؤولاً عن نفسه:

**جدول 14-1: طبقات الـ Frontend ومسؤولية كل طبقة**

| المجلّد | المسؤولية |
|---|---|
| `api/` | عميل Axios ونداءات كل مورد؛ المكان الوحيد الذي يعرف عناوين المسارات |
| `hooks/` | منطق مشترك: الترقيم، الاستطلاع، الجلب، التنبيهات |
| `types/` | أنواع مطابقة لعقد الـ API، تُستورد في كل مكان |
| `features/` | مكوّنات وصفحات كل ميزة: `videos, incidents, analytics, reports` |
| `components/` | مكوّنات عامة: `Button, Table, Modal, Toast, EmptyState` |
| `layouts/` | الهيكل العام: الشريط الجانبي والرأس ومنطقة المحتوى |
| `routes.tsx` | تعريف المسارات وربطها بالصفحات وحمايتها بالمصادقة |

## 14.2 خريطة الصفحات والمسارات

نستخدم react-router-dom v6 في ثلاثة أقسام: المصادقة (اختياري)، التشغيل اليومي، والتحليل والتقارير. المسارات المحمية تُحوَّل إلى login/ عند VITE_ENABLE_AUTH=true.




**شكل 14-1:** خريطة صفحات الـ Frontend ومساراتها

## 14.3 وصف الصفحات العشر

لكل صفحة غرض واحد ونداءات محدّدة وسلوك تحميل/خطأ صريح:

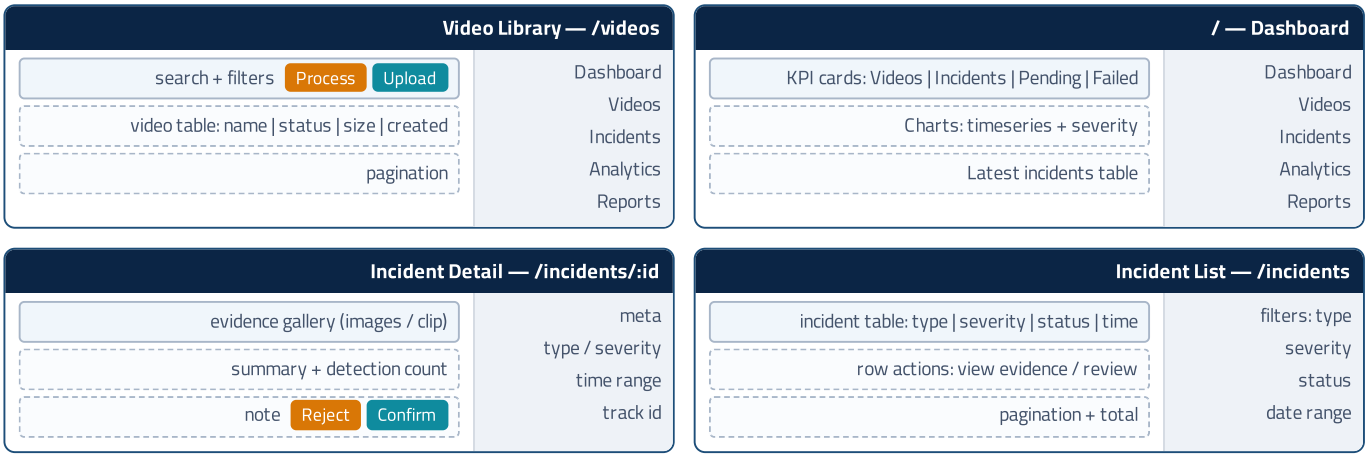
**جدول 14-2: الصفحات العشر ونداءاتها وسلوكها**

| الصفحة / المسار | الغرض والمكوّنات | نداءات الـ API | إجراءات المستخدم وسلوك التحميل/الخطأ |
|---|---|---|---|
| `/login` | دخول آمن (اختياري) مع نموذج بريد وكلمة مرور | `POST /auth/login` | تسجيل دخول، تخزين التوكن؛ خطأ 401 يظهر رسالة عربية |
| `/` | Dashboard: بطاقات KPI، أحدث الوقائع، حالة الـ Jobs | `GET /analytics/overview` | فتح واقعة، الانتقال للأقسام؛ هيكل تحميل عند الجلب |

| الصفحة / المسار | الغرض والمكوّنات | نداءات الـ API | إجراءات المستخدم وسلوك التحميل/الخطأ |
|---|---|---|---|
| `/videos` | Video Library: جدول، رفع، بحث وفلترة، إعادة معالجة | `GET /videos, POST /videos, POST /videos/{id}/process` | رفع ملف، بدء معالجة، حذف؛ شريط تقدّم الرفع |
| `/videos/:videoId` | تفاصيل فيديو + أحدث job + وقائعه | `GET /videos/{id}, GET /incidents?video_id=` | متابعة التقدّم، الانتقال لواقعة؛ 404 صفحة غير موجودة |
| `/jobs` | سجل الـ Jobs وحالاتها وأخطاؤها | `GET /jobs` | فلترة بالحالة؛ عرض `error_message` |
| `/incidents` | جدول الوقائع مع فلترة ونطاق تاريخ وترقيم | `GET /incidents` | فلترة النوع/الخطورة/الحالة، تنقّل الصفحات؛ حالة فارغة واضحة |
| `/incidents/:incidentId` | تفاصيل الواقعة + معرض الدليل + المراجعة | `GET /incidents/{id}, PATCH /incidents/{id}` | تأكيد/رفض/إغلاق + ملاحظة؛ تحديث فوري بعد الـ PATCH |
| `/analytics` | اتجاهات زمنية وتوزيعات حسب النوع والخطورة | `GET /analytics/overview, GET /analytics/timeseries` | تغيير النطاق الزمني؛ رسوم Recharts |
| `/reports` | ملخص يومي/أسبوعي وتصدير CSV | `GET /reports/daily, GET /reports/export` | اختيار التاريخ، تنزيل الملف؛ زر معطّل أثناء التنزيل |
| `/settings` | حالة النظام والإعدادات (admin) | `GET /health` | عرض حالة المكوّنات؛ للقراءة فقط في الـ MVP |

## 14.4 Wireframes للصفحات الرئيسية

رسومات مصغّرة تحدّد التخطيط قبل التنفيذ؛ والشريط الجانبي يتحوّل إلى قائمة منسدلة على الشاشات الصغيرة.




## 14.5 أنواع TypeScript المشتركة

هذا الملف هو الجسر بين عقد الـ API والواجهة. كل واجهة هنا تطابق كائن رد في SPEC.md بنفس أسماء الحقول؛ أي اختلاف حرف واحد خطأ يجب إصلاحه.

frontend/src/types/api.ts

```typescript
// Mirrors SPEC.md response shapes exactly. Do not rename fields.

export type VideoStatus = "uploaded" | "processing" | "processed" | "failed";
export type JobStatus = "queued" | "running" | "completed" | "failed" | "cancelled";
export type IncidentType = "ppe_violation" | "fire_smoke" | "machine_proximity";
export type Severity = "low" | "medium" | "high" | "critical";
export type IncidentStatus =
  | "detected" | "pending_review" | "confirmed" | "rejected" | "closed";
export type EvidenceKind = "image" | "clip" | "metadata";

export interface JobOut {
  id: string;
  video_id: string;
  status: JobStatus;
  progress: number;
  frames_total: number | null;
  frames_processed: number;
  sample_every_n_frames: number;
  started_at: string | null;
  finished_at: string | null;
  error_message: string | null;
  created_at: string;
```

```
}

export interface VideoOut {
  id: string;
  original_filename: string;
  stored_filename: string;
  content_type: string;
  size_bytes: number;
  duration_seconds: number | null;
  fps: number | null;
  width: number | null;
  height: number | null;
  frame_count: number | null;
  status: VideoStatus;
  created_at: string;
  updated_at: string;
  latest_job: JobOut | null;
}

export interface EvidenceOut {
  id: string;
  incident_id: string;
  kind: EvidenceKind;
  storage_key: string;
  content_type: string | null;
  size_bytes: number | null;
  frame_number: number | null;
  timestamp_seconds: number | null;
  file_url: string;
}

export interface IncidentOut {
  id: string;
  video_id: string;
  job_id: string | null;
  incident_type: IncidentType;
  severity: Severity;
  status: IncidentStatus;
  started_at_seconds: number;
  ended_at_seconds: number | null;
  frame_number: number | null;
  track_id: number | null;
  confidence: number | null;
  class_name: string | null;
  zone_name: string | null;
  summary: string | null;
  detection_count: number;
  created_at: string;
  updated_at: string;
  evidence: EvidenceOut[];
}

export interface IncidentPatch {
  status?: IncidentStatus;
  note?: string;
  reviewer_id?: string;
}

export interface AnalyticsOverview {
  total_videos: number;
  processed_videos: number;
  total_incidents: number;
  incidents_by_type: Record<string, number>;
  incidents_by_severity: Record<string, number>;
  incidents_by_status: Record<string, number>;
  pending_review: number;
  jobs_running: number;
  jobs_failed: number;
  last_incident_at: string | null;
}

export interface TimeseriesPoint {
  date: string; // YYYY-MM-DD
  ppe_violation: number;
  fire_smoke: number;
  machine_proximity: number;
  total: number;
}

export interface Paginated<T> {
  items: T[];
  total: number;
```

```
  page: number;
  page_size: number;
  pages: number;
}
```

## 14.6 شجرة frontend/src الكاملة

المرجع الحرفي لأماكن الملفات، ويطابق أسماء الفصل التالي:

frontend/src (tree) — text

```
frontend/src/
  main.tsx                # app bootstrap, router provider
  App.tsx                 # layout shell + route outlet
  routes.tsx              # route table and auth guard
  index.css               # tailwind entry + design tokens
  vite-env.d.ts
  api/
    client.ts             # axios instance, base URL, interceptors
    videos.ts             # list/upload/process/delete calls
    incidents.ts          # list/detail/patch/evidence calls
    analytics.ts          # overview + timeseries calls
    reports.ts            # daily rows + CSV download
  hooks/
    usePolling.ts         # generic interval polling hook
    useJobStatus.ts       # poll one job until terminal state
    useDebounce.ts
  types/
    api.ts                # shared API types (see 14.5)
  components/
    Button.tsx  Table.tsx  Modal.tsx  Toast.tsx
    EmptyState.tsx  ErrorState.tsx  Spinner.tsx  StatusBadge.tsx
  layouts/
    AppLayout.tsx         # sidebar + header + content
    AuthGuard.tsx
  features/
    videos/
      VideoLibraryPage.tsx  VideoDetailPage.tsx
      UploadVideoForm.tsx   VideoTable.tsx
    incidents/
      IncidentListPage.tsx   IncidentDetailPage.tsx
      IncidentTable.tsx      IncidentFilters.tsx  EvidenceGallery.tsx
    analytics/
      AnalyticsPage.tsx      OverviewCards.tsx
      IncidentsTimeseries.tsx  SeverityPie.tsx
    reports/
      ReportsPage.tsx        ExportCsvButton.tsx
    auth/
      LoginPage.tsx          useAuth.ts
```

## 14.7 الحالات والتجربة والهوية البصرية

لكل قائمة أو بطاقة ثلاث حالات صريحة بمكوّنات موحّدة:

**جدول 14-3: الحالات الثلاث ومكوّناتها**

| الحالة | المكوّن | القاعدة |
|---|---|---|
| Loading | `Spinner` | هيكل ثابت يمنع اهتزاز الصفحة أثناء الجلب |
| Error | `ErrorState` | رسالة عربية + زر إعادة المحاولة، دون كشف تفاصيل تقنية |
| Empty | `EmptyState` | نص يشرح الخطوة التالية ("ارفع فيديو للبدء") بدل جدول فارغ |

- **النماذج (`forms`):** تحقق قبل الإرسال وزر معطّل أثناءه لمنع الطلب المزدوج.
- **الإشعارات (`Toast`):** تأكيد نجاح/فشل الإجراء من سياق واحد أعلى الشجرة.
- **حالة المصادقة:** تتبع VITE_ENABLE_AUTH؛ عند false نتجاوز الدخول، وعند true نحمي المسارات ونرسل التوكن.
- **التجاوب وإمكانية الوصول:** تخطيط شبكي، تباين كافٍ، نصوص بديلة، وتنقّل بلوحة المفاتيح.

**الهوية البصرية:** `Dark Navy` للبنية، `Teal` للمعلومات، `Orange` للتحذير، `Red` للخطر، وخلفيات محايدة؛ بلا تدرجات أو حركات زائدة. ألوان الخطورة تتبع الـ enum: low, medium, high, critical.

**لا روابط إنتاج داخل المكوّنات**

العنوان يأتي دائماً من import.meta.env.VITE_API_BASE_URL عبر عميل واحد في api/client.ts، ولا يُكتب أي عنوان داخل مكوّن.

## خلاصة الفصل 14

### ما تعلّمناه

قسّمنا الـ Frontend حسب الميزة، ورسمنا خريطة الصفحات، ووصفنا الشاشات العشر، وأثبتنا أنواع TypeScript المطابقة للـ API، وثبّتنا حالات التحميل/الخطأ/الفراغ والهوية البصرية.

### ما يجب تنفيذه

- إنشاء مجلّدات features/hooks/api/types كما في الشجرة.
- نسخ types/api.ts حرفياً وربطه بعميل الـ API.
- بناء المكوّنات العامة (Button, Table, EmptyState, ErrorState, Spinner).
- تعريف المسارات مع حارس مصادقة يقرأ VITE_ENABLE_AUTH.

### المخرجات المتوقعة

- هيكل frontend/src كامل وفارغ من الأخطاء النوعية.
- ملف أنواع واحد يطابق SPEC دون اختلاف.
- تخطيط عام (Layout) يعمل على الشاشات المختلفة.

### كيف نتحقق من الإنجاز

- npm run build ينجح دون أخطاء TypeScript.
- كل صفحة من العشر لها مسار ومكوّن محدَّد.
- لا يوجد عنوان API مكتوب داخل أي مكوّن.

### أخطاء شائعة

- خلط أنواع الواجهة بأسماء حقول مختلفة عن الـ API.
- كتابة عناوين الـ Backend داخل المكوّنات.
- إهمال حالة الفراغ فيظهر جدول فارغ بلا شرح.
- مبالغة في الحركات والتدرجات فتشتّت المستخدم.

CH15 · DASHBOARD & API WIRING

# 15 بناء لوحة التحكم وربطها بالـ API

من أول نداء Axios إلى رسم بياني يعتمد على بيانات حقيقية

هذا الفصل يحوّل التصميم إلى شاشات تعمل: طبقة نداءات واحدة، ثم ربط الرفع والجداول والرسوم وتنزيل التقارير. القاعدة الحاكمة: **كل رقم يظهر للمستخدم يأتي من استجابة API حقيقية**، ولا يوجد رقم مكتوب في الكود.

## 15.1 عميل الـ API وإعداد الـ Base URL

عميل واحد (api/client.ts) يقرأ العنوان من VITE_API_BASE_URL ويضيف التوكن تلقائياً عند VITE_ENABLE_AUTH=true (مقابل AUTH_ENABLED في الـ Backend).

frontend/src/api/client.ts

```typescript
import axios from "axios";

const baseURL = import.meta.env.VITE_API_BASE_URL ?? "http://localhost:8000";
export const AUTH_ENABLED = import.meta.env.VITE_ENABLE_AUTH === "true";

export const api = axios.create({ baseURL, timeout: 30000 });

api.interceptors.request.use((config) => {
  if (AUTH_ENABLED) {
    const token = localStorage.getItem("safevision_token");
    if (token) config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

**CORS**

المتصفح يمنع الاتصال بمنفذ مختلف إلا بإذن الخادم، لذا نضع عنوان الواجهة في CORS_ORIGINS (مثلاً http://localhost:5173). خطأ CORS سببه هذه القائمة لا كود React.

## 15.2 طبقة نداءات الفيديو

كل نداءات الفيديو في ملف واحد؛ الصفحات تستدعي دوالّ واضحة ولا تلمس Axios مباشرة:

frontend/src/api/videos.ts

```typescript
import { api } from "./client";
import type { JobOut, Paginated, VideoOut } from "../types/api";

export interface VideoListParams {
  page?: number;
  page_size?: number;
  status?: string;
  sort?: string;
}

export async function listVideos(params: VideoListParams = {}) {
  const { data } = await api.get<Paginated<VideoOut>>("/api/v1/videos", { params });
  return data;
}

export async function getVideo(id: string) {
  const { data } = await api.get<VideoOut>(`/api/v1/videos/${id}`);
  return data;
}

export async function uploadVideo(file: File, onProgress?: (p: number) => void) {
  const form = new FormData();
  form.append("file", file);
  const { data } = await api.post<VideoOut>("/api/v1/videos", form, {
    onUploadProgress: (e) => {
      if (onProgress && e.total) onProgress(Math.round((e.loaded / e.total) * 100));
    },
  });
  return data;
}
```

```
export async function processVideo(id: string) {
  const { data } = await api.post<JobOut>(`/api/v1/videos/${id}/process`);
  return data;
}

export async function deleteVideo(id: string) {
  await api.delete(`/api/v1/videos/${id}`);
}
```

## 15.3 الرفع مع FormData وشريط التقدّم

الرفع يستخدم **FormData** لأن الـ endpoint يستقبل multipart/form-data: تحقق قبل الإرسال، زر معطّل أثناءه، وتقدّم من onUploadProgress.

tsx — **frontend/src/features/videos/UploadVideoForm.tsx**

```
import { useState } from "react";
import { uploadVideo } from "../../api/videos";
import type { VideoOut } from "../../types/api";

const ALLOWED = ["mp4", "mov", "avi", "mkv"];

export function UploadVideoForm({ onUploaded }: { onUploaded: (v: VideoOut) => void }) {
  const [progress, setProgress] = useState(0);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function handleFile(file: File) {
    const ext = file.name.split(".").pop()?.toLowerCase() ?? "";
    if (!ALLOWED.includes(ext)) {
      setError("Unsupported video format");
      return;
    }
    setBusy(true);
    setError(null);
    try {
      const video = await uploadVideo(file, setProgress);
      onUploaded(video);
      setProgress(0);
    } catch {
      setError("Upload failed, please retry");
    } finally {
      setBusy(false);
    }
  }

  return (
    <div>
      <input
        type="file"
        accept=".mp4,.mov,.avi,.mkv"
        disabled={busy}
        onChange={(e) => e.target.files?.[0] && handleFile(e.target.files[0])}
      />
      {busy && <progress value={progress} max={100} />}
      {error && <p role="alert">{error}</p>}
    </div>
  );
}
```

## 15.4 نداءات الوقائع والاستطلاع

نداءات الوقائع تعيد غلاف الصفحات نفسه؛ هذا الملف يجمع القائمة والتفاصيل والمراجعة والدليل:

frontend/src/api/incidents.ts

```typescript
import { api } from "./client";
import type { EvidenceOut, IncidentOut, IncidentPatch, Paginated } from "../types/api";

export interface IncidentListParams {
  page?: number;
  page_size?: number;
  incident_type?: string;
  severity?: string;
  status?: string;
  video_id?: string;
  date_from?: string;
  date_to?: string;
  sort?: string;
}

export async function listIncidents(params: IncidentListParams = {}) {
  const { data } = await api.get<Paginated<IncidentOut>>("/api/v1/incidents", { params });
  return data;
}

export async function getIncident(id: string) {
  const { data } = await api.get<IncidentOut>(`/api/v1/incidents/${id}`);
  return data;
}

export async function patchIncident(id: string, patch: IncidentPatch) {
  const { data } = await api.patch<IncidentOut>(`/api/v1/incidents/${id}`, patch);
  return data;
}

export async function listEvidence(id: string) {
  const { data } = await api.get<EvidenceOut[]>(`/api/v1/incidents/${id}/evidence`);
  return data;
}
```

خطّاف استطلاع عام يوقف نفسه عند انتهاء العمل ويطبّق تراجعاً بسيطاً عند الخطأ (الفصل 16):

frontend/src/hooks/usePolling.ts

```typescript
import { useEffect, useRef, useState } from "react";

export function usePolling<T>(
  fetcher: () => Promise<T>,
  options: { intervalMs?: number; enabled?: boolean; shouldStop?: (d: T) => boolean } = {},
) {
  const { intervalMs = 3000, enabled = true, shouldStop } = options;
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState<Error | null>(null);
  const [loading, setLoading] = useState(false);
  const timer = useRef<number | null>(null);
  const stopRef = useRef(shouldStop);
  stopRef.current = shouldStop;

  useEffect(() => {
    if (!enabled) return;
    let cancelled = false;
    const tick = async () => {
      try {
        setLoading(true);
        const result = await fetcher();
        if (cancelled) return;
        setData(result);
        setError(null);
        if (!stopRef.current?.(result)) timer.current = window.setTimeout(tick, intervalMs);
      } catch (e) {
        if (cancelled) return;
        setError(e as Error);
        timer.current = window.setTimeout(tick, intervalMs * 2); // simple backoff
      } finally {
        if (!cancelled) setLoading(false);
      }
    };
    tick();
    return () => {
      cancelled = true;
      if (timer.current) window.clearTimeout(timer.current);
    };
  }, [enabled, intervalMs, fetcher]);
```

```
  return { data, error, loading };
}
```

## 15.5 جدول الوقائع

أعمدة الجدول: النوع، الخطورة، الحالة، وقت البداية، وعدد الكشوفات؛ وكل صف يقود إلى التفاصيل.

tsx — frontend/src/features/incidents/IncidentTable.tsx

```
import { Link } from "react-router-dom";
import type { IncidentOut } from "../../types/api";

const severityClass: Record<string, string> = {
  low: "bg-slate-100 text-slate-700",
  medium: "bg-amber-100 text-amber-800",
  high: "bg-orange-100 text-orange-800",
  critical: "bg-red-100 text-red-800",
};

export function IncidentTable({ items }: { items: IncidentOut[] }) {
  if (items.length === 0) return <p>No incidents found for these filters.</p>;
  return (
    <table className="w-full text-sm">
      <thead>
        <tr><th>Type</th><th>Severity</th><th>Status</th><th>Start (s)</th><th>Detections</th></tr>
      </thead>
      <tbody>
        {items.map((it) => (
          <tr key={it.id}>
            <td><Link to={`/incidents/${it.id}`}>{it.incident_type}</Link></td>
            <td><span className={severityClass[it.severity]}>{it.severity}</span></td>
            <td>{it.status}</td>
            <td>{it.started_at_seconds}</td>
            <td>{it.detection_count}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

## 15.6 بطاقات التحليلات ورسم بياني بـ Recharts

البطاقات تقرأ من AnalyticsOverview والرسوم من TimeseriesPoint[] (كل نقطة يوم)؛ بلا أي حساب أو رقم محلي.

tsx — frontend/src/features/analytics/OverviewCards.tsx

```
import type { AnalyticsOverview } from "../../types/api";

function Card({ label, value }: { label: string; value: number }) {
  return (
    <div className="rounded border p-4">
      <div className="text-xs text-slate-500">{label}</div>
      <div className="text-2xl font-bold">{value}</div>
    </div>
  );
}

export function OverviewCards({ data }: { data: AnalyticsOverview }) {
  return (
    <div className="grid grid-cols-4 gap-3">
      <Card label="Videos" value={data.total_videos} />
      <Card label="Processed" value={data.processed_videos} />
      <Card label="Incidents" value={data.total_incidents} />
      <Card label="Pending review" value={data.pending_review} />
      <Card label="Jobs running" value={data.jobs_running} />
      <Card label="Jobs failed" value={data.jobs_failed} />
    </div>
  );
}
```

tsx — frontend/src/features/analytics/IncidentsTimeseries.tsx

```
import {
  CartesianGrid, Legend, Line, LineChart,
  ResponsiveContainer, Tooltip, XAxis, YAxis,
} from "recharts";
import type { TimeseriesPoint } from "../../types/api";
```

```tsx
export function IncidentsTimeseries({ points }: { points: TimeseriesPoint[] }) {
  return (
    <ResponsiveContainer width="100%" height={280}>
      <LineChart data={points}>
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="date" />
        <YAxis allowDecimals={false} />
        <Tooltip />
        <Legend />
        <Line type="monotone" dataKey="ppe_violation" stroke="#0f766e" />
        <Line type="monotone" dataKey="fire_smoke" stroke="#c2410c" />
        <Line type="monotone" dataKey="machine_proximity" stroke="#1d4ed8" />
      </LineChart>
    </ResponsiveContainer>
  );
}
```

## 15.7 تنزيل تقرير CSV

نطلب CSV كـ `blob` ثم ننشئ رابطاً مؤقتاً ونضغط عليه برمجياً:

**frontend/src/features/reports/ExportCsvButton.tsx** — tsx

```tsx
import { useState } from "react";
import { api } from "../../api/client";

export function ExportCsvButton({ query }: { query: Record<string, string> }) {
  const [busy, setBusy] = useState(false);

  async function download() {
    setBusy(true);
    try {
      const res = await api.get("/api/v1/reports/export", { params: query, responseType: "blob" });
      const url = URL.createObjectURL(res.data as Blob);
      const a = document.createElement("a");
      a.href = url;
      a.download = "safevision-report.csv";
      a.click();
      URL.revokeObjectURL(url);
    } finally {
      setBusy(false);
    }
  }

  return <button onClick={download} disabled={busy}>{busy ? "..." : "Export CSV"}</button>;
}
```

## 15.8 ربط كل صفحة ببياناتها

ما يجلبه كل قسم فعلاً:

**جدول 15-1: الصفحات ومصادر بياناتها**

| الصفحة | البيانات | النداء |
|---|---|---|
| Dashboard Overview | Videos, Incidents, Pending, Jobs running/failed | `GET /analytics/overview` |
| Video Library | قائمة الفيديوهات وحالتها وأحدث job | `GET /videos` |
| Incident Management | وقائع مفلترة بترقيم ونطاق تاريخ | `GET /incidents` |
| Analytics | اتجاهات زمنية وتوزيعات | `GET /analytics/timeseries` |
| Reports | ملخص يومي وتصدير CSV | `GET /reports/daily, /reports/export` |

عند AUTH_ENABLED=false لا تحتاج الصفحات توكن وتعمل بمستخدم افتراضي دوره safety_officer؛ وعند true يُرسل التوكن وتُحوَّل الصفحات المحمية إلى /login عند فقدانه.

### خلاصة الفصل 15

**ما تعلّمناه**

بنينا عميل Axios يقرأ عنوانه من البيئة، وطبقات نداءات لكل مورد، ورفعاً بـ FormData مع تقدّم، وخطّاف استطلاع، وجدول وقائع، وبطاقات تحليلات، ورسم Recharts، وتنزيل CSV.

### ما يجب تنفيذه

- ضبط VITE_API_BASE_URL وVITE_ENABLE_AUTH في ملف .env.
- إضافة عنوان الواجهة إلى CORS_ORIGINS على الـ Backend.
- ربط كل صفحة بنداتها الموحّد بدل استدعاء Axios مباشرة.
- بناء حالات Loading/Error/Empty لكل قائمة.

### المخرجات المتوقعة

- صفحات تعرض بيانات حقيقية من الـ API.
- رفع يعمل مع شريط تقدّم ورسائل خطأ عربية.
- زر تصدير ينزّل ملف CSV فعلياً.

### كيف نتحقق من الإنجاز

- لا يوجد رقم مكتوب في الكود، وكل قيمة تأتي من استجابة.
- لا يوجد عنوان API داخل أي مكوّن.
- الاستطلاع يتوقف عند الحالة النهائية ولا يترك مؤقتات تعمل.

### أخطاء شائعة

- كتابة localhost:8000 داخل المكوّن بدل متغيّر البيئة.
- إرسال JSON بدل FormData في الرفع فيفشل التحقق.
- نسيان إلغاء المؤقت عند مغادرة الصفحة فيستمر الاستطلاع.
- معالجة CSV كأنه JSON فيتلف الملف.

# 16 التحديثات الفورية وحالة المعالجة

Polling الآن، WebSocket لاحقاً

بعد بدء معالجة فيديو، يريد المستخدم أن يرى نسبة التقدّم تتحرّك دون إعادة تحميل الصفحة. هذا الفصل يقارن الطرق الثلاثة لتحديث الحالة، ويوضّح لماذا نختار **Polling** في الـ MVP، وكيف نكتبه دون تسريب مؤقتات، وكيف نترك الباب مفتوحاً لـ **WebSocket** لاحقاً.

## 16.1 المقارنة بين الطرق الثلاثة

**جدول 16-1: Polling مقابل SSE مقابل WebSocket**

| الطريقة | كيف تعمل | المزايا | العيوب وتعقيد التنفيذ |
|---|---|---|---|
| `Polling` | الواجهة تسأل الـ API كل فترة ثابتة عن حالة الـ job | بسيطة جداً، تعمل مع أي خادم، لا تحتاج بنية خاصة | طلبات متكرّرة ولو لم يتغيّر شيء؛ تأخير بحدود الفترة المختارة |
| `SSE` | قناة واحدة من الخادم إلى المتصفح تُرسل تحديثات نصّية | اتجاه واحد بسيط، إعادة اتصال تلقائية في المتصفح | اتجاه واحد فقط، تحتاج إدارة اتصالات طويلة على الخادم |
| `WebSocket` | اتصال ثنائي الاتجاه يبقى مفتوحاً | أسرع تحديث وأقل حمولة عند كثرة العملاء | أعقد: إدارة الاتصالات، إعادة الاتصال، الحالة على الخادم |

## 16.2 لماذا نختار Polling للـ MVP

معدّل تحديث حالة الـ job ليس مرتفعاً، وعدد المستخدمين في العرض التوضيحي صغير، والفائدة من WebSocket لا تبرّر تعقيده الآن. الـ Polling يُبنى في دقائق، ويعمل مع نفس الـ endpoints الموجودة (GET /api/v1/jobs/{job_id}) دون أي تغيير في الـ Backend. نستخدم فترة قصيرة نسبياً (٣ ثوانٍ) أثناء المعالجة، ونتوقف تماماً عند وصول الـ job إلى حالة نهائية.

## 16.3 خطّاف متابعة حالة الـ Job

نبني الخطّاف على usePolling من الفصل السابق، ونمرّر له دالة shouldStop تُرجِع true عند الحالات النهائية (completed, failed, cancelled). هذا يمنع استمرار الطلبات بلا داعٍ بعد انتهاء العمل.

frontend/src/hooks/useJobStatus.ts

```typescript
import { useCallback } from "react";
import { api } from "../api/client";
import type { JobOut } from "../types/api";
import { usePolling } from "./usePolling";

const TERMINAL: string[] = ["completed", "failed", "cancelled"];

export function useJobStatus(jobId: string | null, intervalMs = 3000) {
  const fetcher = useCallback(async () => {
    const { data } = await api.get<JobOut>(`/api/v1/jobs/${jobId}`);
    return data;
  }, [jobId]);

  const shouldStop = useCallback((job: JobOut) => TERMINAL.includes(job.status), []);

  return usePolling<JobOut>(fetcher, {
    enabled: Boolean(jobId),
    intervalMs,
    shouldStop,
  });
}
```

## 16.4 تجنّب تسريب المؤقتات والـ Backoff

الخطأ الأشهر هو ترك setInterval يعمل بعد مغادرة الصفحة، فيستمر الطلب إلى خادم أُغلق أو مستخدم سجّل خروجه. لذلك نستخدم setTimeout متسلسلاً لا setInterval، ونُنظّف المؤقت في دالة الإرجاع الخاصة بـ useEffect. عند فشل الطلب نطبّق تراجعاً بسيطاً بمضاعفة الفترة قبل المحاولة التالية بدل إغراق الخادم بطلبات فاشلة.

- خطّاف واحد مسؤول عن المؤقت، ولا يُنشئ المؤقت إلا عند تفعيل enabled.
- إلغاء الطلب الجاري عبر علم cancelled حتى لا تُحدّث مكوّناً أُزيل.

- التوقّف عند الحالة النهائية أو عند خطأ متكرّر تجاوز حدّاً معقولاً.

## 16.5 WebSocket كتحسين مستقبلي

إذا كثُر المستخدمون أو احتجنا تحديثاً فورياً حقيقياً، ننتقل إلى **WebSocket**. الملف التالي مكتوب كواجهة مخطَّطة فقط، وموضّح صراحةً أنه **غير منفَّذ في الـ MVP**، ويُذكر هنا لبيان شكل التحسين المستقبلي دون أي ادّعاء بإنجازه.

backend/app/api/v1/endpoints/ws.py — python

```python
# backend/app/api/v1/endpoints/ws.py
# FUTURE / NOT IMPLEMENTED IN THE MVP - placeholder only.
from fastapi import APIRouter, WebSocket, WebSocketDisconnect

router = APIRouter()


@router.websocket("/ws/jobs/{job_id}")
async def job_status_socket(websocket: WebSocket, job_id: str) → None:
    await websocket.accept()
    try:
        while True:
            await websocket.receive_text()  # no broadcast wired yet
    except WebSocketDisconnect:
        return
```

## 16.6 تنبيه: الفيديو المسجَّل ليس "نظاماً لحظياً"

**دقّة الوصف**

ما نبنيه هو **تحليل فيديو مسجَّل** (`recorded video analysis`)، وليس بثاً مباشراً. الـ Polling يُحدّث حالة معالجة ملف سبق رفعه، ولا يعني أن الكاميرا متصلة الآن. لا نستخدم كلمة "لحظي" أو "Live" لوصف هذه الوظيفة، لأن ذلك ادّعاء غير صحيح، والبث المباشر (RTSP) خارج نطاق الـ MVP صراحةً.

### خلاصة الفصل 16

**ما تعلّمناه**

قارنّا Polling وSSE وWebSocket، واخترنا Polling للـ MVP لبساطته، وبنينا خطّاف useJobStatus يتوقف عند الحالة النهائية، وشرحنا تجنّب تسريب المؤقتات، ووضعنا WebSocket كتحسين مستقبلي غير منفَّذ.

**ما يجب تنفيذه**

- استخدام useJobStatus في صفحة تفاصيل الفيديو.
- التأكد من إلغاء المؤقت عند إزالة المكوّن.
- ضبط الفترة الزمنية بحيث لا تُثقل الخادم.
- وسم أي كود WebSocket بأنه مستقبلي وغير منفَّذ.

**المخرجات المتوقعة**

- شريط تقدّم يتحرّك أثناء المعالجة ويتوقف عند الانتهاء.
- لا طلبات متبقية بعد مغادرة الصفحة.
- توضيح مكتوب أن النظام يعالج فيديو مسجَّلاً لا بثاً مباشراً.

**كيف نتحقق من الإنجاز**

- افتح تبويب الشبكة: الطلبات تتوقف فور وصول الحالة النهائية.
- لا يظهر تحذير تحديث حالة مكوّن غير مركّب في الـ console.
- لا يوجد نص يدّعي اللحظية في الواجهة.

**أخطاء شائعة**

- استخدام setInterval دون تنظيف فيستمر الطلب دائماً.
- الاستمرار في الاستطلاع بعد الحالة النهائية.

- وصف تحليل فيديو مسجَّل بأنه نظام لحظي.
- فترة استطلاع قصيرة جداً تُثقل الخادم بلا فائدة.

# 17 التحليلات وتقارير الالتزام والتصدير

كل رقم في اللوحة يجب أن يأتي من صف مخزَّن في قاعدة البيانات

التحليلات ليست رسوماً تجميلية، بل إجابة عن أسئلة السلامة: كم واقعة حدثت؟ وفي أي نوع؟ وما خطورتها؟ وما زال ينتظر المراجعة؟ هذا الفصل يبني كل هذه الأرقام من صفوف حقيقية في PostgreSQL، ويصدّرها كملف CSV قابل للمشاركة.

## 17.1 مبدأ واحد: كل رقم من صف مخزَّن

ممنوع كتابة أي رقم في الكود. المؤشرات تُحسب بـ **SQL** عبر استعلامات تجميع (**aggregation**) على جداول incidents وvideos وprocessing_jobs. الفائدة أن اللوحة تتغيّر تلقائياً مع كل واقعة جديدة دون أي تعديل في الكود. الاستعلامات كلها مقيدة بالفلترة بالتاريخ والفيديو والنوع، حتى تُعرض نفس الأرقام لمدى زمني مختلف دون كتابة مسار جديد.

## 17.2 خدمة التحليلات واستعلاماتها

الخدمة تُرجع كائناً مطابقاً حرفياً لـ AnalyticsOverview، وتُبني كل حقول by_status / by_severity / incidents_by_type من مجموعة تجميع واحدة لكل بعد.

backend/app/services/analytics_service.py

```python
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.models.incident import Incident
from app.models.processing_job import ProcessingJob
from app.models.video import Video


def _count_by(db: Session, column):
    rows = db.execute(select(column, func.count()).group_by(column)).all()
    return {str(key): int(count) for key, count in rows}


def get_overview(db: Session) -> dict:
    total_videos = db.scalar(select(func.count()).select_from(Video)) or 0
    processed = db.scalar(
        select(func.count()).select_from(Video).where(Video.status == "processed")
    ) or 0
    total_incidents = db.scalar(select(func.count()).select_from(Incident)) or 0
    pending = db.scalar(
        select(func.count()).select_from(Incident).where(Incident.status == "pending_review")
    ) or 0
    jobs_running = db.scalar(
        select(func.count()).select_from(ProcessingJob).where(ProcessingJob.status == "running")
    ) or 0
    jobs_failed = db.scalar(
        select(func.count()).select_from(ProcessingJob).where(ProcessingJob.status == "failed")
    ) or 0
    last_incident_at = db.scalar(select(func.max(Incident.created_at)))
    return {
        "total_videos": int(total_videos),
        "processed_videos": int(processed),
        "total_incidents": int(total_incidents),
        "incidents_by_type": _count_by(db, Incident.incident_type),
        "incidents_by_severity": _count_by(db, Incident.severity),
        "incidents_by_status": _count_by(db, Incident.status),
        "pending_review": int(pending),
        "jobs_running": int(jobs_running),
        "jobs_failed": int(jobs_failed),
        "last_incident_at": last_incident_at,
    }
```

الاتجاه الزمني يُبنى من تجميع على اليوم والنوع، ثم يُحوَّل إلى صفوف بشكل TimeseriesPoint بحيث يحمل كل يوم المفاتيح الأربعة:

backend/app/services/analytics_service.py (timeseries) python

```
def get_timeseries(db: Session, date_from, date_to) → list[dict]:
    day = func.date(Incident.created_at).label("day")
    rows = db.execute(
        select(day, Incident.incident_type, func.count())
        .where(Incident.created_at ⩾ date_from, Incident.created_at < date_to)
        .group_by(day, Incident.incident_type)
    ).all()
    buckets: dict[str, dict] = {}
    for d, itype, count in rows:
        point = buckets.setdefault(
            str(d), {"date": str(d), "ppe_violation": 0, "fire_smoke": 0,
                     "machine_proximity": 0, "total": 0}
        )
        point[itype] += int(count)
        point["total"] += int(count)
    return sorted(buckets.values(), key=lambda p: p["date"])
```

## 17.3 الملخص اليومي والمخالفات المتكررة

الملخص اليومي هو تجميع بسيط على التاريخ والنوع والخطورة والحالة، ويُستخدم في صفحة التقارير وفي التصدير معاً، فلا يوجد منطق مكرّر. أما المخالفات المتكررة فتُكتشف بالعدّ حسب zone_name أو class_name، أو بإعادة عدّ وقائع نفس video_id خلال فترة.

**جدول 17-1: المؤشرات ومصدر حسابها**

| المؤشر | التجميع | الفلترة المدعومة |
|---|---|---|
| عدد الوقائع حسب النوع | GROUP BY incident_type | تاريخ، فيديو، كاميرا |
| عدد الوقائع حسب الخطورة | GROUP BY severity | تاريخ، نوع، فيديو |
| حالة المراجعة | GROUP BY status | تاريخ، نوع |
| الاتجاه اليومي | GROUP BY date(created_at), incident_type | نطاق تاريخ |
| المخالفات المتكررة | GROUP BY zone_name / video_id | تاريخ، نوع |

المدخلات كلها معاملات استعلام (date_from, date_to, video_id, incident_type) تُطبَّق في جملة WHERE نفسها، فلا نحسب مؤشراً بلا فلترة ثم نفلتر النتيجة في الواجهة.

## 17.4 مسار تصدير CSV

التصدير يعيد text/csv مباشرة، مع ترويسة Content-Disposition التي تجعل المتصفح ينزّل الملف. نكتب الصفوف من نفس دالة الملخص اليومي، ونجبر الترميز على utf-8-sig حتى تظهر العربية سليمة في Excel.

backend/app/api/v1/endpoints/reports.py python

```
import csv
import io

from fastapi import APIRouter, Depends
from fastapi.responses import StreamingResponse
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.services.analytics_service import get_daily_rows

router = APIRouter(prefix="/reports", tags=["reports"])


@router.get("/daily")
def daily(date_from: str | None = None, date_to: str | None = None,
          incident_type: str | None = None, db: Session = Depends(get_db)):
    return get_daily_rows(db, date_from, date_to, incident_type)


@router.get("/export")
def export_csv(date_from: str | None = None, date_to: str | None = None,
               incident_type: str | None = None, db: Session = Depends(get_db)):
    rows = get_daily_rows(db, date_from, date_to, incident_type)
    buffer = io.StringIO()
    writer = csv.writer(buffer)
    writer.writerow(["date", "incident_type", "severity", "status", "count"])
    for r in rows:
        writer.writerow([r["date"], r["incident_type"], r["severity"], r["status"], r["count"]])
    content = buffer.getvalue().encode("utf-8-sig")
```

```
    return StreamingResponse(
        iter([content]),
        media_type="text/csv",
        headers={"Content-Disposition": "attachment; filename=safevision-report.csv"},
    )
```

## 17.5 تنزيل الملف في الواجهة

نطلب الملف بـ responseType: "blob"، ثم ننشئ رابطاً مؤقتاً ونضغط عليه برمجياً، ونحرّر الرابط بعد ذلك لتوفير الذاكرة. لا نحاول قراءة CSV كأنه JSON.

frontend/src/api/reports.ts

```typescript
import { api } from "./client";

export interface ReportParams {
  date_from?: string;
  date_to?: string;
  incident_type?: string;
  video_id?: string;
}

export async function downloadReportCsv(params: ReportParams) {
  const res = await api.get("/api/v1/reports/export", {
    params,
    responseType: "blob",
  });
  const url = URL.createObjectURL(res.data as Blob);
  const link = document.createElement("a");
  link.href = url;
  link.download = "safevision-report.csv";
  link.click();
  URL.revokeObjectURL(url);
}
```

## 17.6 البيانات الاصطناعية والوسم الواضح

> **لا أرقام وهمية بلا وسم**
>
> قبل توفر بيانات حقيقية كافية، يجوز استخدام **بيانات اصطناعية** (`synthetic demo data`) لعرض الشكل فقط، بشرط أن تكون في جدول أو ملف منفصل بوضوح، وأن يظهر على الشاشة وسم مثل **"بيانات تجريبية اصطناعية"**. يُمنع خلط الأرقام الاصطناعية ببيانات حقيقية في نفس البطاقة، ويُمنع تقديمها كأنها نتائج قياس فعلية.

## 17.7 PDF كخيار مستقبلي

تصدير **PDF** مفيد للإدارة لكنه يحتاج مكتبة إضافية وتحكّماً في التخطيط، لذا نتركه `Future`. الـ CSV يكفي الـ MVP: يُفتح في Excel، ويمكن بناؤه من نفس صفوف الملخص دون منطق جديد.

### خلاصة الفصل 17

**ما تعلّمناه**

حسبنا المؤشرات كلها من صفوف مخزَّنة عبر استعلامات تجميع، وبنينا الاتجاه الزمني بشكل TimeseriesPoint، وصمّمنا مسار تصدير CSV، وكتبنا كود التنزيل، ووضعنا ضوابط للبيانات الاصطناعية، وأجّلنا PDF.

**ما يجب تنفيذه**

- [ ] كتابة استعلامات التحليلات والتحقق من أسمائها مقابل SPEC.
- [ ] ربط صفحة التقارير بمسار /reports/daily وزر التصدير.
- [ ] إضافة وسم صريح لأي بيانات اصطناعية.
- [ ] التحقق من ظهور العربية سليمة في ملف CSV.

**المخرجات المتوقعة**

- لوحة تعرض أرقاماً تتغيّر مع كل واقعة جديدة.
- ملف CSV قابل للفتح في Excel بعربية سليمة.

- فلترة بالنطاق الزمني والنوع والفيديو تعمل على الخادم.

### كيف نتحقق من الإنجاز

- إضافة واقعة يدوياً تغيّر أرقام اللوحة.
- لا يوجد رقم ثابت مكتوب في الـ service.
- الملف المُنزَّل يطابق صفوف /reports/daily.

### أخطاء شائعة

- حساب المؤشرات في الواجهة من صفحة واحدة بدل الخادم.
- نسيان الفلترة فيرجع الرقم لكامل البيانات.
- عرض أرقام اصطناعية دون وسم.
- معالجة CSV كنص عادي فيتلف عند التنزيل.

CH18 · SECURITY & PRIVACY

# 18 الأمان والخصوصية والمصادقة

أمان عملي لمشروع طلابي صناعي، بدون مبالغة وبدون ثغرات واضحة

النظام يتعامل مع فيديو أماكن عمل، والخطأ الأمني فيه لا يهدّد البيانات فقط بل خصوصية العاملين. هذا الفصل يضع حدّاً أدنى صلباً من الأمان: كلمات مرور مشفَّرة، توكن صالح، أدوار محدودة، رفع محكم، أسرار خارج الكود، وخصوصية محفوظة بعدم التعرف على الوجوه. الهدف عملي: لا ثغرة واضحة، ودون تعقيد يعطّل الـ MVP.

## 18.1 المصادقة مقابل التصريح

**المصادقة (Authentication):** مَن أنت؟ تُتحقّق من هوية المستخدم عبر بريد وكلمة مرور. **التصريح (Authorization):** ما المسموح لك فعله؟ يُحدَّد بدورك في النظام. الخطأ الشائع هو الخلط بينهما، فيكفي تسجيل الدخول لأداء أي إجراء.

**جدول 18-1: الفرق بين المصادقة والتصريح**

| الجانب | المصادقة | التصريح |
|---|---|---|
| السؤال | مَن أنت؟ | ماذا يُسمح لك؟ |
| الأداة | كلمة مرور + bcrypt + توكن JWT | دور المستخدم (user_role) |
| عند الفشل | 401 Unauthorized | 403 Forbidden |

**وضع العرض التوضيحي**

عند AUTH_ENABLED=false يتجاوز النظام المصادقة ويعمل بمستخدم افتراضي دوره safety_officer، وهذا مقبول لعرض الـ MVP. عند true تُفرض المصادقة على كل المسارات المعلَّمة بـ yes*، وتُرسل الواجهة التوكن في ترويسة Authorization. الفرق بين الوضعين يجب أن يكون **معلَماً بصراحة في الواجهة**.

## 18.2 تشفير كلمات المرور

لا تُخزَّن كلمة المرور أبداً كنص صريح. نستخدم bcrypt الذي يضيف **salt** عشوائياً لكل كلمة، فيستحيل مقارنة الهاشين لكلمتين متطابقتين، ويصبح كسر الهاش مكلفاً. عمود القاعدة hashed_password يحمل الهاش فقط.

backend/app/core/security.py

```python
# backend/app/core/security.py
from datetime import datetime, timedelta, timezone

import bcrypt
import jwt

from app.core.config import settings


def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")


def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))


def create_access_token(subject: str, role: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES
    )
    payload = {"sub": subject, "role": role, "exp": expire}
    return jwt.encode(payload, settings.SECRET_KEY, algorithm=settings.JWT_ALGORITHM)


def decode_token(token: str) -> dict:
    return jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.JWT_ALGORITHM])
```

## 18.3 التوكن والاعتماديات والتصريح بالأدوار

الـ JWT توكن موقّع يحتوي هوية المستخدم ودوره ووقت انتهائه. الـ dependency get_current_user تُستخدم في كل مسار محمي، وتتجاوز المصادقة عند إيقاف العلم. ثم require_role يمنع الأدوار غير المصرّح لها.

backend/app/api/deps.py — python

```python
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.config import settings
from app.core.security import decode_token
from app.db.session import get_db
from app.models.user import User

bearer = HTTPBearer(auto_error=False)
DEFAULT_USER = {"id": None, "email": "demo@safevision.local", "role": "safety_officer"}


def get_current_user(
    creds: HTTPAuthorizationCredentials | None = Depends(bearer),
    db: Session = Depends(get_db),
):
    if not settings.AUTH_ENABLED:
        return DEFAULT_USER  # MVP demo mode
    if creds is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Missing token")
    try:
        payload = decode_token(creds.credentials)
    except Exception:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid token")
    user = db.get(User, payload["sub"])
    if user is None or not user.is_active:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Inactive user")
    return user


def require_role(*roles: str):
    def checker(user=Depends(get_current_user)):
        role = user["role"] if isinstance(user, dict) else user.role
        if settings.AUTH_ENABLED and role not in roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Insufficient role")
        return user
    return checker
```

**جدول 18-2: الأدوار والصلاحيات (RBAC)**

| الدور | الصلاحيات |
|---|---|
| admin | كل شيء: إدارة المستخدمين، حذف الفيديوهات، الإعدادات |
| safety_officer | الرفع، بدء المعالجة، مراجعة الوقائع، التقارير |
| viewer | قراءة فقط: القوائم والتحليلات دون مراجعة أو حذف |

## 18.4 أمان رفع الملفات

الرفع هو أكبر مدخل خطر لأنه يستقبل ملفاً من خارج النظام. نطبّق عليه قيوداً متراكبة:

- **حد الحجم:** نرفض ما يتجاوز MAX_UPLOAD_MB ونعيد 413 بدل إسقاط الاتصال.
- **الامتداد والمحتوى:** نتحقق من ALLOWED_VIDEO_EXTENSIONS ومن `magic bytes` ومن فتح الملف بـ `OpenCV` (تفصيله في الفصل 9).
- **أسماء الملفات:** لا نستخدم الاسم الأصلي على القرص أبداً؛ نولّد اسم UUID ونبني storage_key منه، وهذا يمنع `path traversal`.
- **عدم التنفيذ:** مجلّدات التخزين لا تُخدَم كملفات تنفيذية، والوصول للفيديو والدليل يمرّ عبر endpoint يتحقق من الصلاحية بدل كشف المسار المباشر.

## 18.5 تقييد CORS والتحقق من المدخلات ومنع SQL Injection

- **CORS:** نسمح فقط بأصول الواجهة المحدّدة في CORS_ORIGINS، ولا نستخدم "*" في الإنتاج.
- **التحقق من المدخلات:** كل جسم طلب يمرّ على مخطط `Pydantic`، وكل معامل استعلام محدَّد النوع، فالفشل يعيد 422 قبل الوصول للمنطق.

- **منع SQL Injection:** نستخدم SQLAlchemy مع معاملات مرتبطة (bound parameters) ولا نُركّب نصوص SQL يدوياً، فلا يمكن للمدخل أن يغيّر الاستعلام.
- **Rate limiting:** غير ضروري في العرض المحلي، لكن يُضاف على /auth/login عند النشر العام لمنع التخمين المتكرّر.

## 18.6 إدارة الأسرار

لا يُكتب أي سرّ في الكود ولا في Git. المفتاح SECRET_KEY وكلمات قاعدة البيانات تُقرأ من البيئة، ويحتوي المستودع ملف .env.example بقيم Placeholder فقط. إذا رُفع سرّ إلى Git بالخطأ، يجب تغييره فوراً لا حذفه فقط.

.env.example (placeholders only) — ini

```
APP_NAME=SafeVision AI
APP_ENV=development
APP_DEBUG=true
API_V1_PREFIX=/api/v1
AUTH_ENABLED=false
SECRET_KEY=<CHANGE_ME_RANDOM_32_BYTES>
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
CORS_ORIGINS=http://localhost:5173

POSTGRES_USER=postgres
POSTGRES_PASSWORD=<CHANGE_ME>
POSTGRES_DB=safevision
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
DATABASE_URL=postgresql+psycopg://postgres:<CHANGE_ME>@localhost:5432/safevision

STORAGE_BACKEND=local
LOCAL_STORAGE_ROOT=storage
UPLOAD_DIR=storage/uploads
EVIDENCE_DIR=storage/evidence
REPORT_DIR=storage/reports
MAX_UPLOAD_MB=<SET_MAX_UPLOAD_MB>
ALLOWED_VIDEO_EXTENSIONS=mp4,mov,avi,mkv

VITE_API_BASE_URL=http://localhost:8000
VITE_ENABLE_AUTH=false
```

## 18.7 الخصوصية: تتبّع مجهول بلا تعرّف على الوجوه

**لا تعرّف على الوجوه**

النظام **لا يستخدم التعرف على الوجوه ولا يخزّن هويات حقيقية للعاملين**. الـ track_id مجرّد رقم مؤقت داخل فيديو واحد لربط كشوفات الشخص نفسه في الإطارات المتتالية؛ هو **ليس** هوية ولا يمكن ربطه بشخص في فيديو آخر. عند رفع فيديو جديد يبدأ الترقيم من جديد.

- **التتبّع المجهول:** نخزّن رقم المسار وزمنه ودرجة الخطورة، دون أي بيانات تُعرّف الشخص.
- **الوصول للفيديو والدليل:** محمي بالصلاحيات؛ الـ file_url يمرّ عبر endpoint يتحقق من المستخدم بدل رابط مباشر قابل للمشاركة.
- **سياسات الاحتفاظ:** الفيديوهات وأدلة الوقائع لها مدة احتفاظ محدّدة تُحذف بعدها (بما فيها الملفات من طبقة التخزين)، والوقائع تبقى كسجلات إحصائية.
- **الموافقة والغرض:** الفيديو يُستخدم لغرض السلامة فقط، ولا يُعاد استخدامه لأي غرض آخر.

## 18.8 السجلات والنسخ الاحتياطي

الـ logs لا تحتوي كلمات مرور ولا توكنات ولا مسارات ملفات حساسة؛ نُسجّل المعرّفات والأخطاء فقط. ولحماية البيانات نأخذ نسخة احتياطية دورية من PostgreSQL (مثلاً pg_dump) مع نسخة من مجلّدات التخزين، ونختبر استرجاع نسخة واحدة فعلياً لأن نسخة لم تُجرَّب ليست نسخة.

### خلاصة الفصل 18

**ما تعلّمناه**

فرّقنا بين المصادقة والتصريح، وشفّرنا كلمات المرور بـ bcrypt، وبنينا توكن JWT مع get_current_user وrequire_role، وأحكمنا الرفع وCORS ومنع SQL Injection، وأخرجنا الأسرار للبيئة، وثبّتنا خصوصية بلا تعرّف على الوجوه.

### ما يجب تنفيذه

- توليد SECRET_KEY عشوائي ووضعه في البيئة لا في الكود.
- ربط المسارات الحساسة بـ Depends(require_role(...)).
- التأكد أن كل مسار محمي يمرّ بـ get_current_user.
- كتابة سياسة احتفاظ مكتوبة وضبط مدة الحذف.

### المخرجات المتوقعة

- جدول مستخدمين بكلمات مرور مشفَّرة فقط.
- ملف .env.example بقيم Placeholder فقط.
- توضيح مكتوب أن التتبّع مجهول وأن الـ track_id مؤقت.

### كيف نتحقق من الإنجاز

- لا يوجد نص صريح لكلمة مرور في قاعدة البيانات.
- طلب مسار محمي بلا توكن يعيد 401 عند تفعيل المصادقة.
- دور غير مصرّح له يحصل على 403.
- لا يوجد سرّ مكتوب في الكود أو في Git.

### أخطاء شائعة

- تخزين كلمة المرور كنص صريح أو بـ md5.
- السماح بـ CORS "*" في الإنتاج.
- بناء استعلام SQL بتركيب نصي مباشر.
- استخدام الـ track_id كهوية شخص أو ربطه بين فيديوهات.
- وضع الأسرار في الكود أو رفع .env إلى Git.

CH19 · DOCKER

# 19 Docker والنشر المحلي

تشغيل الـ Backend و الـ Frontend و PostgreSQL بأمر واحد

هذا الفصل يحلّ مشكلة "يعمل عندي ولا يعمل عندك": نغلّف كل مكوّن في حاوية **Container** ونربطها بأمر واحد، فيستطيع أي عضو — أو لجنة المناقشة — تشغيل النظام كاملاً على جهاز نظيف دون تثبيت Python أو Node أو PostgreSQL يدوياً.

## 19.1 شرح Docker و Docker Compose من البداية

**Docker:** أداة تُغلّف التطبيق مع كل ما يحتاجه في وحدة قابلة للتشغيل تسمّى **Image (صورة)**؛ وتشغيلها يُنتج **Container (حاوية)**: عملية معزولة تعطي نفس النتيجة على Windows و Linux و macOS.

**Docker Compose:** أداة تُشغّل عدة حاويات معاً من ملف واحد docker-compose.yml، وتنشئ لهنّ **network (شبكة)** خاصة و**volumes** مشتركة تلقائياً. نظامنا يحتاج أربع حاويات متصلة.

**جدول 19-1: المصطلحات الأربعة التي يجب ألا يخلط بينها الفريق**

| المصطلح | المعنى بالعربية | مثال في مشروعنا |
|---|---|---|
| `Image` | قالب ثابت غير قابل للتغيير يصف التطبيق ومتطلباته | `safevision-backend` |
| `Container` | نسخة عاملة من الصورة، لها حالة وتُشغَّل وتُوقَف | حاوية الـ backend التي تخدم المنفذ 8000 |
| `Volume` | مساحة تخزين دائمة تبقى بعد حذف الحاوية | مجلد الـ models والفيديوهات وقاعدة البيانات |
| `Network` | شبكة داخلية تربط الحاويات بأسماء ثابتة | `safevision-net` |

## 19.2 الحاويات الأربع ومسؤولية كل واحدة

أربع حاويات هي الحد المعقول لمشروع طلابي:

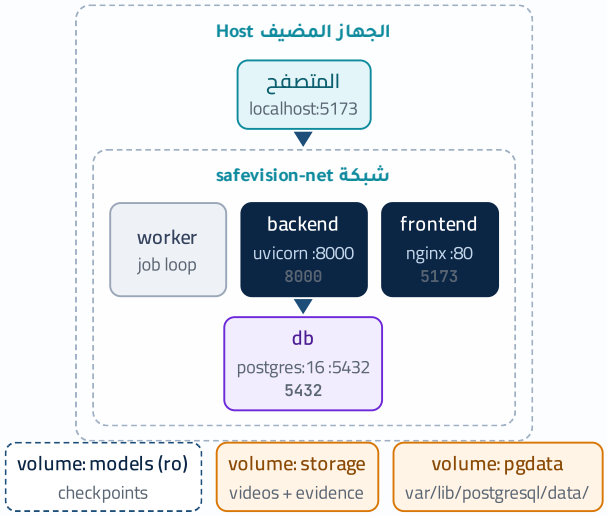
**جدول 19-2: خدمات Docker Compose ومسؤولية كل خدمة**

| اسم الخدمة | المسؤولية | الصورة / البناء | المنفذ المنشور |
|---|---|---|---|
| `db` | قاعدة بيانات PostgreSQL: كل البيانات الوصفية | `postgres:16` | 5432 |
| `backend` | واجهة FastAPI وكل نقاط `/api/v1` | بناء من `backend/Dockerfile` | 8000 |
| `worker` | معالجة الـ jobs (نفس صورة الـ backend، أمر تشغيل مختلف) | نفس صورة الـ backend | لا يوجد |
| `frontend` | خدمة ملفات React عبر nginx | بناء من `frontend/Dockerfile` | 5173 |

**لماذا الـ worker منفصل؟**

لأن معالجة الفيديو تستهلك الوقت والذاكرة ولا نريدها أن تُجمّد الـ API. الـ worker عملية مستقلة تقرأ الـ jobs من قاعدة البيانات بالترتيب (`DB-backed queue`).

## 19.3 مخطط النشر بـ Docker




**شكل 19-1:** مخطط نشر Docker: الحاويات الأربع على شبكة واحدة مع الـ volumes الدائمة

## 19.4 شبكة الحاويات: أسماء الخدمات بدل localhost

داخل شبكة Docker لكل حاوية عنوان خاص، و**لا يمكن استخدام localhost** للوصول إلى حاوية أخرى: localhost داخل الـ backend يشير إليه هو لا إلى قاعدة البيانات. لذلك نستخدم **اسم الخدمة** كاسم نطاق داخلي (db، backend).

**جدول 19-3: من يتحدّث إلى من، وبأي عنوان**

| المتصل | الهدف | العنوان داخل الحاويات | العنوان من المتصفح على الجهاز |
|---|---|---|---|
| backend | db | db:5432 | لا ينطبق (اتصال داخلي) |
| worker | db | db:5432 | لا ينطبق (اتصال داخلي) |
| المتصفح | backend | لا ينطبق | http://localhost:8000 |
| المتصفح | frontend | لا ينطبق | http://localhost:5173 |

**الخطأ الأشهر**

كتابة DATABASE_URL=...@localhost:5432/... داخل حاوية الـ backend: صحيحة على الجهاز، خاطئة داخل Docker (تظهر connection refused). داخل الحاويات نستخدم اسم الخدمة db، ومن المتصفح نستخدم localhost لأن المنافذ منشورة على الجهاز المضيف.

## 19.5 الـ Volumes: كيف تصل ملفات الـ Models والفيديوهات للحاويات

الحاوية قابلة للحذف، فلا نضع فيها شيئاً نفقد فقدانه. ثلاثة أنواع تخزين:

- (named volume) **pgdata**: يحفظ ملفات PostgreSQL فيبقى كل شيء بعد إعادة تشغيل الحاوية.
- **storage**: مجلد مشترك بين الـ backend والـ worker يحتوي الفيديوهات المرفوعة (uploads) وصور الدليل (evidence) والتقارير.
- **models** (bind mount للقراءة فقط): يُربط بمجلد models/. على الجهاز، لأن ملفات الـ checkpoints كبيرة ولا تُرفع إلى Git ولا تُنسخ داخل الصورة.

**لماذا bind mount للـ models؟**

مسار الـ checkpoint يُحدَّد من PPE_MODEL_PATH و FIRE_SMOKE_MODEL_PATH؛ فربط المجلد يجعل تغييره نسخ ملف دون إعادة بناء الصورة، مع ro: لضمان القراءة فقط.

## 19.6 ملف الـ Backend Dockerfile

صورة الـ backend مبنية على Python 3.11، وتُثبّت المتطلبات أولاً ثم يُنسخ الكود للاستفادة من `layer cache`. يستخدم الـ worker نفس الصورة بأمر تشغيل مختلف.

backend/Dockerfile

```dockerfile
# SafeVision AI - backend image (also used by the worker service)
FROM python:3.11-slim
```

```
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# System libraries required by OpenCV and psycopg
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgl1 libglib2.0-0 curl \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies first so the layer is cached
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

# Application code
COPY . .

EXPOSE 8000

# Default command runs the API; the worker overrides this in compose
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

### 19.7 ملف الـ Frontend Dockerfile

نمط **`multi-stage build`**: نبني الواجهة ثم ننقل الناتج الثابت إلى صورة nginx صغيرة، فتبقى Node وnode_modules خارج الصورة النهائية. تُثبَّت قيمة VITE_API_BASE_URL وقت البناء لأن Vite يدمج المتغيّرات في ملفات الإنتاج.

dockerfile **frontend/Dockerfile**

```
# SafeVision AI - frontend image (multi-stage build)
# ---- Stage 1: build the static assets ----
FROM node:20-alpine AS builder
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .
# Vite inlines env vars at build time, so it must be present here
ARG VITE_API_BASE_URL=http://localhost:8000
ARG VITE_ENABLE_AUTH=false
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
ENV VITE_ENABLE_AUTH=$VITE_ENABLE_AUTH
RUN npm run build

# ---- Stage 2: serve the static files ----
FROM nginx:1.27-alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

### 19.8 ملف .dockerignore

.dockerignore يمنع إرسال الملفات غير الضرورية إلى محرّك Docker: بناء أسرع، وحماية من تسريب الأسرار أو الفيديوهات داخل الصورة.

text .dockerignore

```
# Never ship secrets, big binaries or local artifacts inside an image
.git
.gitignore
.env
.venv
venv
__pycache__
*.pyc
.pytest_cache
.mypy_cache

# Large local data must stay on the host, not in the image
models/
data/
storage/

# Frontend build output and dependencies are rebuilt inside the image
frontend/node_modules
frontend/dist

# Local docs and editor folders
docs/
.vscode
.idea
*.md
```

### 19.9 ملف docker-compose.yml

قلب التشغيل. لاحظ: أسماء الخدمات (db, backend, worker, frontend)، وDATABASE_URL باستخدام db، وdepends_on مع healthcheck حتى لا يبدأ الـ backend قبل جاهزية قاعدة البيانات.

yaml docker-compose.yml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - safevision-net

  backend:
    build: ./backend
    env_file: .env
    environment:
      # Inside the network the host name is the service name, not localhost
      DATABASE_URL: postgresql+psycopg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
    ports:
      - "8000:8000"
    volumes:
      - ./storage:/app/storage
      - ./models:/app/models:ro
    depends_on:
      db:
        condition: service_healthy
    networks:
      - safevision-net

  worker:
    build: ./backend
    command: ["python", "-m", "app.workers.main"]
    env_file: .env
    environment:
      DATABASE_URL: postgresql+psycopg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      WORKER_ID: worker-docker-1
    volumes:
```

```
      - ./storage:/app/storage
      - ./models:/app/models:ro
    depends_on:
      db:
        condition: service_healthy
    networks:
      - safevision-net

  frontend:
    build:
      context: ./frontend
      args:
        VITE_API_BASE_URL: ${VITE_API_BASE_URL}
        VITE_ENABLE_AUTH: ${VITE_ENABLE_AUTH}
    ports:
      - "5173:80"
    depends_on:
      - backend
    networks:
      - safevision-net

volumes:
  pgdata:

networks:
  safevision-net:
    driver: bridge
```

**ترتيب البدء**

depends_on مع condition: service_healthy يجعل Compose ينتظر فحص pg_isready قبل تشغيل الـ backend، فيمنع بدء الـ API قبل جهوزية قاعدة البيانات.

## 19.10 ملف .env.example الخاص بـ Docker

القيم الحقيقية في .env (مستثنى من Git)؛ و.env.example يُرفع ويحتوي قيماً توضيحية فقط، مع استخدام اسم الخدمة db.

.env.example — text

```
# Copy to .env and replace every placeholder before running.
APP_NAME=SafeVision AI
APP_ENV=development
APP_DEBUG=true
API_V1_PREFIX=/api/v1

# Inside Docker the database host is the service name "db"
POSTGRES_USER=safevision
POSTGRES_PASSWORD=change-me
POSTGRES_DB=safevision
POSTGRES_HOST=db
POSTGRES_PORT=5432
DATABASE_URL=postgresql+psycopg://safevision:change-me@db:5432/safevision

STORAGE_BACKEND=local
LOCAL_STORAGE_ROOT=/app/storage
UPLOAD_DIR=/app/storage/uploads
EVIDENCE_DIR=/app/storage/evidence
REPORT_DIR=/app/storage/reports

MODEL_DIR=/app/models
PPE_MODEL_PATH=/app/models/<PPE_MODEL_CHECKPOINT_PATH>
FIRE_SMOKE_MODEL_PATH=/app/models/<FIRE_SMOKE_MODEL_CHECKPOINT_PATH>
PERSON_TRACKER_PATH=/app/models/<PERSON_TRACKER_CHECKPOINT_PATH>
MODEL_DEVICE=cpu

# Frontend build-time variables
VITE_API_BASE_URL=http://localhost:8000
VITE_ENABLE_AUTH=false
```

## 19.11 الخطوات السبع للتشغيل

1. **Build (البناء):** نبني صور الـ backend والـ frontend من الـ Dockerfiles.
2. **Up (التشغيل):** نرفع الحاويات الأربع مع الشبكة والـ volumes.
3. **Logs (السجلات):** نتابع مخرجات كل خدمة لاكتشاف أخطاء البدء.

4. **Migrate (الترحيل):** ننفّذ ترحيلات Alembic داخل حاوية الـ backend لإنشاء الجداول.
5. **Health (الفحص):** نتأكد من GET /api/v1/health ومن فتح الواجهة.
6. **Restart (إعادة التشغيل):** نعيد خدمة واحدة بعد تعديل الكود دون إسقاط الباقي.
7. **Data persistence (دوام البيانات):** نتأكد أن الفيديوهات والوقائع باقية بعد إعادة التشغيل.

terminal — run the full stack

```bash
# 1) Build images from the two Dockerfiles
$ docker compose build

# 2) Start db + backend + worker + frontend
$ docker compose up -d

# 3) Follow logs (Ctrl+C stops following, not the containers)
$ docker compose logs -f backend worker

# 4) Create the database schema with Alembic
$ docker compose exec backend alembic upgrade head

# 5) Health check and open the UI
$ curl http://localhost:8000/api/v1/health
$ start http://localhost:5173

# 6) Restart one service after a code change
$ docker compose restart backend

# 7) Verify data persistence
$ docker compose down
$ docker compose up -d
$ curl http://localhost:8000/api/v1/videos
```

## 19.12 جدول أوامر Docker Compose

**جدول 19-4: أوامر Docker Compose اليومية**

| الأمر | ماذا يفعل | متى نستخدمه |
|---|---|---|
| `docker compose build` | يبني الصور من الـ Dockerfiles | بعد تغيير متطلبات أو ملف Dockerfile |
| `docker compose up -d` | يشغّل كل الخدمات في الخلفية | بدء العمل اليومي |
| `docker compose ps` | يعرض الحاويات وحالتها وصحتها | عند الشك في خدمة متوقفة |
| `docker compose logs -f backend` | يتابع سجلات خدمة واحدة | تتبّع خطأ في الـ API |
| `docker compose exec backend alembic upgrade head` | ينفّذ الترحيلات داخل الحاوية | بعد إضافة جدول أو عمود |
| `docker compose exec db psql -U safevision -d safevision` | يفتح صدفة SQL داخل قاعدة البيانات | فحص البيانات يدوياً |
| `docker compose restart worker` | يعيد خدمة واحدة | بعد تعديل كود المعالجة |
| `docker compose down` | يوقف ويحذف الحاويات مع بقاء الـ volumes | إنهاء العمل دون فقدان البيانات |
| `docker compose down -v` | يحذف الحاويات والـ **volumes أيضاً** | تنظيف كامل (يحذف كل البيانات) |

> **تحذير**
>
> -v docker compose down يحذف قاعدة البيانات والفيديوهات وصور الدليل نهائياً. لا يُشغَّل أبداً على نسخة فيها نتائج عرض توضيحي.

## 19.13 حدود دعم GPU في Docker وبديل CPU

تشغيل PyTorch على GPU داخل Docker يحتاج تعريف الحاوية بالبطاقة: على Linux عبر `NVIDIA Container Toolkit` وقسم deploy.resources، وعلى Windows يعتمد على Docker Desktop و WSL2 ودعم البطاقة وقد لا يتوفّر. لذلك قاعدتان:

- **وضع GPU:** يُضبط MODEL_DEVICE=cuda على جهاز يملك بطاقة متوافقة.
- **وضع CPU (الافتراضي للعرض):** MODEL_DEVICE=cpu، يعمل على أي جهاز بلا بطاقة.

**حدود GPU غير مضمونة**

لا نضمن توفر GPU على أي جهاز أو استضافة، ولا نفترض أن الحاوية سترى البطاقة تلقائياً. إن فشل CUDA فالحل الآمن العودة إلى MODEL_DEVICE=cpu.

**docker-compose.override.yml — optional GPU block (Linux + NVIDIA)** yaml

```yaml
# Optional file. Only use it on a machine that really exposes a GPU.
services:
  worker:
    environment:
      MODEL_DEVICE: cuda
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
```

## خلاصة الفصل 19

### ما تعلّمناه

غلّفنا النظام في أربع حاويات على شبكة واحدة تتخاطب بأسماء الخدمات لا بـ localhost، مع بيانات دائمة في الـ volumes، ووضع CPU افتراضي ووضع GPU مشروط.

### ما يجب تنفيذه

- كتابة backend/Dockerfile وfrontend/Dockerfile.
- كتابة .dockerignore وdocker-compose.yml.
- ضبط DATABASE_URL لاستخدام اسم الخدمة db.
- تجهيز مجلد models/. بملفات الـ checkpoints قبل البناء.

### المخرجات المتوقعة

- أمر واحد docker compose up -d يشغّل النظام كاملاً.
- واجهة على localhost:5173 وAPI على localhost:8000.
- بيانات باقية بعد docker compose down ثم up.

### كيف نتحقق من الإنجاز

- GET /api/v1/health يعيد 200 بعد التشغيل النظيف.
- الـ worker يلتقط job ويحدّث التقدّم داخل الحاوية.
- رفع فيديو من الواجهة ينجح ويظهر في قائمة /api/v1/videos.

### أخطاء شائعة

- استخدام localhost في DATABASE_URL داخل الحاويات.
- نسيان ربط مجلد الـ models فتظهر رسالة "checkpoint not found".
- افتراض أن GPU سيعمل تلقائياً على Windows.
- تشغيل down -v على بيانات حقيقية.

# 20 النشر في بيئة الإنتاج

خياران واقعيان لمشروع تخرّج، وما يجب الحذر منه

بعد أن صار النظام يعمل محلياً عبر Docker، يبقى سؤال لجنة المناقشة: "هل يمكن أن يعمل خارج جهازكم؟". هذا الفصل يعرض خيارين واقعيين لمشروع تخرّج، ويشرح ما يجب الحذر منه: الأسرار، وHTTPS، والترحيلات، والنسخ الاحتياطي، وفحوص الصحة. القاعدة الأولى والأهم: **القدرات والأسعار وحدود الاستخدام المجاني وتوفّر الـ GPU تتغيّر باستمرار، ويجب التحقق منها من التوثيق الرسمي لمزوّد الخدمة وقت النشر**. لا نكتب أي سعر ولا نَعِد بأي GPU مجاني في هذا الدليل.

## 20.1 الفرق بين بيئة التطوير وبيئة الإنتاج

بيئة التطوير (`development`) هي جهازنا، فيها APP_DEBUG=true والأخطاء ظاهرة والـ AUTH_ENABLED=false. بيئة الإنتاج (`production`) هي الخادم العام، وفيها يجب أن يكون APP_DEBUG=false، APP_ENV=productiong، AUTH_ENABLED=trueg، وقائمة CORS_ORIGINS مقيّدة بعنوان الواجهة الحقيقي فقط. أي قيمة تُخالف ذلك قد تكشف تفاصيل داخلية للمهاجمين.

**جدول 20-1: أهم الفروق بين التطوير والإنتاج**

| العنصر | التطوير | الإنتاج |
|---|---|---|
| APP_ENV | development | production |
| APP_DEBUG | true | false |
| AUTH_ENABLED | false | true |
| CORS_ORIGINS | عناوين محلية | عنوان الواجهة الرسمي فقط |
| STORAGE_BACKEND | local | local أو s3 حسب الخيار |
| التخزين | مجلد على الجهاز | قرص دائم أو Object Storage |

## 20.2 الخيار A: نشر بسيط للعرض

الهدف هنا ليس خدمة إنتاج حقيقية، بل رابط يعمل للجنة المناقشة. المكوّنات: الواجهة الثابتة على استضافة ملفات ثابتة، والـ Backend على خدمة تشغيل تطبيقات، وقاعدة بيانات PostgreSQL مُدارة، وتخزين كائنات للفيديوهات والدليل. هذا الخيار مناسب لأنه يخفّض عبء الإدارة إلى أدنى حد، لكنه يضع قيوداً يجب فهمها: قد تحدّ الخدمة من حجم الملف المرفوع، وقد تُنيم الخدمة عند عدم الاستخدام فيزيد زمن أول طلب (`cold start`)، وقد لا توفّر GPU أصلاً.

لذلك في الخيار A نعتمد على MODEL_DEVICE=cpu، ونرفع فيديو عرض قصيراً، ونضبط FRAME_SAMPLE_EVERY_N وMAX_FRAMES_PER_JOB بقيم معقولة حتى لا يطول زمن المعالجة. ولأن خدمة تشغيل التطبيقات قد تكون بلا قرص دائم، نستخدم STORAGE_BACKEND=s3 مع مفاتيح S3_ENDPOINT_URL و S3_BUCKET لضمان بقاء الفيديوهات والدليل.

## 20.3 الخيار B: استضافة ذاتية أو قادرة على GPU

الهدف هنا أن نتحكّم في الجهاز كاملاً. نستخدم خادماً (VPS) أو خادماً ببطاقة رسوميات، ونشغّل عليه نفس docker-compose.yml، ونضع أمامه **Reverse Proxy** مثل nginx يوزّع المسارات ويمنح شهادة **HTTPS**. هنا يمكن ضبط MODEL_DEVICE=cuda إن كان الخادم يوفّر GPU فعلاً، مع الحفاظ على نفس ملف الترحيلات ونفس الـ volumes. العيب أن المسؤولية كلها علينا: تحديثات الأمان، النسخ الاحتياطي، والمراقبة.

> **لا وعود GPU ولا أسعار**
>
> توفر الـ GPU على أي استضافة، وسعر الخطة، وحدود الـ free tier، وحجم القرص الدائم، وحد الرفع — كلها تتغيّر من مزوّد لآخر ومن شهر لآخر. لا نكتب رقماً ولا نَعِد بوصول مجاني إلى GPU. المطلوب من الفريق وقت النشر: قراءة التوثيق الرسمي للمزوّد والتحقق من كل قيمة قبل الاعتماد عليها، وتوثيق النتيجة في docs/DEPLOYMENT.md.

## 20.4 مقارنة الخيارين

**جدول 20-2: الخيار A مقابل الخيار B (المعايير تُقاس وقت النشر)**

| المعيار | الخيار A: نشر بسيط | الخيار B: ذاتي / GPU |
|---|---|---|
| التكلفة | تُحدَّد من توثيق المزوّد الرسمي | تُحدَّد من توثيق المزوّد الرسمي |

| المعيار | الخيار A: نشر بسيط | الخيار B: ذاتي / GPU |
|---|---|---|
| توفّر GPU | غالباً غير متوفر — نستخدم CPU | ممكن إن كان الخادم يوفّره فعلاً |
| حد رفع الفيديو | مقيّد من المزوّد والـ proxy | نضبطه نحن عبر nginx |
| التخزين الدائم | Object Storage (s3) | volumes على قرص الخادم |
| سرعة الشبكة | تعتمد على المزوّد | تعتمد على خط الخادم |
| زمن المعالجة | أطول (CPU + cold start) | أقصر إن توفّر GPU |
| النوم / البدء البارد | شائع في الطبقات المجانية | لا يوجد إن بقي الخادم يعمل |
| التحكّم الكامل | محدود | كامل |

## 20.5 لماذا لا تصلح الاستضافة الثابتة المجانية للـ inference الطويل

الاستضافة الثابتة (`static hosting`) تخدم ملفات HTML و JS فقط، ولا تشغّل Python ولا تتصل بقاعدة بيانات، فهي صالحة للواجهة وحدها. وحتى خدمات تشغيل التطبيقات المجانية تنيم العملية عند الخمول، وتحدّ زمن الطلب الواحد، وتحدّ الذاكرة. معالجة فيديو تستغرق دقائق وتستهلك مئات الميغابايت من الذاكرة لا يمكن أن تعيش داخل طلب HTTP واحد محدود الزمن. لهذا نفصل دائماً: الـ API يعيد 202 Accepted فوراً، والمعالجة تجري في الـ worker. حتى في الإنتاج، الـ worker هو من يحتاج الموارد الثقيلة، لا الـ API.

## 20.6 الأسرار و HTTPS والـ Migrations والنسخ الاحتياطي والـ Logs

**جدول 20-3: الجوانب التشغيلية الخمسة وكيف نتعامل معها**

| الجانب | القاعدة العملية | متغيّرات / أوامر |
|---|---|---|
| الأسرار (Secrets) | لا تُكتب في الكود ولا في `.env` المرفوع؛ تُضبط كمتغيّرات بيئة في لوحة المزوّد أو مدير أسرار | `SECRET_KEY, POSTGRES_PASSWORD, S3_SECRET_ACCESS_KEY` |
| HTTPS | شهادة TLS عبر Reverse proxy أو خدمة المزوّد؛ إعادة توجيه HTTP إلى HTTPS | `nginx + certbot` |
| Migrations | تُنفَّذ كخطوة نشر منفصلة قبل تشغيل النسخة الجديدة | `alembic upgrade head` |
| Backups | نسخة دورية من قاعدة البيانات + نسخة من مجلد الـ storage | `pg_dump` |
| Logs | سجلات منظّمة تُجمع من الـ backend والـ worker دون كشف الأسرار | `docker compose logs` |

نقطة مهمة: الترحيلات تُنفَّذ مرة واحدة في كل نشر، ولا تعتمد على إعادة تشغيل الحاوية. أمّا النسخ الاحتياطي فيجب أن يُجرَّب فعلاً (استرجاع نسخة إلى قاعدة تجريبية) لا أن يُفترض أنه يعمل. أمّا **Health checks** فتُبنى على `GET /api/v1/health`، وهو endpoint عام لا يحتاج مصادقة ويعيد حالة المكوّنات، ويستخدمه المزوّد للتأكد من أن الخدمة حيّة.

terminal — deploy and verify steps

```bash
# 1) Apply database migrations before starting the new version
$ docker compose exec backend alembic upgrade head

# 2) Rebuild and restart only the changed services
$ docker compose up -d --build backend worker

# 3) Verify health through the public URL
$ curl -s https://<your-domain>/api/v1/health

# 4) Take a database backup (store it outside the server)
$ docker compose exec db pg_dump -U safevision safevision > backup_$(date +%F).sql
```

## 20.7 قائمة تحقق النشر والترتيب الآمن

الترتيب مهم: كل خطوة تعتمد على التي قبلها. لا نبدأ بالـ API قبل قاعدة البيانات، ولا نفتح الموقع للجنة قبل فحص الصحة.

**جدول 20-4: الترتيب الآمن للنشر**

| # | الخطوة | لماذا هذا الترتيب |
|---|---|---|
| 1 | تجهيز قاعدة البيانات المُدارة والتحقق من الاتصال | لا شيء يعمل بدون قاعدة بيانات |
| 2 | ضبط كل متغيّرات البيئة والأسرار في لوحة المزوّد | منع الاعتماد على ملف محلي |
| 3 | بناء الصور ورفعها / سحب الكود على الخادم | الحصول على نسخة محدّدة قابلة للتشغيل |
| 4 | تنفيذ `alembic upgrade head` | إنشاء الجداول قبل قبول أي طلب |
| 5 | تشغيل الـ backend ثم الـ worker | الـ API أولاً ثم المعالجة |
| 6 | فحص `/api/v1/health` محلياً على الخادم | عزل الأخطاء قبل الوصول العام |

| # | الخطوة | لماذا هذا الترتيب |
|---|---|---|
| 7 | ربط الـ Reverse proxy و HTTPS | تأمين الاتصال للمستخدم |
| 8 | بناء وخدمة الواجهة بعنوان API الصحيح | Vite يدمج العنوان وقت البناء |
| 9 | اختبار رحلة كاملة: رفع ← معالجة ← واقعة ← تقرير | إثبات أن النظام يعمل من طرفه لطرفه |
| 10 | جدولة النسخ الاحتياطي وتجربة استرجاع | ضمان عدم فقدان النتائج |

### نصيحة أخيرة

وثّق كل قرار نشر في docs/DEPLOYMENT.md: اسم المزوّد، الخيار (A أو B)، قيم الحدود التي تحقّقت منها من التوثيق الرسمي، وعنوان الرابط. هذا الملف هو ما تعرضه اللجنة عند سؤال "كيف نشرتم النظام؟".

## خلاصة الفصل 20

### ما تعلّمناه

فرّقنا بين التطوير والإنتاج، وعرضنا خيارين: نشر بسيط للعرض على CPU، واستضافة ذاتية قابلة لـ GPU مع Reverse proxy و HTTPS. وأكّدنا أن الأسعار والحدود وتوفّر GPU تُتحقَّق من التوثيق الرسمي وقت النشر، وأن الاستضافة الثابتة لا تصلح للـ inference الطويل.

### ما يجب تنفيذه

- اختيار الخيار A أو B وتوثيق السبب.
- ضبط APP_ENV=production APP_DEBUG=falseg AUTH_ENABLED=trueg.
- إعداد HTTPS ونسخ احتياطي مجرَّب.
- كتابة docs/DEPLOYMENT.md.

### المخرجات المتوقعة

- رابط عام يعمل للواجهة والـ API عبر HTTPS.
- قاعدة بيانات مُدارة مع نسخة احتياطية مجرَّبة.
- فحص صحة ناجح على الرابط العام.

### كيف نتحقق من الإنجاز

- فتح الرابط من جهاز خارج شبكة الفريق.
- تنفيذ رحلة كاملة من الرفع حتى التقرير على النسخة المنشورة.
- التأكد من أن الأسرار غير ظاهرة في أي سجل أو مستودع.

### أخطاء شائعة

- كتابة كلمات المرور داخل الكود أو رفع .env.
- تشغيل الـ API قبل تنفيذ الترحيلات.
- افتراض وجود GPU أو قرص دائم دون التحقق.
- ترك APP_DEBUG=true على الإنتاج.

# 21 الـ CI/CD وسير عمل GitHub

كيف يعمل خمسة طلاب على مستودع واحد دون فوضى

أكبر خطر في مشروع فريق هو فقدان العمل أو دمج كود يكسر ما بناه غيرك. هذا الفصل يثبّت قواعد عمل بسيطة على مستودع safevision-ai، ثم يعرض ملف CI اختيارياً يفحص الكود تلقائياً عند كل تغيير. القاعدة الحاكمة: **لا أحد يعمل مباشرة على الفرع main**.

## 21.1 نموذج الفروع الذي نستخدمه

**Branch (فرع):** نسخة مستقلة من الكود يمكن التعديل عليها دون التأثير على الفرع الرئيسي. نستخدم نموذجاً بسيطاً يكفي خمسة أعضاء: فرع main يمثّل النسخة المستقرّة دائماً، وفرع مؤقت لكل مهمة يُنشأ منه ثم يُدمج فيه عبر Pull Request.

**جدول 21-1: قواعد التسمية**

| النوع | الصيغة | أمثلة |
|---|---|---|
| فرع ميزة | `feat/<short-name>` | `feat/video-upload`, `feat/incident-list` |
| إصلاح خطأ | `fix/<short-name>` | `fix/cors-origins`, `fix/job-progress` |
| توثيق | `docs/<short-name>` | `docs/api-contract` |
| الوسوم | `v<major>.<minor>.<patch>` | `v0.1.0`, `v1.0.0` |

## 21.2 Pull Requests ومراجعة الكود

**Pull Request (PR):** طلب دمج فرعك في main مع عرض التغييرات للنقاش. لا يُدمج أي PR قبل مراجعة عضو آخر على الأقل. تُبقي الـ PR صغيراً: مهمة واحدة، تغيير محدود، وصف واضح لما تغيّر وكيف اختُبر. هذا يجعل المراجعة سريعة ويمنع تعارضات الدمج الكبيرة.

**PR جيّد**

- مهمة واحدة فقط ورسالة واضحة.
- يربط بـ Issue برقمه.
- يشرح كيف اختُبر يدوياً.
- صغير بما يكفي لمراجعته في دقائق.

**PR سيّئ**

- يخلط بين ميزات متعددة.
- يحتوي ملفات كبيرة أو أسراراً بالخطأ.
- بدون وصف ولا خطوات اختبار.
- يبقى مفتوحاً أسابيع فيتعارض مع غيره.

## 21.3 حل تعارضات الدمج ورسائل الـ Commit

يحدث **Merge Conflict** عندما يعدّل فرعان نفس الأسطر. الحل دائماً: اسحب أحدث نسخة من main إلى فرعك مبكراً، ثم حلّ التعارض يدوياً واختبار الكود قبل الدمج. أما رسائل الـ commit فنتبع صيغة مختصرة type(scope): message بالإنجليزية.

bash — terminal — daily Git workflow

```
# Start a task from an up-to-date main
$ git checkout main
$ git pull origin main
$ git checkout -b feat/video-upload

# Work, then stage and commit in small logical steps
$ git add backend/app/api/v1/videos.py
$ git commit -m "feat(videos): add multipart upload endpoint"

# Pull main into your branch early to resolve conflicts while they are small
$ git fetch origin
$ git rebase origin/main

# Push and open a Pull Request
$ git push -u origin feat/video-upload

# After the PR is reviewed and merged, clean up the branch
$ git checkout main
$ git pull origin main
$ git branch -d feat/video-upload
```

**قاعدة الدمج المبكر**

كلما طال بقاء الفرع منفصلاً كبرت التعارضات. القاعدة: اسحب أحدث نسخة من main إلى فرعك مرة كل يوم عمل على الأقل، ولا تُبقِ فرعاً بدون دمج أكثر من بضعة أيام. وإن كبر التعارض، فالحل ليس إجبار الدمج (push --force) على فرع مشترك، بل حلّ التعارض ملفاً ملفاً واختبار الكود بعد الحل.

**جدول 21-2: أمثلة رسائل commit مقبولة**

| النوع | المعنى | مثال |
|---|---|---|
| `feat` | ميزة جديدة | `feat(jobs): expose progress field` |
| `fix` | إصلاح خطأ | `fix(ai): clamp bbox to frame size` |
| `docs` | توثيق فقط | `docs(readme): add run instructions` |
| `refactor` | إعادة تنظيم بدون تغيير السلوك | `refactor(services): split incident service` |
| `test` | إضافة أو تعديل اختبارات | `test(rules): add ppe cooldown case` |

## 21.4 Issues و Milestones و Release Tags

نستخدم **Issues** لتسجيل كل مهمة أو خطأ، و**Milestones** لتجميع الإصدارات في مجموعات قابلة للقياس (مثل: الأساس، خط المعالجة، الواجهة، العرض النهائي). وكلما وصل النظام إلى حالة مستقرة نضع وسم إصدار **Tag** بالإنجليزية. هذا يعطي الفريق خط أساس يعود إليه عند كسر شيء.

ويُفضّل أن يحمل كل Issue قائمة تحقق صغيرة (`definition of done`) تُغلق المهمة فقط عند إتمامها، وأن يُربط اسم الفرع برقم الـ Issue ليعرف المراجع سياق التغيير فوراً.

## 21.5 GitHub Actions اختياري

**GitHub Actions:** خدمة تشغيل أوامر تلقائياً عند كل push أو PR. الـ CI مفيد فعلاً بعد أن يصبح لدينا اختبارات قابلة للتشغيل. الملف التالي يفحص صياغة Python، ويشغّل الاختبارات، ويبني الواجهة ويفحص الأنواع، ويبني صورة Docker للـ backend. نستخدم services لتوفير PostgreSQL مؤقتة للاختبارات، ونأخذ الأسرار من secrets لا من الكود.

**.github/workflows/ci.yml** yaml

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  backend:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: safevision
          POSTGRES_PASSWORD: safevision
          POSTGRES_DB: safevision_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U safevision"
          --health-interval 10s --health-timeout 5s --health-retries 5
    env:
      DATABASE_URL: postgresql+psycopg://safevision:safevision@localhost:5432/safevision_test
      MODEL_DEVICE: cpu
      AUTH_ENABLED: "false"
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r backend/requirements.txt
      - run: ruff check backend
      - run: pytest backend/tests -q

  frontend:
    runs-on: ubuntu-latest
```

```
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
          cache-dependency-path: frontend/package-lock.json
      - run: npm ci
      - run: npm run lint
      - run: npm run build
      - run: npx tsc --noEmit

  docker:
    runs-on: ubuntu-latest
    needs: backend
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t safevision-backend ./backend
```

**تحذير أمني صريح**

لا تُرفع إلى GitHub أبداً: فيديوهات العملاء أو أي فيديو حقيقي، ملفات الـ checkpoints الضخمة (/models)، ملف .env الذي يحوي كلمات مرور، أو أي مفتاح S3_SECRET_ACCESS_KEY. كل هذه مستثناة في .gitignore وفي .dockerignore. إن رُفع سرّ بالخطأ فالحل ليس حذف الملف في commit جديد، بل إلغاء السرّ فوراً من المصدر لأنه يبقى في تاريخ Git.

## 21.6 متى يصبح الـ CI/CD مفيداً فعلاً

الـ CI ليس هدفاً في ذاته، بل أداة. يصبح مفيداً عندما تتحقق ثلاثة شروط: أن توجد اختبارات يمكن تشغيلها آلياً، وأن يكون الكود على فرع main صالحاً للتشغيل دائماً، وأن يستطيع عضو جديد تشغيل المشروع من التعليمات وحدها. قبل ذلك يتحوّل الـ CI إلى عمل زائد يستهلك وقت الفريق.

لذلك الترتيب الصحيح: نكتب الكود والاختبارات أولاً، ونضبط .gitignore، ثم نضيف ملف الـ workflow. البوابة الأولى (**gate**) هي تشغيل pytest وbuild npm run محلياً قبل الـ push، لأن تشغيل الأمر على جهازك أسرع بكثير من انتظار CI. أمّا CI فقيمته الحقيقية أنه يمنع دمج كود يكسر فرع main دون أن يلاحظه أحد.

نقطة أخيرة: لا نعتمد على CI لتشغيل الـ AI على GPU. الاختبارات في CI تعمل على CPU وبفيديو صناعي صغير جداً (الفصل 22)، وأي شيء يحتاج GPU أو بيانات حقيقية يبقى اختباراً يدوياً موثّقاً. هكذا تبقى بوابة CI سريعة وغير مكلفة، ولا تفشل بسبب موارد غير متوفرة.

### خلاصة الفصل 21

**ما تعلّمناه**

ثبّتنا نموذج فروع بسيطاً: main مستقر دائماً، وفرع لكل مهمة، ودمج عبر Pull Request بمراجعة. واتفقنا على صيغة رسائل الـ commit، وعلى Issues وMilestones وTags، وعلى ملف CI اختباري يفحص الـ backend والـ frontend ويبني الصورة.

**ما يجب تنفيذه**

- تفعيل حماية فرع main ومنع الدفع المباشر.
- إنشاء قالب للـ Pull Request يطلب وصف التغيير وخطوات الاختبار.
- التحقق من .gitignore قبل أول commit.
- إضافة .github/workflows/ci.yml عند جاهزية الاختبارات.

**المخرجات المتوقعة**

- مستودع فيه فرع main نظيف ووسوم إصدار.
- لوحة Issues وMilestones تغطّي كل مهام الفصل 26.
- تشغيل CI أخضر على آخر PR.

**كيف نتحقق من الإنجاز**

- لا يمكن الدفع مباشرة إلى main.
- كل PR له مراجع واحد على الأقل ووصف واضح.

- لا يوجد ملف سرّ أو فيديو داخل المستودع.

**أخطاء شائعة**

- العمل مباشرة على main من عدة أعضاء.
- ترك فروع مفتوحة طويلاً حتى تكبر التعارضات.
- رفع ملف .env أو مجلد models/ بالخطأ.
- توقّع أن CI يصلح الكود بدل أن يكتشف الأخطاء فقط.

# 22 الاختبارات وضمان الجودة

كيف نتأكد أن النظام يعمل فعلاً قبل المناقشة

هذا الفصل يحوّل الثقة إلى دليل: اختبارات وحدات، وتكامل بين الـ API وقاعدة البيانات، ورحلة كاملة من الطرف للطرف، وجدول حالات اختبار تُملأ خاناته بعد التنفيذ. القاعدة: **لا نكتب نتيجة لم تُنفَّذ**.

## 22.1 ثلاثة مستويات من الاختبار

**جدول 22-1: مستويات الاختبار الثلاثة ونطاق كل واحد**

| المستوى | ما يختبره | أدوات | سرعة |
|---|---|---|---|
| `Unit Testing` | دالة واحدة معزولة: فلترة الثقة، تقييم قاعدة، حساب تقرير | `pytest` | أجزاء من الثانية |
| `Integration Testing` | عدة مكوّنات معاً: API + قاعدة بيانات، Job + AI، وقائع + تقارير | `pytest + httpx` | ثوانٍ |
| `End-to-End Testing` | رحلة المستخدم الكاملة من الرفع حتى التقرير | تشغيل كامل + المتصفح | دقائق |

**Unit Test (اختبار وحدة):** يفحص أصغر قطعة منطق بمعزل، بلا قاعدة بيانات أو GPU أو شبكة. و**Integration Test (اختبار تكامل):** يفحص أن قطعتين تعملان معاً، مثل رفع الـ API فيديو فيظهر في جدول videos.

## 22.2 اختبارات الوحدة: ماذا نغطّي بالضبط

- التحقق من الفيديو: امتداد مسموح/مرفوض، تجاوز MAX_UPLOAD_MB، ملف تالف.
- خريطة الأصناف: اسم class من الـ Model إلى نوع واقعة صحيح.
- فلترة الثقة: كشف أقل من DETECTION_CONF_THRESHOLD يُستبعد.
- تقييم القواعد: مخالفة PPE تتطلب إطارات متتالية، والدخان عتبة أعلى.
- منع تكرار الوقائع: واقعة داخل INCIDENT_COOLDOWN_SECONDS لا تُنشأ مرتين.
- حسابات التقارير: تجميع الوقائع يومياً يعطي نفس المجموع.

## 22.3 اختبارات التكامل: نقاط الوصل الحرجة

نركّز على وصلات الخطأ الشائعة: API مع قاعدة البيانات، الرفع مع التخزين، Job مع AI، مخرجات الـ AI مع الوقائع، الوقائع مع التقارير، وFrontend مع Backend. في كل اختبار نتحقق من رمز الحالة (201، 202، 200) ومن شكل الاستجابة المطابق للعقد ومن وصول البيانات للجدول الصحيح.

## 22.4 اختبار الرحلة الكاملة (End-to-End)

1. فتح الواجهة والتأكد من نجاح نداء GET /api/v1/health.
2. رفع فيديو صغير عبر POST /api/v1/videos وظهوره في القائمة.
3. بدء المعالجة POST /api/v1/videos/{video_id}/process والحصول على 202.
4. متابعة GET /api/v1/jobs/{job_id} حتى completed.
5. ظهور وقائع في GET /api/v1/incidents.
6. فتح تفاصيل واقعة وعرض صورة الدليل عبر /api/v1/evidence/{evidence_id}/file.
7. تنفيذ مراجعة PATCH /api/v1/incidents/{incident_id} وتغيّر الحالة.
8. فتح التحليلات GET /api/v1/analytics/overview وظهور الأرقام.
9. تصدير التقرير GET /api/v1/reports/export وفتح ملف CSV.
10. حذف الفيديو والتأكد من اختفاء وقائعه وملفاته.

## 22.5 جدول حالات الاختبار

الخانتان **Actual** و**Pass/Fail** تُتركان فارغتين وتُملآن بعد التنفيذ الفعلي فقط.

**جدول 22-2: حالات الاختبار (يُملأ بعد التنفيذ)**

| Test ID | Scenario | Preconditions | Steps | Expected | Actual | Pass/Fail |
|---|---|---|---|---|---|---|
| TC-01 | رفع فيديو صالح | Backend و DB يعملان | اختيار ملف mp4 وإرساله | 201 status=uploadedg | | |
| TC-02 | رفض امتداد غير مسموح | نفس ما سبق | رفع ملف .txt | 400 مع رسالة واضحة | | |
| TC-03 | بدء معالجة | يوجد فيديو مرفوع | نداء process | 202 مع JobOut queued | | |
| TC-04 | توليد وقائع | job مكتمل | فتح قائمة الوقائع | وقائع بأنواع من الـ enum | | |
| TC-05 | منع job مزدوج | job نشط لنفس الفيديو | نداء process مرة أخرى | رفض واضح (تعارض) | | |
| TC-06 | مراجعة واقعة | واقعة pending_review | PATCH بالحالة confirmed | تغيّر الحالة وسجل مراجعة | | |
| TC-07 | تصدير CSV | توجد وقائع | نداء reports/export | ملف CSV صالح | | |
| TC-08 | فشل الـ checkpoint | مسار model خاطئ | بدء المعالجة | job failed ورسالة خطأ | | |

## 22.6 الاختبار بدون GPU وفيديو صناعي صغير

الاختبارات الآلية يجب ألا تحتاج GPU ولا فيديو حقيقي. ننشئ **fixture** — بيانات ثابتة تُجهَّز تلقائياً — على شكل فيديو صناعي صغير جداً نولّده بإطارات ملوّنة، ونضبط MODEL_DEVICE=cpu وAUTH_ENABLED=falseg، ونستبدل الـ Model الثقيل بـ **stub** يعيد كشوفات ثابتة.

## 22.7 ملف الـ fixtures

يجهّز هذا الملف بيانات الاختبار المشتركة: فيديو صناعي صغير، وعميل HTTP، وجلسة قاعدة بيانات تُلغى في النهاية.

backend/tests/conftest.py

```python
import numpy as np
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.main import app

# Tests always run on CPU and without auth so they work in CI and on any laptop.
CPU_ENV = {"MODEL_DEVICE": "cpu", "AUTH_ENABLED": "false"}


@pytest.fixture(scope="session")
def client() → TestClient:
    return TestClient(app)


@pytest.fixture()
def tiny_video(tmp_path):
    """Write a very short synthetic mp4 with a few solid-colour frames."""
    import cv2

    path = tmp_path / "synthetic_3f.mp4"
    writer = cv2.VideoWriter(
        str(path), cv2.VideoWriter_fourcc(*"mp4v"), 5.0, (64, 64)
    )
    for colour in ((10, 10, 10), (200, 200, 200), (10, 10, 10)):
        frame = np.full((64, 64, 3), colour, dtype=np.uint8)
        writer.write(frame)
    writer.release()
    return path


@pytest.fixture()
def db_session():
    """A transaction that is rolled back after each test."""
    engine = create_engine("sqlite+pysqlite: ///:memory:", future=True)
    TestingSession = sessionmaker(bind=engine, future=True)
    session = TestingSession()
    try:
        yield session
    finally:
        session.rollback()
        session.close()
```

## 22.8 اختبار قواعد السلامة

نختبر تقييم القواعد بكشوفات مصنوعة يدوياً بلا Model أو GPU: مخالفة PPE أقل من الإطارات المطلوبة لا تُنتج واقعة، وواقعة داخل فترة التهدئة لا تُكرَّر.

backend/tests/test_rules.py

```python
from app.ai.rules import evaluate_ppe_violation, should_create_incident


def make_detection(frame_number, class_name="person_no_helmet", confidence=0.9):
    return {
        "class_name": class_name,
        "confidence": confidence,
        "bbox": [10, 10, 50, 50],
        "frame_number": frame_number,
        "timestamp_seconds": frame_number / 5.0,
    }


def test_single_frame_below_confirm_threshold_makes_no_incident():
    # INCIDENT_CONFIRM_FRAMES is higher than one, so a single frame is not enough.
    detections = [make_detection(1)]
    assert evaluate_ppe_violation(detections, confirm_frames=3) is None


def test_repeated_frames_produce_one_incident():
    detections = [make_detection(n) for n in range(1, 6)]
    incident = evaluate_ppe_violation(detections, confirm_frames=3)
    assert incident is not None
    assert incident["incident_type"] == "ppe_violation"
    assert incident["detection_count"] >= 3


def test_cooldown_blocks_duplicate_incident():
    last_seen_seconds = 4.0
    new_detection_seconds = 6.0
    assert should_create_incident(
        last_seen_seconds, new_detection_seconds, cooldown_seconds=10.0
    ) is False
```

## 22.9 اختبار تكامل الـ API مع قاعدة البيانات

يمرّ هذا الاختبار عبر الـ API الحقيقي ويتحقق من رمز الحالة وشكل الاستجابة وتغيّر الحالة. نستخدم TestClient أو httpx.AsyncClient للـ endpoints غير المتزامنة.

backend/tests/test_api_incidents.py

```python
def test_list_incidents_returns_pagination_envelope(client):
    response = client.get("/api/v1/incidents")
    assert response.status_code == 200
    body = response.json()
    # The pagination envelope is defined in the API contract.
    for key in ("items", "total", "page", "page_size", "pages"):
        assert key in body


def test_incident_status_change_is_recorded(client):
    # Assumes a seeded incident exists in the test database.
    incident_id = client.get("/api/v1/incidents").json()["items"][0]["id"]
    response = client.patch(
        f"/api/v1/incidents/{incident_id}",
        json={"status": "confirmed", "note": "checked by reviewer"},
    )
    assert response.status_code == 200
    assert response.json()["status"] == "confirmed"


def test_reject_invalid_status_value(client):
    incident_id = client.get("/api/v1/incidents").json()["items"][0]["id"]
    response = client.patch(
        f"/api/v1/incidents/{incident_id}", json={"status": "not_a_real_status"}
    )
    assert response.status_code == 422
```

## 22.10 اختبار الواجهة بـ Vitest

على الـ Frontend نستخدم **Vitest** لاختبار الدوال النقية: تنسيق الأرقام، والتحقق من الامتداد قبل الرفع، وتحويل قيم الـ enums إلى نصوص للعرض.

frontend/src/utils/format.test.ts

```typescript
import { describe, expect, it } from "vitest";
import { formatProgress, isAllowedVideo } from "./format";

describe("isAllowedVideo", () => {
  it("accepts mp4 and rejects txt", () => {
    expect(isAllowedVideo("clip.mp4")).toBe(true);
    expect(isAllowedVideo("notes.txt")).toBe(false);
  });
});

describe("formatProgress", () => {
  it("rounds a fraction to a percentage string", () => {
    expect(formatProgress(0.425)).toBe("43%");
  });
});
```

### أوامر التشغيل

Backend: pytest backend/tests -q. الـ Frontend: npm run test -- --run. شغّلهما محلياً قبل كل push؛ أسرع من انتظار CI.

## خلاصة الفصل 22

### ما تعلّمناه

قسّمنا الاختبارات إلى وحدة وتكامل ورحلة كاملة، وجهّزنا فيديو صناعياً يعمل بدون GPU، وربطنا كل اختبار برمز الحالة وشكل الاستجابة في العقد.

### ما يجب تنفيذه

- كتابة backend/tests/conftest.py بالـ fixtures.
- كتابة اختبارات القواعد والـ API وتشغيلها محلياً.
- تعبئة جدول حالات الاختبار بعد تنفيذ فعلي.
- إضافة Vitest لاختبار دوال الواجهة.

### المخرجات المتوقعة

- حزمة اختبارات تعمل بالكامل على CPU.
- جدول حالات اختبار مكتمل بالنتائج الحقيقية.
- تقرير مختصر بعدد الاختبارات الناجحة والفاشلة.

### كيف نتحقق من الإنجاز

- تشغيل pytest دون أخطاء جمع أو فشل غير متوقع.
- كل صف في جدول الحالات له نتيجة مكتوبة وسبب.
- إمكانية تشغيل الاختبارات على جهاز بلا GPU.

### أخطاء شائعة

- كتابة Expected في خانة Actual لمجرد أن الاختبار "يجب" أن ينجح.
- الاعتماد على بيانات حقيقية في الاختبارات الآلية.
- اختبارات تعتمد على ترتيب التنفيذ أو تترك بيانات في قاعدة البيانات.
- نسيان تشغيل الاختبارات قبل كل push.

# 23 تقييم الـ AI وأداء النظام

تقييم الـ Models منفصلة عن تقييم التطبيق، وبأرقام حقيقية فقط

نفصل تقييم الـ **Model** (هل الكشف دقيق؟) عن تقييم **التطبيق** (هل يُنتج وقائع مفيدة؟)، ونثبّت قاعدة صارمة: **لا نكتب أي رقم لم نقِسه فعلاً**؛ كل الجداول هنا فارغة وموسومة (يُملأ بعد التجربة).

## 23.1 لماذا نفصل تقييم الـ Model عن تقييم النظام

الـ Model يرى إطارات ويعيد صناديق وفئات، أمّا المستخدم فيرى **واقعة** لها وقت وخطورة وحالة مراجعة. بينهما قواعد السلامة (الإطارات المتتالية، فترة التهدئة، منطقة الخطر) والتتبّع واقتطاع الدليل؛ لذلك قد يكون الـ Model ممتازاً في الكشف ومع ذلك ينتج نظاماً مزعجاً لكثرة الوقائع المكرّرة أو المفقودة.

## 23.2 مقاييس الكشف: Precision و Recall و F1

**Precision (الدقة):** نسبة الكشوفات الصحيحة. **Recall (الاستدعاء):** نسبة الأهداف المكتشفة. **F1** المتوسط التوافقي بينهما. وفي السلامة الـ Recall غالباً أهم، لأن تفويت حادث أخطر من إنذار زائد.

**جدول 23-1: المقاييس ومعناها وما يمثّله الخطأ فيها**

| المقياس | المعنى | الخطأ المقابل |
|---|---|---|
| Precision | نسبة الكشوفات الصحيحة من كل الكشوفات | False Positive: إنذار كاذب يُتعب المراجع |
| Recall | نسبة الأهداف المكتشفة من كل الأهداف | False Negative: حادث مفوّت |
| F1 | توازن بين الدقة والاستدعاء | رقم مضلّل إن اختلّ أحد الطرفين |
| mAP | متوسط الدقة عبر عتبات الثقة وباقي الأصناف | لا يعكس سلوك النظام في الإنتاج |

## 23.3 Confusion Matrix و mAP

**Confusion Matrix (مصفوفة الالتباس):** جدول يقارن التنبؤ بالحقيقة ويُظهر أين يخطئ الـ Model (خلط بين "خوذة" و"بدون خوذة"، أو تفويت الدخان)؛ وهو أنفع من رقم واحد لأن كل خطأ له وزن مختلف في السلامة. أمّا **mAP** فيجمع الدقة عبر عتبات الثقة ويُستخدم لمقارنة نسختين على نفس مجموعة الاختبار.

## 23.4 مقاييس التتبّع (Tracking)

- **ID switches:** عدد المرات التي "قفز" فيها معرّف التتبّع من شخص إلى آخر.
- **Track fragmentation:** انقطاع مسار شخص ثم بدؤه من جديد بنفس المعرّف أو بمعرّف آخر.
- **Track continuity:** نسبة الإطارات التي بقي فيها المسار متصلاً.

خطأ التتبّع يفسد قواعد "الإطارات المتتالية"، فقد تُدمج واقعتان أو تتكرّر واقعة بسبب انقطاع المسار.

## 23.5 التقييم على مستوى الواقعة

نقيس ما يراه المستخدم: هل أُنشئت واقعة عند المخالفة؟ هل فُقدت أو تكرّرت؟ نُعطي كل واقعة حقيقية صفة مرجعية يدوياً ثم نقارنها بالمخرجات بعد قواعد السلامة، فتكون النتيجة: صحيحة (`true incident`)، زائفة (`false incident`)، ومفقودة (`missed incident`). هذا ما يهم مسؤول السلامة، وليس mAP.

## 23.6 مقاييس أداء النظام

**جدول 23-2: مقاييس الأداء التشغيلي وكيف تُقاس**

| المقياس | التعريف | كيف يُقاس |
|---|---|---|
| FPS | عدد الإطارات المعالجة في الثانية | frames_processed / زمن المعالجة |
| زمن معالجة الفيديو | من بدء الـ job حتى اكتماله | finished_at − started_at |
| زمن استجابة الـ API | زمن رد نقاط القراءة | قياس الطلبات على `/api/v1/incidents` |
| نسبة فشل الـ Jobs | jobs بحالة failed من الإجمالي | استعلام على `processing_jobs` |

| المقياس | التعريف | كيف يُقاس |
|---|---|---|
| نسبة نجاح توليد الدليل | وقائع لها صورة دليل من الإجمالي | مقارنة `incidents` بـ `evidence` |

## 23.7 لماذا لا تضمن mAP العالية نظام إدارة وقائع مفيداً

الـ mAP يقيس الكشف على مجموعة اختبار ثابتة بإعدادات مثالية، لكن النظام الحقيقي يختلف:

- **العتبات:** عتبة منخفضة ترفع الـ Recall وتغرق المراجع في إنذارات كاذبة، والعكس صحيح.
- **القواعد الزمنية:** الإطارات المتتالية وفترة التهدئة تحدّدان إن كانت الواقعة تُنشأ أم تُدمج أو تُكرّر.
- **التتبّع:** انقطاع المسار أو قفز المعرّف يفسد ربط الإطارات في واقعة واحدة.
- **التوزيع الحقيقي:** بيانات المصنع تختلف عن بيانات الاختبار (إضاءة، زوايا، ازدحام).
- **جودة الدليل:** واقعة صحيحة بلا دليل واضح أقل فائدة من واقعة مع دليل واضح.

لذلك نعرض دائماً mAP للـ Model مع تقييم الواقعة للنظام ككل.

## 23.8 خطة تقييم خط الأنابيب الكامل

نقيّم النظام على مقاطع حسب الظرف: إضاءة عادية وضعيفة، حجب جزئي (`occlusion`)، زوايا مختلفة، مشاهد مزدحمة، وحالات صعبة. نُسجّل نفس المقاييس لكل ظرف لأن الضعف غالباً يظهر في ظرف واحد.

**جدول 23-3: نتائج تقييم الـ Model (يُملأ بعد التجربة)**

| الـ Model / الصنف | Precision | Recall | F1 | mAP | ملاحظات |
|---|---|---|---|---|---|
| PPE detector | | | | | |
| Fire / Smoke detector | | | | | |
| Person tracker | | | | | |

**جدول 23-4: تقييم على مستوى الواقعة (يُملأ بعد التجربة)**

| الظرف | وقائع صحيحة | وقائع زائفة | وقائع مفقودة | ملاحظات |
|---|---|---|---|---|
| إضاءة عادية | | | | |
| إضاءة ضعيفة | | | | |
| حجب جزئي | | | | |
| مشهد مزدحم | | | | |

**جدول 23-5: مقاييس أداء النظام (يُملأ بعد التجربة)**

| المقياس | القيمة المقاسة | جهاز / إعداد القياس |
|---|---|---|
| FPS على CPU | | |
| FPS على GPU إن توفّر | | |
| زمن معالجة فيديو العرض | | |
| زمن استجابة API (متوسط) | | |
| نسبة فشل الـ Jobs | | |
| نسبة نجاح توليد الدليل | | |

## 23.9 سكربت التقييم

يقرأ السكربت الحقيقة المرجعية (ground_truth.json) وتنبؤات النظام ثم يحسب المقاييس. مخرجاته **شكلية فقط**: لا رقم حقيقي فيها لأن الأرقام تأتي من التشغيل الفعلي.

scripts/evaluate_model.py

```python
"""Compute detection metrics from ground truth and system predictions.

Usage:
    python scripts/evaluate_model.py --truth data/gt.json --pred data/pred.json

Expected output shape (numbers are produced only by a real run):
    class: ppe_violation
      precision: 0.0000
      recall:    0.0000
```

```
      f1:        0.0000
      tp: 0  fp: 0  fn: 0
"""
import argparse
import json


def load_pairs(path):
    with open(path, encoding="utf-8") as handle:
        return json.load(handle)["items"]


def iou(box_a, box_b):
    ax1, ay1, ax2, ay2 = box_a
    bx1, by1, bx2, by2 = box_b
    ix1, iy1 = max(ax1, bx1), max(ay1, by1)
    ix2, iy2 = min(ax2, bx2), min(ay2, by2)
    inter = max(0, ix2 - ix1) * max(0, iy2 - iy1)
    area_a = (ax2 - ax1) * (ay2 - ay1)
    area_b = (bx2 - bx1) * (by2 - by1)
    union = area_a + area_b - inter
    return inter / union if union else 0.0


def match_frame(truth_items, pred_items, iou_threshold=0.5):
    """Greedy one-to-one matching per frame; returns tp, fp, fn counts."""
    used = set()
    tp = 0
    for t in truth_items:
        best_idx, best_iou = -1, iou_threshold
        for idx, p in enumerate(pred_items):
            if idx in used or p["class_name"] ≠ t["class_name"]:
                continue
            score = iou(t["bbox"], p["bbox"])
            if score ⩾ best_iou:
                best_idx, best_iou = idx, score
        if best_idx ⩾ 0:
            used.add(best_idx)
            tp += 1
    return tp, len(pred_items) - len(used), len(truth_items) - tp


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--truth", required=True)
    parser.add_argument("--pred", required=True)
    parser.add_argument("--iou", type=float, default=0.5)
    args = parser.parse_args()

    truth = load_pairs(args.truth)
    pred = load_pairs(args.pred)

    tp = fp = fn = 0
    for frame_number in {t["frame_number"] for t in truth}:
        t_items = [t for t in truth if t["frame_number"] == frame_number]
        p_items = [p for p in pred if p["frame_number"] == frame_number]
        a, b, c = match_frame(t_items, p_items, args.iou)
        tp, fp, fn = tp + a, fp + b, fn + c

    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0.0

    # These are format examples; real values appear only after a real run.
    print(f"precision: {precision:.4f}")
    print(f"recall:    {recall:.4f}")
    print(f"f1:        {f1:.4f}")
    print(f"tp: {tp}  fp: {fp}  fn: {fn}")


if __name__ == "__main__":
    main()
```

**قاعدة صارمة**

لا تُنسخ أي قيمة إلى التقارير أو العرض إلا من تشغيل فعلي موثّق (الفيديو، التاريخ، الجهاز، الإعدادات). أي رقم بلا مصدر قياس يُحذف.

## خلاصة الفصل 23

### ما تعلّمناه

فرّقنا بين مقاييس الـ Model (Precision, Recall, F1, mAP, Confusion Matrix) ومقاييس التتبّع والنظام، وثبّتنا أن التقييم الحقيقي على مستوى الواقعة، وشرحنا لماذا لا تضمن mAP عالية نظاماً مفيداً.

### ما يجب تنفيذه

- تجهيز ملف الحقيقة المرجعية لفيديو التقييم.
- تشغيل scripts/evaluate_model.py وتسجيل مخرجاته.
- تعبئة جداول 23-3 و23-4 و23-5 بعد التشغيل الفعلي فقط.
- قياس زمن المعالجة وFPS على الجهاز المستخدم فعلاً.

### المخرجات المتوقعة

- تقرير تقييم يحتوي مقاييس الـ Model ومقاييس الواقعة معاً.
- جدول أداء تشغيلي يوضّح زمن المعالجة ونسبة فشل الـ Jobs.
- ملاحظات صريحة عن الظروف التي ضعف فيها الأداء.

### كيف نتحقق من الإنجاز

- كل رقم في التقرير له مصدر قياس مكتوب.
- لا يوجد جدول نتيجة ممتلئ قبل تنفيذ التجربة.
- التمييز واضح بين mAP وبين مقياس الواقعة.

### أخطاء شائعة

- عرض mAP فقط وإغفال تقييم الواقعة.
- إعلان نتيجة GPU كنتيجة عامة.
- مقارنة نتائج من مجموعات اختبار مختلفة.
- كتابة أرقام متوقّعة قبل التشغيل.

# 24 معالجة الأخطاء والـ Logging والمراقبة

كيف يتصرف النظام عندما تسوء الأمور

النظام الجيد لا يُقاس بعدد المرات التي يعمل فيها، بل بسلوكه عندما يفشل شيء. هذا الفصل يحدّد ماذا يحدث عند تلف فيديو أو نقص checkpoint أو انقطاع قاعدة البيانات، وكيف نجعل السجلات تكفي لتشخيص المشكلة بسرعة دون كشف أي سرّ، وكيف نمنع تكرار الواقعة عند إعادة المحاولة.

## 24.1 مبدأ عام: الفشل متوقّع وليس استثناءً

معالجة الفيديو عملية طويلة تمرّ عبر ملفات وبطاقة رسوميات وقاعدة بيانات وقرص، وأي واحدة منها قد تفشل. لذلك نتعامل مع الفشل كحالة عادية لها مسار واضح: يُسجَّل الخطأ بسياقه، تُحدَّث حالة الـ job إلى failed، ويُخزَّن error_message، ويستطيع المستخدم رؤية السبب وإعادة المحاولة. الممنوع أن يتوقف النظام بصمت أو أن يترك job عالقاً في running للأبد.

## 24.2 سيناريوهات الفشل التي نغطّيها

**جدول 24-1: سيناريوهات الفشل التسعة والسلوك المتوقّع**

| السيناريو | السلوك المتوقّع |
|---|---|
| فيديو تالف أو غير قابل للفك | رفض عند التحقق أو فشل الـ job مع رسالة واضحة، دون crash |
| حجم ملف يتجاوز MAX_UPLOAD_MB | رفض الطلب قبل حفظ الملف كاملاً |
| مسار checkpoint مفقود | فشل الـ job مبكراً برسالة تذكر المسار المطلوب |
| ذاكرة GPU غير كافية | التراجع إلى CPU إن أمكن، وإلا فشل واضح لا انهيار |
| قاعدة البيانات غير متاحة | الـ API يعيد خطأ خدمة، والـ worker ينتظر ويعيد المحاولة |
| فشل الـ job لأي سبب | حالة failed + error_message + عدد المحاولات |
| انقطاع اتصال الـ Frontend | Polling يستمر، والواجهة تعرض حالة "غير متصل" ثم تستأنف |
| فشل تخزين الدليل | الواقعة تُحفظ، ويُسجَّل فشل الدليل ليُعاد توليده لاحقاً |
| مخرجات Model غير صالحة | استبعاد الكشف الشاذ بدل تخزينه في قاعدة البيانات |

## 24.3 الـ Logging المنظّم مع سياق

**Structured Logging (سجلات منظّمة):** كتابة السجلات بصيغة حقول ثابتة (وقت، مستوى، رسالة، job_id، video_id) بدل نصوص حرّة. الفائدة أن البحث عن كل سطور job واحد يصبح استعلاماً بسيطاً على job_id. نمنع تماماً تسجيل كلمات المرور أو التوكنات أو مفاتيح S3_SECRET_ACCESS_KEY، ونكتفي بذكر المعرّفات.

## 24.4 الأخطاء القابلة للاسترجاع وغير القابلة

**Recoverable error (خطأ قابل للاسترجاع):** خطأ مؤقّت يمكن أن ينجح لو أُعيد المحاولة بعد فترة، مثل انقطاع اتصال قصير بقاعدة البيانات أو مشكلة شبكة. و**Non-recoverable error (غير قابل للاسترجاع):** خطأ ثابت لا يتغيّر بإعادة المحاولة، مثل فيديو تالف أو مسار checkpoint خاطئ. القاعدة: نعيد المحاولة على الأول فقط، وحدّه الأعلى JOB_MAX_RETRIES، مع تأخير متزايد بين المحاولات، وبعد استنفاد المحاولات تُثبَّت الحالة failed.

## 24.5 منع تكرار الوقائع بعد إعادة المحاولة

هذا أخطر تفصيل. لو أُعيد تشغيل job بعد فشله في منتصف الفيديو، فقد يُنشئ وقائع جديدة لنفس الحدث الذي أُنشئت له وقائع في المحاولة الأولى، فيتضاعف السجل ويصبح غير موثوق. لذلك نتبع قاعدتين معاً: الأولى نحذف أو نتجاهل وقائع المحاولة الفاشلة لنفس الـ job قبل إعادة التشغيل، والثانية نحترم INCIDENT_COOLDOWN_SECONDS داخل نفس الفيديو بحيث لا تُنشأ واقعتان لنفس النوع داخل فترة التهدئة. بهذا تكون إعادة المحاولة آمنة (**idempotent**) بالنسبة للوقائع.

## 24.6 مراقبة صحة الخدمة

نراقب ثلاث نقاط: GET /api/v1/health للتأكد من حياة الـ API واتصاله بقاعدة البيانات، وحالة الـ jobs لرصد أي job عالق في running، ونسبة فشل الـ Jobs كمؤشر عام على صحة خط المعالجة.

### 24.7 ملف الـ Logging

هذا الملف يضبط سجلاً موحّداً لكل التطبيق، ويضيف job_id و video_id تلقائياً عند توفّرهما، ويحجب أي حقل حسّاس من الظهور.

backend/app/core/logging.py

```python
import logging
import sys

# Never let these values reach the log stream.
SENSITIVE_KEYS = {"password", "token", "authorization", "secret_key", "s3_secret_access_key"}


def configure_logging(level: str = "INFO") -> None:
    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(
        logging.Formatter(
            "%(asctime)s %(levelname)s %(name)s "
            "job_id=%(job_id)s video_id=%(video_id)s %(message)s"
        )
    )
    root = logging.getLogger()
    root.handlers.clear()
    root.addHandler(handler)
    root.setLevel(level.upper())


def scrub(extra: dict) -> dict:
    """Drop sensitive fields before they are written to the log."""
    return {
        key: value
        for key, value in extra.items()
        if key.lower() not in SENSITIVE_KEYS
    }


def job_logger(name: str, job_id: str | None = None, video_id: str | None = None):
    """Return a logger that always carries the job context."""
    logger = logging.getLogger(name)
    return logging.LoggerAdapter(
        logger, {"job_id": job_id or "-", "video_id": video_id or "-"}
    )
```

### 24.8 ملف الأخطاء المخصّصة

نعرّف أنواع الأخطاء بأنفسنا حتى نفرّق بين ما يُعاد تشغيله وما لا يُعاد، ولنعطي المستخدم رسالة واضحة بدل تفاصيل داخلية.

backend/app/core/errors.py

```python
class SafeVisionError(Exception):
    """Base class for all application errors."""

    code = "safevision_error"
    recoverable = False

    def __init__(self, message: str, detail: str | None = None):
        super().__init__(message)
        self.message = message
        self.detail = detail


class InvalidVideoError(SafeVisionError):
    code = "invalid_video"
    recoverable = False  # a corrupt file will never succeed on retry


class ModelCheckpointMissing(SafeVisionError):
    code = "checkpoint_missing"
    recoverable = False  # fix the path, then retry manually


class InsufficientGpuMemory(SafeVisionError):
    code = "gpu_oom"
    recoverable = True  # retry on CPU or after freeing memory


class TransientDatabaseError(SafeVisionError):
    code = "db_unavailable"
    recoverable = True  # connection blips succeed on retry
```

```
class EvidenceStorageError(SafeVisionError):
    code = "evidence_storage_failed"
    recoverable = True  # the incident is kept and evidence is retried


def should_retry(error: Exception, attempt: int, max_retries: int) → bool:
    if not isinstance(error, SafeVisionError):
        return attempt < max_retries
    return error.recoverable and attempt < max_retries
```

## 24.9 جدول استكشاف الأخطاء

هذا الجدول مرجع سريع وقت العرض. القاعدة أن نبدأ بالسبب الأكثر شيوعاً، ثم نتبعه بخطوات التشخيص بالترتيب.

**جدول 24-2: الأعراض والأسباب المحتملة والحل**

| العَرَض (Symptom) | السبب المحتمل (Cause) | الحل (Solution) |
|---|---|---|
| job عالق في `running` بلا تقدّم | الـ worker متوقف أو انهار | فحص سجلات الـ worker وإعادة تشغيله |
| job يفشل فوراً | مسار checkpoint غير صحيح | التحقق من `PPE_MODEL_PATH` داخل الحاوية |
| `connection refused` | استخدام `localhost` بدل اسم الخدمة | تصحيح `DATABASE_URL` لاستخدام db |
| واقعة بلا صورة دليل | فشل كتابة الدليل على القرص | التحقق من صلاحيات الـ volume وإعادة التوليد |
| خطأ CORS في المتصفح | عنوان الواجهة غير مدرج | إضافة العنوان إلى `CORS_ORIGINS` |
| واقعة مكرّرة لنفس الحدث | إعادة محاولة دون تنظيف | حذف وقائع المحاولة الفاشلة واحترام فترة التهدئة |
| خطأ `gpu_oom` | الذاكرة غير كافية | العودة إلى `MODEL_DEVICE=cpu` |

### خلاصة الفصل 24

#### ما تعلّمناه

غطّينا تسعة سيناريوهات فشل بسلوك واضح، وفرّقنا بين الأخطاء القابلة للاسترجاع وغير القابلة، وربطنا السجلات بـ job_id، ومنعنا كشف الأسرار، وضبطنا إعادة محاولة آمنة لا تكرّر الوقائع.

#### ما يجب تنفيذه

- [ ] كتابة backend/app/core/logging.py وerrors.py.
- [ ] ربط كل خطأ في الـ worker بـ error_message وحالة failed.
- [ ] تطبيق حذف وقائع المحاولة الفاشلة قبل إعادة التشغيل.
- [ ] التحقق من عدم تسجيل أي حقل حسّاس.

#### المخرجات المتوقعة

- سجلات منظّمة قابلة للبحث بـ job_id.
- رسائل خطأ واضحة تظهر للمستخدم بدل تفاصيل داخلية.
- جدول استكشاف أخطاء معتمد من الفريق.

#### كيف نتحقق من الإنجاز

- تشغيل job بمسار model خاطئ ومشاهدة الفشل الواضح.
- إعادة تشغيل job فاشل والتأكد من عدم تكرار الوقائع.
- البحث عن أي كلمة مرور في السجلات وعدم العثور عليها.

#### أخطاء شائعة

- تسجيل كائن الطلب كاملاً بما فيه التوكن.
- إعادة المحاولة على خطأ ثابت بلا فائدة.
- إعادة المحاولة دون حذف وقائع المحاولة السابقة.
- ترك job عالقاً في running دون مهلة زمنية.

CH25 · TEAM DIVISION

# 25 توزيع العمل على خمسة أعضاء

من يفعل ماذا، وما يعتمد على من، وكيف نمنع أن يصبح عضو واحد عنق الزجاجة

نجاح المشروع لا يعتمد على مهارة فرد واحد، بل على وضوح الحدود بين الأعضاء. هذا الفصل يثبّت الأدوار الخمسة كما في SPEC.md (من M1 إلى M5)، ويحدّد لكل دور مخرجاته ومعايير قبوله واعتمادياته، ثم يشرح كيف يمنع الفريق أن يتحوّل عضو واحد إلى عنق زجاجة يعطّل الجميع.

## 25.1 الأدوار الخمسة

كل دور مالك (owner) لوحدة واضحة، أي أنه المسؤول الأول عن صحتها، وليس الشخص الوحيد الذي يكتب فيها. الأدوار الخمسة هي:

### M1 – AI Models & Inference

- تدريب/تجهيز الـ checkpoints ودمجها في app/ai/.
- مخرج الكشف الموحّد Detection ونماذج PPE fire/smoke.
- التتبّع عبر ByteTrack وتوثيق أثر الأخطاء في الكشف.

### M2 – Video Processing & Safety Logic

- قراءة الفيديو بالـ OpenCV والتقطيع والتسلسل الزمني.
- قواعد السلامة التي تحوّل الكشوفات إلى incidents.
- مناطق الخطر، عتبات الثقة، ومنع التكرار cooldown.

### M3 – Backend & Database

- تطبيق FastAPI والراوترات وطبقات Repository/Service.
- مخطط PostgreSQL وترحيلات Alembic.
- مالك عقد الـ API (/api/v1) وهياكل الاستجابة.

### M4 – Frontend

- تطبيق React + TypeScript + Vite.
- صفحات الرفع والتقدّم والوقائع والمراجعة والتحليلات.
- الاتصال بالـ API عبر Axios ومتابعة الـ jobs بالـ polling.

### M5 – Integration, QA & Deployment

- دمج المكوّنات مبكراً وإدارة الفروع والمراجعات.
- الاختبارات Docker Composeg وتعليمات التشغيل.
- سجل المخاطر وقائمة التحقق النهائية للعرض.

### المسؤولية الجماعية

- كل عضو يراجع كود غيره قبل الدمج.
- الجميع يحترم عقد الـ API ولا يغيّره منفرداً.
- الجميع يكتب توثيقاً في docs/ لِما أنجزه.

أسماء الأعضاء تُكتب لاحقاً بالشكل: <TEAM_MEMBER_5_NAME> ... <TEAM_MEMBER_1_NAME>، واسم المشرف <SUPERVISOR_NAME>، والجامعة <UNIVERSITY_NAME>. لا تُخترع أي أسماء في هذا الدليل.

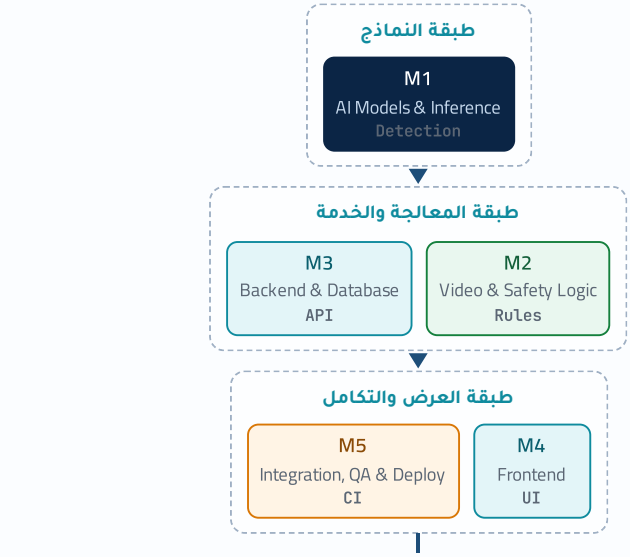
## 25.2 جدول المسؤوليات والمخرجات والاعتماديات ومعايير القبول

**جدول 25-1: تفصيل الدور الخمسة ومعايير القبول**

| العضو | المسؤولية | المخرجات | الاعتماديات | معايير القبول |
|---|---|---|---|---|
| M1 | النماذج والاستدلال | وحدة app/ai/، ملفات الأوزان، سكربت تقييم | مسارات الـ checkpoints والجهاز MODEL_DEVICE | كشف يعمل على فيديو العرض وينتج المخرج الموحّد بلا أخطاء |
| M2 | معالجة الفيديو وقواعد السلامة | وحدة المعالجة، محرّك القواعد، ملف DANGER_ZONE_CONFIG | M1 للمخرج الموحّد، M3 لحفظ الوقائع | واقعة واحدة على الأقل لكل نوع تنشأ من فيديو حقيقي مع بداية ونهاية |
| M3 | الـ Backend وقاعدة البيانات | راوترات api/v1/، الجداول، الترحيلات | عقد الـ API المعتمد بين الفريق | كل مسارات الـ API تُجيب بالهياكل المتفق عليها في SPEC.md |
| M4 | الواجهة الأمامية | صفحات React، عميل Axios، الرسم البياني Recharts | M3 لعقد الـ API، M1/M2 للبيانات الحقيقية | دورة كاملة من الرفع حتى التقرير دون تعديل يدوي في قاعدة البيانات |
| M5 | التكامل والاختبارات والنشر | docker-compose.yml، سكربتات الاختبار، docs/RUNBOOK.md | جميع الأعضاء لتسليم وحداتهم القابلة للتشغيل | تشغيل النظام كاملاً بأمر واحد على جهاز نظيف واجتياز قائمة التحقق |

## 25.3 مخطط الاعتماديات بين الأعضاء

الاعتماد ليس عيباً بل واقع، لكن الخطر أن يكون الاعتماد في اتجاه واحد ومتأخراً. لذلك نثبّت عقد الـ API مبكراً ليصبح "جداراً" يفصل M4 عن الداخل، ويعمل كل عضو بالتوازي.




M1 → M2 → M3 → M4 (□□□□□□□□□)□ □ M5 □□□□□□ □□□□□□□ □□□ □□□□□□ □□□□□ □□□□□

**شكل 25-1:** مخطط الاعتماديات بين الأعضاء الخمسة واتجاه البيانات

## 25.4 العمل على نفس عقد الـ API ونفس هياكل البيانات

**API contract (عقد الواجهة):** اتفاق مكتوب على المسارات والحقول وأنواع البيانات، لا يتغيّر إلا باتفاق الفريق. هذا الاتفاق هو أهم أداة لمنع التعطّل، لأن M4 يستطيع بناء الواجهة على بيانات وهمية (`mock`) قبل أن يجهز الـ Backend فعلاً. القواعد الملزمة:

- مصدر الحقيقة الوحيد هو SPEC.md: نفس الأسماء (VideoOut، JobOut، IncidentOut) ونفس قيم التعدادات.
- أي تغيير في الحقول يُكتب في docs/API_CHANGELOG.md ويُعلَن في الاجتماع الأسبوعي.
- الـ Backend ينشر مخططه عبر /docs (Swagger) ليراه M4 مباشرة.
- لا يحفظ أي عضو نسخة خاصة من هياكل البيانات على جهازه؛ الملف المشترك هو المرجع.

> **مثال عملي**
>
> بما أن الحقول مثبّتة، يستطيع M4 بناء صفحة قائمة الوقائع ببيانات وهمية مطابقة لـ IncidentOut، ثم يبدّل العنوان من ملف mock إلى عنوان الـ API الحقيقي دون تغيير أي كود في العرض. هذا هو معنى "العمل بالتوازي".

## 25.5 الاجتماع الأسبوعي وقواعد التواصل

اجتماع قصير (15–20 دقيقة) في نفس اليوم كل أسبوع، وبجدول ثابت: ما أُنجز، ما تعطّل، ما يحتاجه كل عضو من غيره، ثم تحديث جدول المخاطر. أما التواصل اليومي فبأداة واحدة فقط، وبقواعد واضحة.

**جدول 25-2: قواعد التواصل**

| القناة | الاستخدام | القاعدة |
|---|---|---|
| محادثة الفريق | أسئلة سريعة وتعطّل فوري | لا قرارات تقنية مهمة إلا بعد تدوينها |
| الاجتماع الأسبوعي | حالة كل عضو وتحديث الخطة | كل عضو يأتي بثلاثة أسطر جاهزة |
| Pull Request | مراجعة الكود قبل الدمج | مراجعة عضو واحد على الأقل إلزامية |
| docs/ | التوثيق والقرارات | أي قرار يُكتب في ملف وليس في المحادثة فقط |

## 25.6 منع أن يصبح عضو واحد عنق الزجاجة

الخطأ الشائع هو ترك التكامل كله لـ M5 في الأسبوع الأخير. الصحيح أن M5 **ليس مسؤولاً وحده عن إصلاح مشاكل الدمج في النهاية**؛ بل كل عضو مسؤول أن يكون مخرجه قابلاً للدمج من اليوم الأول. نطبّق ثلاثة قواعد:

- **التكامل يبدأ مبكراً:** من الأسبوع الثاني يشغّل M5 تكاملاً أسبوعياً بين الـ Backend والـ Frontend حتى لو كانت البيانات وهمية.
- **تعريف "مُنجز" موحّد:** لا تُعدّ الوحدة منجزة إلا إذا عملت مع بقية النظام واجتازت معيار قبولها.
- **مراجعة متقاطعة:** كل عضو يراجع كود عضو آخر، فلا تعتمد الجودة على شخص واحد.

**علامة خطر**

إذا قال عضو في الأسبوع الثامن: "انتظر حتى أنتهي من وحدتي ثم ندمج"، فهذا إنذار مبكر. الحل أن يسلّم نسخة مبدئية تعمل على فيديو قصير ولو بنتائج غير مكتملة، حتى يبدأ الدمج والتكامل فوراً.

## خلاصة الفصل 25

### ما تعلّمناه

قسّمنا العمل إلى خمسة أدوار واضحة (M1 حتى M5)، وحدّدنا لكل دور مسؤوليته ومخرجاته واعتمادياته ومعيار قبوله، وأثبتنا أن عقد الـ API المشترك هو ما يسمح بالعمل بالتوازي، وأن التكامل مسؤولية الجميع لا مسؤولية M5 وحده.

### ما يجب تنفيذه

- تعبئة أسماء الأعضاء في docs/TEAM.md بدل الـ Placeholders.
- تثبيت عقد الـ API من SPEC.md والموافقة عليه كتابةً.
- تحديد يوم ووقت الاجتماع الأسبوعي ومدته.
- إنشاء قالب Pull Request فيه معيار القبول المطلوب.

### المخرجات المتوقعة

- جدول المسؤوليات (جدول 25-1) مكتمل ومعتمد.
- ملف docs/API_CHANGELOG.md جاهز لأول تغيير.
- خطة تكامل أسبوعية من الأسبوع الثاني.

### كيف نتحقق من الإنجاز

- كل عضو يشرح مسؤوليته ومعيار قبوله في دقيقة.
- لا توجد وحدة تُوصف بأنها "منجزة" وهي غير قابلة للتشغيل مع بقية النظام.
- يوجد تاريخ واضح لأول تكامل حقيقي بين الـ Frontend والـ Backend.

### أخطاء شائعة

- ترك الدمج لـ M5 في الأسبوع الأخير.
- تغيير أسماء الحقول منفرداً دون إعلان.
- اعتبار الاجتماع الأسبوعي تقريراً شفهياً بلا تدوين.
- كتابة أسماء حقيقية أو تقديرات درجات داخل الدليل.

CH26 · TIMELINE

# 26 الجدول الزمني والمراحل

خطة 8–10 أسابيع قابلة للتعديل

الخطة أداة لتوزيع الوقت وكشف التعطّل. يقسّم الفصل المشروع إلى عشر مراحل بمدد ومهام واعتماديات ومعايير قبول ومخاطر؛ وترتيب الاعتماديات ثابت.

## 26.1 نظرة عامة

خطة من **عشرة أسابيع** قابلة للضغط إلى ثمانية: الأول للتثبيت، والثاني إلى الثامن للتنفيذ المتوازي، والأخيران للتجميد والعرض. **لا نؤجّل التكامل إلى النهاية**.

**جدول 26-1: توزيع الأسابيع على المحاور**

| الأسابيع | التركيز الأساسي | المخرج الرئيسي |
|---|---|---|
| 1 | تثبيت المتطلبات وإعداد المستودع | مستودع يعمل وعقد API متفق عليه |
| 2–4 | قاعدة البيانات، الـ Backend، دمج الـ inference | رفع فيديو ومعالجة أولية |
| 5–7 | خط الوقائع والدليل، لوحة التحكم، التقارير | دورة كاملة من الرفع إلى التقرير |
| 8–9 | التكامل والاختبارات Dockerg | نظام يعمل بأمر واحد ومُختبر |
| 10 | تجميد وتدريب على العرض | عرض توضيحي جاهز ومُختبر |

## 26.2 المراحل العشر

**جدول 26-2: المراحل العشر بمهامها ومعايير قبولها ومخاطرها**

| # | المرحلة | المدة | المهام | الاعتماديات | المخرجات | معيار القبول | المخاطر |
|---|---|---|---|---|---|---|---|
| 1 | تثبيت المتطلبات وإعداد المستودع | أسبوع 1 | جدول النطاق، عقد الـ API، `.gitignore`، `.env.example` | لا شيء | `docs/SCOPE.md` `docs/API_CHANGELOG.mdg` | كل عضو يشغّل المستودع | توسّع النطاق |
| 2 | بيئة التطوير وعقد الـ API | أسبوع 1–2 | venv، Node، PostgreSQL، Swagger، الهياكل | المرحلة 1 | Frontendg Backend يعملان ويتحدثان | طلب `GET /api/v1/health` ينجح من الواجهة | اختلاف الإصدارات |
| 3 | قاعدة البيانات وأساس الـ Backend | أسبوع 2–3 | الجداول، ترحيلات `Alembic`، الراوترات، الـ Repository | المرحلة 2 | قاعدة بيانات مهاجرة ومسارات رفع | رفع فيديو وحفظ بياناته | تغيّر المخطط دون ترحيل |
| 4 | دمج الـ inference | أسبوع 3–4 | تحميل الـ checkpoints، المخرج الموحّد، التتبّع | المرحلة 1 للنطاق | وحدة `app/ai/` تعمل على فيديو | مخرج موحّد صالح لكل إطار | نقص بيانات/أوزان |
| 5 | معالجة الفيديو والـ jobs | أسبوع 4–5 | الـ worker، طابور المهام، التقدّم، `polling` | المراحل 3 و4 | job من `queued` إلى `completed` | تحديث التقدّم | تعليق الـ worker |
| 6 | خط الوقائع والدليل | أسبوع 5–6 | محرّك القواعد، إنشاء الـ incidents، حفظ الصور | المرحلة 5 | جدولا `incidents` و`evidence` | واقعة لكل نوع من فيديو حقيقي | تكرار الوقائع |
| 7 | لوحة التحكم | أسبوع 6–7 | صفحات الرفع والتقدّم والوقائع والمراجعة والتحليلات | المرحلة 3 (العقد) | واجهة React متصلة بالـ API | دورة كاملة من الرفع إلى المراجعة بلا تدخّل يدوي | ربط خاطئ للحقول |
| 8 | التقارير والمراجعة | أسبوع 7–8 | التحليلات، التصدير CSV، إجراءات المراجعة | المرحلتان 6 و7 | `/api/v1/reports/export` يعمل | تقرير من سجلات حقيقية | حساب مؤشرات من بيانات وهمية |
| 9 | التكامل والاختبارات | أسبوع 8–9 | اختبارات الوحدة والتكامل، إصلاح الأخطاء، إدارة الفروع | كل المراحل السابقة | نسخة مستقرة واختبار | اجتياز سيناريو العرض بلا أخطاء حاجبة | أخطاء دمج متأخرة |
| 10 | Docker والعرض التوضيحي | أسبوع 9–10 | التغليف `docker-compose`، فيديو العرض، التدريب | المرحلة 9 | تعليمات تشغيل + عرض مُختبر | النظام يعمل بأمر واحد على جهاز نظيف | فشل الشبكة/GPU |

## 26.3 ما يمكن تنفيذه بالتوازي

التوازي مشروط بتثبيت عقد الـ API:

**مسار A (بعد الأسبوع 1)**

- M3: الـ Backend والجداول.
- M4: الواجهة على بيانات وهمية مطابقة للعقد.
- M1: تجهيز النماذج بمعزل.

**مسار B (بعد الأسبوع 3)**

- M2: ربط القواعد بمخرج M1.
- M5: الاختبار Dockerg مبكراً.
- M4: تبديل mock بعنوان الـ API.

**ما لا يمكن توازيه**

لا يمكن بناء خط الوقائع قبل مخرج الكشف، ولا التحليلات قبل وقائع حقيقية؛ هذه المهام متسلسلة وما عداها متوازٍ.

## 26.4 المخطط الزمني المبسّط




**شكل 26-1:** مخطط Gantt مبسّط: المربّعات الملوّنة أسابيع نشطة والمربّعات المتقطّعة أسابيع خاملة

## 26.5 قائمة تحقق المراحل

- [ ] عقد الـ API مثبّت و.env.example بلا أسرار.
- [ ] الـ Frontend يقرأ /api/v1/health.
- [ ] ترحيل Alembic ينشئ كل الجداول.
- [ ] المخرج الموحّد صالح على فيديو قصير.
- [ ] job يصل إلى completed بتقدّم حقيقي.
- [ ] واقعة لكل نوع مع صورة دليل.
- [ ] دورة كاملة من الرفع إلى المراجعة.
- [ ] تصدير CSV من سجلات حقيقية.
- [ ] سيناريو العرض يمر بلا أخطاء حاجبة.
- [ ] تشغيل بأمر واحد على جهاز نظيف.

**خلاصة الفصل 26**

**ما تعلّمناه**

قسّمنا المشروع إلى عشر مراحل بترتيب اعتماديات واضح، وحدّدنا لكل مرحلة مدتها ومهامها ومخرجاتها ومعيار قبولها ومخاطرها،

وعرضنا مسارات العمل المتوازية والمخطط الزمني المبسّط.

### ما يجب تنفيذه

- تثبيت تواريخ فعلية للمراحل في docs/PLAN.md.
- تحديد مالك كل مرحلة من M1 إلى M5.
- تسجيل المخاطر المتوقعة وخطة تخفيفها.

### المخرجات المتوقعة

- جدول مراحل معتمد بمعايير قبول قابلة للقياس.
- مخطط زمني مرئي محدَّث أسبوعياً.
- سجل مخاطر حيّ يُراجَع في الاجتماع الأسبوعي.

### كيف نتحقق من الإنجاز

- كل معيار قبول يمكن إثباته بعرض عملي لا بوصف شفهي.
- لا مرحلة تبدأ قبل إغلاق اعتمادياتها.
- يوجد تكامل واحد على الأقل كل أسبوع من الأسبوع الثاني.

### أخطاء شائعة

- البدء في الواجهة قبل تثبيت عقد الـ API.
- تأجيل Docker والاختبارات إلى الأسبوع الأخير.
- إضافة مراحل غير مذكورة في النطاق أثناء التنفيذ.
- تفسير التأخير بتفاؤل بدل تعديل الخطة.

# 27 تعريف الـ MVP ومعايير القبول

أصغر نظام مكتمل يمكن عرضه بنجاح

**MVP (Minimum Viable Product):** أصغر نسخة مكتملة من النظام تعمل من أول رفع الفيديو حتى تصدير التقرير. الـ MVP ليس "نسخة ناقصة"، بل نسخة **مكتملة الدورة** وإن كانت محدودة القدرات. هذا الفصل يثبّت قائمة الـ MVP، ويذكر صراحةً ما لا يدخل فيه، ويضع تعريف "منجز" لكل وحدة ومصفوفة قبول بحالة تُملأ بعد التجربة.

## 27.1 قائمة الـ MVP

هذه الميزات هي الحد الأدنى المتفق عليه، وأي ميزة خارجها تُصنَّف `Future`. الهدف أن يعمل النظام كدورة كاملة، لا أن يعمل كل مكوّن بمفرده.

**جدول 27-1: ميزات الـ MVP**

| # | الميزة | الوصف المختصر |
|---|---|---|
| 1 | واجهة أمامية تعمل | React + TypeScript + Vite مع صفحات أساسية مترابطة |
| 2 | Backend يعمل | FastAPI مع طبقات Route/Schema/Model/Repository/Service |
| 3 | PostgreSQL | قاعدة بيانات مهاجَرة عبر `Alembic` |
| 4 | رفع وتحقق | رفع فيديو مع التحقق من النوع والحجم وتخزين الملف |
| 5 | خط الـ jobs | job غير متزامن مع تقدّم وحالات من `queued` إلى `completed` |
| 6 | دمج الـ checkpoints | تحميل الأوزان وتشغيل الاستدلال عبر واجهة موحّدة |
| 7 | كشف وقواعد | كشف PPE والحرائق والدخان وتحويله إلى وقائع بقواعد واضحة |
| 8 | توليد وقائع | سجل `incidents` بنوع وخطورة وحالة |
| 9 | تخزين الدليل | حفظ صور الدليل وربطها بالواقعة عبر `evidence` |
| 10 | سجل وفلترة | قائمة وقائع مع ترقيم صفحات وفلترة وترتيب |
| 11 | إجراءات مراجعة | تأكيد/رفض/إغلاق مع إضافة ملاحظة في `incident_reviews` |
| 12 | تحليلات من سجلات حقيقية | مؤشرات ولوحة `/api/v1/analytics/overview` من بيانات فعلية |
| 13 | تصدير CSV | تقرير قابل للتنزيل عبر `/api/v1/reports/export` |
| 14 | أمان وتحقق أساسي | تحقق المدخلات وضبط CORS وخيار `AUTH_ENABLED` |
| 15 | تعليمات تشغيل محلي | خطوات واضحة و`docker-compose`ق للتشغيل |
| 16 | فيديو عرض مُختبر | مقطع قصير يعمل عليه السيناريو كاملاً |

## 27.2 ما لا يدخل الـ MVP

نذكر هذه العناصر صراحةً كـ Pending حتى لا تُفهَم خطأً كإنجاز. بعضها مُخطَّط وبعضها مستقبلي تماماً.

**جدول 27-2: عناصر مؤجّلة خارج الـ MVP**

| العنصر | السبب | التصنيف |
|---|---|---|
| بث مباشر RTSP | يحتاج معالجة لحظية وبنية مختلفة | `Future` |
| إشعارات بريد/رسائل | لا يلزم لإكمال الدورة | `Future` |
| معايرة الكاميرا للمسافات الحقيقية | المناطق تقريبية في الـ MVP | `Future` |
| تسجيل دخول كامل وJWT | خارج الـ MVP، يعمل خلف `AUTH_ENABLED` | `[illegible]` |
| تقارير PDF | التصدير الأساسي CSV | `Future` |
| `Kubernetes / Kafka / Redis / Celery` | تعقيد غير مبرَّر لفريق طلابي | `Future` |
| تدريب نموذج جديد من الصفر | نستخدم أوزاناً متاحة ومتفقاً عليها | `Future` |
| لوحة إدارة مستخدمين كاملة | دور واحد افتراضي في وضع العرض | `[illegible]` |

## 27.3 تعريف "منجز" لكل وحدة

**Definition of Done (تعريف الإنجاز):** الشروط التي إن تحققت تُعدّ الوحدة منتهية. لا نقبل وصفاً مثل "الكود جاهز"، بل دليلاً قابلاً للتشغيل.

**جدول 27-3: تعريف الإنجاز لكل وحدة**

| الوحدة | تعريف الإنجاز |
|---|---|
| Backend | كل مسارات `/api/v1/` المطلوبة تُجيب بالهياكل المتفق عليها وترد بأخطاء واضحة عند الفشل |
| Database | ترحيل نظيف ينشئ كل الجداول والعلاقات والقيود على قاعدة فارغة |
| AI | تحميل الأوزان وتشغيل الاستدلال على فيديو وينتج المخرج الموحّد دون أخطاء حاجبة |
| Pipeline | job يكتمل من الرفع إلى توليد الوقائع مع تحديث التقدّم |
| Frontend | دورة كاملة من الرفع إلى المراجعة من المتصفح دون تدخّل يدوي في قاعدة البيانات |
| Reports | تصدير CSV من سجلات حقيقية وحساب المؤشرات من نفس البيانات |
| Deployment | تشغيل النظام بأمر واحد على جهاز نظيف مع تعليمات مكتوبة |

## 27.4 مصفوفة القبول

عمود الحالة يُترك فارغاً ويُملأ بعد التجربة الفعلية، ولا نكتب أي نتيجة قبل قياسها.

**جدول 27-4: مصفوفة القبول — الحالة (يُملأ بعد التجربة)**

| الميزة | معيار القبول | كيف نقيسه | الحالة |
|---|---|---|---|
| رفع فيديو | ملف صالح يُقبل وملف غير صالح يُرفض | رفع ملفين (صالح/تالف) ومراقبة الرد | (يُملأ بعد التجربة) |
| خط الـ jobs | job يصل إلى `completed` مع تقدّم متزايد | مراقبة `GET /api/v1/jobs/{job_id}` | (يُملأ بعد التجربة) |
| الكشف | مخرج موحّد صالح لكل إطار معالج | تشغيل على فيديو قصير ومراجعة السجل | (يُملأ بعد التجربة) |
| توليد الوقائع | واقعة واحدة على الأقل لكل نوع | فلترة قائمة الوقائع حسب النوع | (يُملأ بعد التجربة) |
| الدليل | صورة دليل مرتبطة بكل واقعة وتُعرض | فتح `/api/v1/evidence/{evidence_id}/file` | (يُملأ بعد التجربة) |
| المراجعة | تغيير الحالة مع تسجيل صف في `incident_reviews` | `PATCH /api/v1/incidents/{incident_id}` ثم فحص السجل | (يُملأ بعد التجربة) |
| التحليلات | المؤشرات مطابقة لعدد السجلات الفعلي | مقارنة `overview` بعدد الصفوف في قاعدة البيانات | (يُملأ بعد التجربة) |
| تصدير CSV | ملف ينزل بأعمدة صحيحة وقيم مطابقة | تنزيل الملف وفتحه ومقارنة القيم | (يُملأ بعد التجربة) |
| التشغيل الموحّد | النظام يعمل بأمر واحد على جهاز نظيف | تشغيل `docker compose up` ومتابعة الخدمات | (يُملأ بعد التجربة) |

> **لا أرقام بلا قياس**
>
> لا نكتب أي دقة أو سرعة أو معدّل نجاح في هذه المصفوفة. الخلية تُملأ فقط بعد تنفيذ القياس وتسجيل الظروف (الفيديو، الجهاز، الإعدادات).

### خلاصة الفصل 27

**ما تعلّمناه**

عرّفنا الـ MVP كدورة كاملة لا كنسخة ناقصة، وثبّتنا قائمة الميزات الست عشرة، وأعلنّا العناصر المؤجّلة صراحةً، ووضعنا تعريف "منجز" لكل وحدة ومصفوفة قبول بحالة تُملأ بعد التجربة.

**ما يجب تنفيذه**

- تثبيت قائمة الـ MVP في docs/MVP.md.
- ربط كل ميزة بمالك من M1 إلى M5.
- تعبئة عمود الحالة بعد كل تجربة مع تاريخها.

### المخرجات المتوقعة

- مصفوفة قبول معتمدة وقابلة للتحديث.
- قائمة Pending واضحة تُعرض في المناقشة.
- تعريف إنجاز موحّد يستخدم في كل Pull Request.

### كيف نتحقق من الإنجاز

- كل خلية "الحالة" لها دليل تشغيل لا وصف شفهي.
- لا ميزة مصنّفة `MVP` بلا معيار قبول مكتوب.
- لا عنصر Pending يُعرض كأنه منفَّذ.

### أخطاء شائعة

- توسيع الـ MVP تدريجياً حتى يصبح غير قابل للتسليم.
- كتابة نتائج مبدئية في مصفوفة القبول قبل قياسها.
- اعتبار أن عمل الـ Model وحده يعني اكتمال الـ MVP.
- إغفال تعريف "منجز" فيعتمد القبول على الرأي الشخصي.

CH28 · FINAL DEMO

# 28 العرض النهائي لمشروع التخرّج

خطة عرض من 8 إلى 12 دقيقة بالترتيب

العرض النهائي ليس جولة عشوائية في الشاشات، بل قصة مرتّبة: مشكلة، حل، ثم إثبات عملي. هذا الفصل يثبّت خطوات العرض الثلاث عشرة بالترتيب، ويوزّع الوقت، ويضع خطة بديلة عند فشل أي مكوّن، ويميّز صراحةً بين ما يُعرض حياً وما يُعرض مسجَّلاً، ويجهّز الفريق لأسئلة اللجنة.

## 28.1 تمييز أساسي: العرض الحي مقابل المواد المسجَّلة

هذا التمييز شرط للنزاهة أمام اللجنة. **العرض الحي (live):** نشغّل النظام أمام اللجنة ونرفع الفيديو وننتظر النتيجة فعلياً. **المادة المسجَّلة مسبقاً (prerecorded):** فيديو جاهز نُعدّه للاستخدام فقط عند فشل البيئة، ولا نقدّمه كأنه حي. نُعلن قبل العرض ماذا سيُعرض حياً وماذا سيكون تسجيلاً احتياطياً.

**جدول 28-1: ما يُعرض حياً وما يُسجَّل مسبقاً**

| العنصر | حي (Live) | مسجَّل مسبقاً (Prerecorded) |
|---|---|---|
| رفع فيديو قصير وبدء المعالجة | نعم | احتياطي فقط عند الفشل |
| متابعة التقدّم وفتح الواقعة | نعم | احتياطي فقط |
| نتائج تقييم الـ AI | تُعرض من جدول مُعدّ مسبقاً | لا تُختلق أرقاماً جديدة |
| التحليلات وتصدير التقرير | نعم من سجلات حقيقية | احتياطي فقط |

## 28.2 خطوات العرض الثلاث عشرة بالترتيب

1. المشكلة: ساعات فيديو لا يستطيع أحد مشاهدتها.
2. المعمارية: مخطط المكوّنات من الرفع إلى التقرير.
3. فتح التطبيق وشرح الصفحات الرئيسية بسرعة.
4. رفع فيديو قصير مُعدّ مسبقاً من المتصفح.
5. بدء المعالجة وإنشاء الـ job.
6. عرض التقدّم أثناء تشغيل الـ worker.
7. فتح قائمة الوقائع بعد الاكتمال.
8. فتح واقعة واحدة وعرض تفاصيلها وصورة الدليل.
9. تنفيذ إجراء مراجعة (تأكيد أو رفض) مع ملاحظة.
10. عرض لوحة التحليلات من السجلات الحقيقية.
11. تصدير تقرير CSV وفتحه.
12. عرض نتائج تقييم الـ AI من جدول مُعدّ، مع توضيح ظروف القياس.
13. الحدود والتحسينات المستقبلية وما لم يدخل الـ MVP.

## 28.3 توزيع الوقت

**جدول 28-2: توزيع الوقت على العرض (8–12 دقيقة)**

| الجزء | الخطوات | الزمن المستهدف |
|---|---|---|
| المشكلة والمعمارية | 2–1 | 2 دقيقة |
| العرض الحي | 7–3 | 4 دقائق |
| الوقائع والدليل والمراجعة | 9–8 | 2 دقيقة |
| التحليلات والتقرير | 11–10 | 2 دقيقة |
| النتائج والحدود | 13–12 | 2–1 دقيقة |

الجدول هدف لا قاعدة صارمة؛ الأهم ألا يستهلك شرح المقدمة وقت العرض الحي.

## 28.4 الخطة البديلة

أُجهّز بديلاً لكل مكوّن قد يفشل، ونعلن الانتقال إليه بصراحة دون محاولة إخفائه.

**جدول 28-3: خطة الطوارئ أثناء العرض**

| الفشل المحتمل | الاستجابة الفورية |
|---|---|
| عدم توفر GPU | التبديل إلى `MODEL_DEVICE=cpu` أو عرض الفيديو المسجَّل |
| انقطاع الإنترنت | تشغيل النظام محلياً عبر `docker compose` |
| فشل النشر السحابي | العرض من الجهاز المحلي بنفس الخطوات |
| تعطّل الـ worker | عرض نتائج محفوظة مسبقاً في قاعدة بيانات جاهزة |
| بطء المعالجة | الاعتماد على فيديو قصير مُعدّ لهذا الغرض |

> **قاعدة ذهبية**
>
> جهّز نسخة احتياطية كاملة من البيئة (قاعدة بيانات بها نتائج، وفيديو، وتسجيل للعرض) على نفس الجهاز، واختبر سيناريو العرض مرتين قبل يوم المناقشة.

## 28.5 أسئلة متوقعة وإجابات مقترحة

**جدول 28-4: أسئلة اللجنة وإجابات قصيرة**

| السؤال | إجابة مقترحة قصيرة |
|---|---|
| ما الذي نفّذتموه فعلاً؟ | نعرض دورة كاملة حية، ونشير إلى قائمة الـ MVP ومصفوفة القبول. |
| ما لم تنفّذوه ولماذا؟ | البث المباشر والإشعارات ومعايرة الكاميرا؛ مذكورة صراحةً كـ Pending. |
| كيف قيّستم دقة النموذج؟ | نعرض جدول التقييم وظروف القياس؛ الأرقام تُذكر فقط بعد القياس الفعلي. |
| لماذا لا تستخدمون Microservices؟ | لأن Modular Monolith أنسب لفريق طلابي ويقلّل تعقيد النشر. |
| كيف تمنعون تكرار الوقائع؟ | عبر عتبات مدة `INCIDENT_COOLDOWN_SECONDS` وربطها بـ `track_id`. |
| هل النظام آمن؟ | تحقق مدخلات وضبط CORS وعدم تخزين أسرار؛ `AUTH_ENABLED` خيار جاهز. |
| ما أكبر تحدٍ واجهكم؟ | التكامل بين الـ pipeline والـ jobs وتوحيد هياكل البيانات. |
| ما خطتكم لو أتيح وقت أكثر؟ | البث المباشر، والإشعارات، ومعايرة الكاميرا، وتقارير PDF. |

> **لا مبالغة**
>
> لا نذكر أي رقم لم يُقَس، ولا ندّعي أن أي وحدة "انتهت تماماً". الصدق في حدود الإنجاز يرفع ثقة اللجنة أكثر من أي ادّعاء.

### خلاصة الفصل 28

**ما تعلّمناه**

بنينا سيناريو عرض من ثلاث عشرة خطوة بالترتيب، ووزّعنا الوقت، ووضعنا خطة بديلة لكل فشل محتمل، وميّزنا بوضوح بين العرض الحي والمواد المسجَّلة، وتدرّبنا على أسئلة اللجنة.

**ما يجب تنفيذه**

- [ ] تجهيز فيديو عرض قصير وقاعدة بيانات بنتائج محفوظة.
- [ ] تسجيل نسخة احتياطية من العرض كخطة بديلة.
- [ ] تجربة السيناريو كاملاً مرتين وقياس زمن كل جزء.
- [ ] توزيع أدوار الحديث بين الأعضاء الخمسة.

**المخرجات المتوقعة**

- سيناريو عرض مكتوب في `docs/DEMO_SCRIPT.md`.
- جدول تقييم الـ AI مُعدّ مسبقاً بظروف القياس.
- قائمة أسئلة متوقعة مع إجابات قصيرة.

**كيف نتحقق من الإنجاز**

- العرض يمر من البداية للنهاية في الزمن المستهدف.

- الانتقال للخطة البديلة يعمل خلال أقل من دقيقة.
- لا رقم غير مقيس ولا ادّعاء بإنجاز غير موجود.

**أخطاء شائعة**

- إطالة المقدمة النظرية على حساب العرض الحي.
- تقديم مادة مسجَّلة كأنها حية دون إعلان.
- عدم تجهيز بديل فيتوقف العرض كله عند أول خطأ.
- ذكر أرقام دقة لم تُقَس فعلياً.

CH29 · FINAL CHECKLIST

# 29 قائمة التحقق النهائية واستكشاف الأخطاء

قبل المناقشة، تحقّق من كل شيء

هذا الفصل أداة عمل لا فصل نظري. الجزء الأول قائمة تحقق شاملة تغطي كل مجالات المشروع، والجزء الثاني مرجع لاستكشاف الأخطاء، ولكل خطأ **تسلسل تشخيص مرتّب** لا تخميناً لسبب واحد. القاعدة: نتحقق من الطبقة الأدنى أولاً (البيئة والاتصال) قبل الطبقة الأعلى (الكود والواجهة).

## 29.1 قائمة التحقق الشاملة

**جدول 29-1: قائمة التحقق قبل المناقشة (يُملأ بعد التجربة)**

| المجال | بنود التحقق | الحالة |
|---|---|---|
| تنظيم الكود | لا ملفات مؤقتة، ولا أسرار في Git، وكل وحدة في مكانها حسب الهيكل | (يُملأ بعد التجربة) |
| الـ Backend | كل مسارات `/api/v1` تعمل، والأخطاء تُعاد بهيئة واضحة | (يُملأ بعد التجربة) |
| الـ Frontend | لا أخطاء في الطرفية، وكل صفحة تتعامل مع حالات التحميل والخطأ | (يُملأ بعد التجربة) |
| قاعدة البيانات | ترحيل نظيف على قاعدة فارغة، والعلاقات والقيود سليمة | (يُملأ بعد التجربة) |
| دمج الـ Models | الأوزان تُحمّل من `MODEL_DIR` والمخرج موحّد | (يُملأ بعد التجربة) |
| خط الفيديو | job يكتمل من الرفع إلى الوقائع مع تقدّم حقيقي | (يُملأ بعد التجربة) |
| تخزين الملفات | الفيديوهات والدليل في `/storage/` ومستثناة من Git | (يُملأ بعد التجربة) |
| الأمان | لا مفاتيح في الكود، و`CORS_ORIGINS` مقيّدة، و`APP_DEBUG=false` للإنتاج | (يُملأ بعد التجربة) |
| الاختبارات | سيناريو العرض يمر، والوحدات الحرجة مغطّاة باختبارات | (يُملأ بعد التجربة) |
| النشر | تشغيل بأمر واحد عبر `docker compose` على جهاز نظيف | (يُملأ بعد التجربة) |
| التقارير | تصدير CSV من سجلات حقيقية بأعمدة صحيحة | (يُملأ بعد التجربة) |
| التوثيق | `README` وتعليمات التشغيل و`docs/` محدّثة | (يُملأ بعد التجربة) |
| جاهزية الفريق | كل عضو يعرف دوره وما سيقوله، والخطة البديلة جاهزة | (يُملأ بعد التجربة) |

## 29.2 مرجع استكشاف الأخطاء

لكل فئة أعراضها، ثم تسلسل تشخيص مرتّب من الأبسط إلى الأعمق. ننفّذ الخطوات بالترتيب ونتوقف عند أول سبب يظهر.

**جدول 29-2: فئات الأخطاء وتسلسل تشخيصها**

| الفئة | الأعراض | تسلسل التشخيص (بالترتيب) |
|---|---|---|
| بيئات Python | تعذّر تنفيذ `python` أو تثبيت الحزم | 1) تحقق من الإصدار. 2) تأكد أن الـ venv مُنشَّط. 3) افحص مسار `python` الفعلي. 4) أنشئ venv جديداً وأعد التثبيت. |
| تعارض الإصدارات | خطأ عند تثبيت حزمة أو تعارض تبعيات | 1) اقرأ نص الخطأ كاملاً. 2) اطبع قائمة الحزم المثبّتة. 3) حدّد الحزمة المتعارضة. 4) ثبّت نسخة متوافقة أو اعزل البيئة. |
| FastAPI | الخدمة لا تُقلع أو مسار يعيد 404/500 | 1) اقرأ أثر الخطأ في الطرفية. 2) تأكد من تسجيل الراوتر. 3) افحص مسار الـ endpoint. 4) راجع `/docs/` للتأكد من ظهوره. |
| اتصال PostgreSQL | رفض الاتصال أو خطأ مصادقة | 1) تحقق أن الحاوية/الخدمة تعمل. 2) تأكد من المنفذ. 3) افحص بيانات الدخول في `DATABASE_URL`. 4) اختبر الاتصال بأداة مباشرة. |
| SQLAlchemy | خطأ في استعلام أو جلسة مغلقة | 1) اطبع الاستعلام المولَّد. 2) تأكد من فتح/إغلاق الجلسة. 3) راجع أسماء الأعمدة مقابل المخطط. 4) تحقق من العلاقات والتحميل الكسول. |
| Alembic | ترحيل يفشل أو حالة غير متزامنة | 1) اطبع الحالة الحالية. 2) راجع آخر ترحيل مُطبَّق. 3) تأكد أن نموذج الجدول يطابق الترحيل. 4) أنشئ ترحيلاً جديداً صحيحاً بدل تعديل القديم. |
| CORS | المتصفح يحجب الطلب رغم نجاحه في الطرفية | 1) افتح أدوات المتصفح. 2) تأكد أن الخطأ CORS لا شبكة. 3) افحص `CORS_ORIGINS`. 4) طابق الأصل والمنفذ تماماً وأعد التشغيل. |
| طلبات React | الواجهة لا تعرض بيانات أو تعيد خطأ | 1) افحص التبويب Network. 2) تأكد من `VITE_API_BASE_URL`. 3) راجع شكل الرد مقابل الحقول. 4) تحقق من معالجة الخطأ في الواجهة. |
| رفع الملفات | الرفع يفشل أو يُرفض ملف صالح | 1) تحقق من حجم الملف مقابل `MAX_UPLOAD_MB`. 2) راجع `ALLOWED_VIDEO_EXTENSIONS`. 3) افحص الـ `content_type`. 4) تحقق من صلاحيات مجلد التخزين. |

| الفئة | الأعراض | تسلسل التشخيص (بالترتيب) |
|---|---|---|
| OpenCV | تعذّر فتح الفيديو أو قراءة الإطارات | 1) تأكد من وجود الملف. 2) اطبع عدد الإطارات والـ fps. 3) راجع الـ codec المدعوم. 4) جرّب ملفاً آخر لاستبعاد تلف الملف. |
| الـ checkpoints | تعذّر تحميل الأوزان | 1) تأكد من وجود الملف في MODEL_DIR. 2) افحص PPE_MODEL_PATH و FIRE_SMOKE_MODEL_PATH. 3) تحقق من توافق إصدار مكتبة النماذج. 4) راجع أسماء الفئات مقابل النموذج. |
| ذاكرة GPU | خطأ نفاد الذاكرة أثناء الاستدلال | 1) راقب استهلاك الذاكرة. 2) قلّل حجم الدفعة أو INFERENCE_IMG_SIZE. 3) زد FRAME_SAMPLE_EVERY_N. 4) حوّل إلى MODEL_DEVICE=cpu عند الحاجة. |
| شبكة Docker | الـ Backend لا يصل إلى قاعدة البيانات داخل الحاويات | 1) تأكد أن الخدمات على نفس الشبكة. 2) استخدم اسم الخدمة لا localhost. 3) افحص POSTGRES_HOST. 4) راجع سجلات الحاوية. |
| الـ volumes | البيانات تختفي بعد إعادة التشغيل | 1) تأكد من تعريف الـ volume. 2) افحص مسار التركيب مقابل متغيرات التخزين. 3) تحقق من صلاحيات الكتابة. 4) أعد التشغيل وافحص بقاء الملفات. |
| متغيرات بيئة الإنتاج | النظام يعمل محلياً ويفشل بعد النشر | 1) قارن قيم .env بين البيئتين. 2) تأكد من APP_ENV و APP_DEBUG. 3) راجع CORS_ORIGINS و DATABASE_URL. 4) اقرأ سجلات الخدمة عند الإقلاع. |

### ترتيب التشخيص العام

ابدأ دائماً من الأسفل إلى الأعلى: (1) هل العملية تعمل؟ (2) هل الاتصال سليم؟ (3) هل الإعدادات صحيحة؟ (4) هل البيانات بالشكل المتوقع؟ (5) هل الكود يعالج الحالة؟ لا تنتقل لطبقة أعلى قبل تأكيد التي تحتها.

### سجل الأخطاء

عند أي خطأ، دوّن: الأمر المستخدم، نص الخطأ، الحل، والسبب الجذري. سجل الأخطاء هذا يوفّر ساعات على الفريق ويُظهر نضجاً في المناقشة.

## خلاصة الفصل 29

### ما تعلّمناه

بنينا قائمة تحقق شاملة تغطي ثلاثة عشر مجالاً، ومرجع استكشاف أخطاء يغطي خمس عشرة فئة، ولكل فئة تسلسل تشخيص مرتّب من الأبسط إلى الأعمق.

### ما يجب تنفيذه

- [ ] تعبئة عمود الحالة في جدول 29-1 بعد كل تجربة.
- [ ] تشغيل النظام على جهاز نظيف والتحقق من كل بند.
- [ ] تدوين الأخطاء الفعلية وحلولها في docs/TROUBLESHOOTING.md.

### المخرجات المتوقعة

- قائمة تحقق موقّعة من الفريق قبل المناقشة.
- مرجع استكشاف أخطاء محدَّث بأخطاء حقيقية.
- نسخة احتياطية جاهزة من البيئة والنتائج.

### كيف نتحقق من الإنجاز

- كل بند تحقق له دليل عملي لا وصف شفهي.
- لا خطأ حاجب مفتوح قبل يوم المناقشة.
- الخطة البديلة مُختبرة وتعمل خلال أقل من دقيقة.

### أخطاء شائعة

- القفز إلى تخمين سبب واحد بدل اتباع تسلسل التشخيص.
- تعديل ترحيل قديم بدل إنشاء ترحيل جديد.
- ترك مفاتيح حقيقية في ملفات مشتركة.
- اعتبار أن العمل محلياً يعني أن النشر جاهز.

CH30 · REFERENCES

# 30 المراجع والمواد المرجعية

كل ما يحتاجه الفريق في مكان واحد

مرجع ختامي يجمع روابط التوثيق الرسمي، وقاموس مصطلحات، وقائمة متغيرات البيئة، وملخص الهيكل وعقد الـ API، وخريطة الاعتماديات. نستخدم جذور الروابط الرسمية فقط؛ وأي رابط لم يُتحقق منه يدوياً يجب فحصه.

## 30.1 روابط التوثيق الرسمي

روابط جذور رسمية عامة، لا صفحات داخلية قد تتغيّر. عمود "يحتاج تحققاً يدوياً" يعني أنه لم يُفتح ويُتأكد منه بعد.

**جدول 30-1: روابط التوثيق الرسمي**

| الأداة | الرابط الرسمي | يحتاج تحققاً يدوياً |
|---|---|---|
| Python | `docs.python.org` | نعم |
| FastAPI | `fastapi.tiangolo.com` | نعم |
| Pydantic | `docs.pydantic.dev` | نعم |
| SQLAlchemy | `docs.sqlalchemy.org` | نعم |
| Alembic | `alembic.sqlalchemy.org` | نعم |
| PostgreSQL | `postgresql.org/docs` | نعم |
| Docker | `docs.docker.com` | نعم |
| OpenCV | `docs.opencv.org` | نعم |
| PyTorch | `pytorch.org/docs` | نعم |
| Ultralytics YOLO | `docs.ultralytics.com` | نعم |
| NumPy | `numpy.org/doc` | نعم |
| Pandas | `pandas.pydata.org/docs` | نعم |
| Node.js | `nodejs.org/docs` | نعم |
| React | `react.dev` | نعم |
| TypeScript | `www.typescriptlang.org/docs` | نعم |
| Vite | `vitejs.dev` | نعم |
| Tailwind CSS | `tailwindcss.com/docs` | نعم |
| Recharts | `recharts.org` | نعم |
| Axios | `axios-http.com/docs` | نعم |
| GitHub Actions | `docs.github.com/actions` | نعم |

**تحقق يدوي إلزامي**

لم يُفتح كل رابط أعلاه ويُتأكد منه. على الفريق فتح كل رابط وتغيير "نعم" إلى "تم التحقق" مع التاريخ، واستبدال أي رابط لا يعمل بجذر رسمي آخر.

## 30.2 قاموس المصطلحات

**جدول 30-2: مصطلحات المشروع (إنجليزي + شرح عربي)**

| المصطلح | النطق/الاختصار | الشرح بالعربية |
|---|---|---|
| `Backend` | باك-إند | الجزء الذي يستقبل الطلبات ويدير البيانات والمنطق. |
| `Frontend` | فرونت-إند | الواجهة التي يراها المستخدم. |
| `API` | إيه-بي-آي | واجهة يتبادل بها الـ Frontend البيانات مع الـ Backend. |
| `Endpoint` | إند-بوينت | مسار في الـ API ينفّذ عملية واحدة. |
| `REST` | رست | نمط تصميم يعتمد على الموارد وأفعال HTTP. |
| `Schema` | سكيما | تعريف شكل البيانات وحقولها. |
| `Migration` | مايقريشن | تغيير منظّم في بنية قاعدة البيانات قابل للتطبيق والتراجع. |

| المصطلح | النطق/الاختصار | الشرح بالعربية |
|---|---|---|
| ORM | أو-آر-إم | وسيط يحوّل الجداول إلى كائنات والعكس. |
| Repository | ريبوزيتوري | طبقة الوصول إلى قاعدة البيانات. |
| Service | سيرفس | طبقة منطق العمل التي تستخدم الـ Repository. |
| Job | جوب | مهمة طويلة تُنفَّذ في الخلفية. |
| Worker | ووركر | عملية تسحب المهام من الطابور وتنفّذها. |
| Polling | بولينج | تكرار سؤال الـ Frontend عن حالة الـ job. |
| Inference | إنفيرنس | تشغيل النموذج المدرب على بيانات جديدة. |
| Checkpoint | تشيك-بوينت | ملف أوزان النموذج بعد التدريب. |
| Detection | ديتكشن | كشف عنصر: فئة وثقة وصندوق. |
| Bounding Box | باوندينج بوكس | مستطيل يحيط بالعنصر المكتشف. |
| Confidence | كونفيدنس | درجة ثقة النموذج في الكشف. |
| Tracking | تراكينج | متابعة العنصر عبر الإطارات برقم ثابت. |
| ByteTrack | بايت-تراك | خوارزمية تربط الكشوفات بين الإطارات. |
| Frame | فريم | صورة واحدة داخل الفيديو. |
| FPS | إف-بي-إس | عدد الإطارات في الثانية. |
| PPE | بي-بي-إي | معدات الوقاية الشخصية (خوذة، سترة). |
| Incident | إنسيدنت | واقعة سلامة مسجَّلة لها نوع وخطورة وحالة. |
| Evidence | إفيدنس | الدليل المحفوظ من إطار الواقعة. |
| Severity | سيفيريتي | درجة خطورة الواقعة. |
| Rule | رول | شرط منطقي يحوّل الكشوفات إلى واقعة. |
| Zone | زون | منطقة داخل الإطار لمنطق السلامة. |
| Analytics | أناليتكس | مؤشرات ورسوم مبنية على الوقائع. |
| CSV | سي-إس-في | صيغة ملف جدولي للتصدير. |
| JWT | جوت | رمز مصادقة موقّع عند تفعيل الدخول. |
| CORS | كورس | سياسة تسمح للمتصفح بالاتصال بالـ API. |
| Docker Compose | دوكر كومبوز | أداة تشغّل عدة حاويات بأمر واحد. |
| Volume | فوليوم | تخزين دائم يبقى بعد إعادة التشغيل. |
| Environment Variable | إنفايرونمنت فاريابل | قيمة إعداد تُقرأ من البيئة. |
| Placeholder | بليس-هولدر | قيمة مؤقتة بصيغة <...> حتى تُعرف الحقيقة. |

## 30.3 قائمة متغيرات البيئة

القائمة كاملة من SPEC.md. القيم هنا Placeholder، والحقيقية في .env غير المرفوع.

**جدول 30-3: متغيرات البيئة كاملة (الاسم، الوصف، قيمة Placeholder، الإلزامية)**

| الاسم | الوصف | قيمة Placeholder | إلزامي/اختياري |
|---|---|---|---|
| APP_NAME | اسم التطبيق | SafeVision AI | إلزامي |
| APP_ENV | بيئة التشغيل | development | إلزامي |
| APP_DEBUG | وضع التطوير | true | إلزامي |
| API_V1_PREFIX | بادئة الـ API | /api/v1 | إلزامي |
| AUTH_ENABLED | تفعيل المصادقة | false | اختياري |
| SECRET_KEY | مفتاح التوقيع | <SECRET_KEY> | إلزامي عند تفعيل المصادقة |
| JWT_ALGORITHM | خوارزمية الـ JWT | HS256 | اختياري |
| ACCESS_TOKEN_EXPIRE_MINUTES | صلاحية الرمز (دقائق) | 60 | اختياري |
| CORS_ORIGINS | الأصول المسموح بها | http://localhost:5173 | إلزامي |
| POSTGRES_USER | مستخدم القاعدة | <POSTGRES_USER> | إلزامي |
| POSTGRES_PASSWORD | كلمة مرور القاعدة | <POSTGRES_PASSWORD> | إلزامي |
| POSTGRES_DB | اسم القاعدة | safevision | إلزامي |

| الاسم | الوصف | قيمة Placeholder | إلزامي/اختياري |
|---|---|---|---|
| POSTGRES_HOST | مضيف القاعدة | localhost | إلزامي |
| POSTGRES_PORT | منفذ القاعدة | 5432 | إلزامي |
| DATABASE_URL | اتصال قاعدة البيانات | postgresql+psycopg://… | إلزامي |
| STORAGE_BACKEND | نوع التخزين | local | إلزامي |
| LOCAL_STORAGE_ROOT | جذر التخزين | ./storage | إلزامي |
| UPLOAD_DIR | مجلد الفيديوهات | storage/uploads | إلزامي |
| EVIDENCE_DIR | مجلد الدليل | storage/evidence | إلزامي |
| REPORT_DIR | مجلد التقارير | storage/reports | إلزامي |
| S3_ENDPOINT_URL | عنوان التخزين الكائني | <S3_ENDPOINT_URL> | اختياري |
| S3_BUCKET | اسم الحاوية | <S3_BUCKET> | اختياري |
| S3_ACCESS_KEY_ID | معرّف الوصول | <S3_ACCESS_KEY_ID> | اختياري |
| S3_SECRET_ACCESS_KEY | المفتاح السري | <S3_SECRET_ACCESS_KEY> | اختياري |
| S3_REGION | منطقة الخدمة | <S3_REGION> | اختياري |
| MAX_UPLOAD_MB | أقصى حجم للفيديو (MB) | 500 | إلزامي |
| ALLOWED_VIDEO_EXTENSIONS | الامتدادات المسموحة | mp4,mov,avi | إلزامي |
| UPLOAD_CHUNK_MB | حجم قطعة الرفع (MB) | 8 | اختياري |
| MODEL_DIR | مجلد الأوزان | ./models | إلزامي |
| PPE_MODEL_PATH | مسار أوزان معدات الوقاية | <PPE_MODEL_CHECKPOINT_PATH> | إلزامي |
| FIRE_SMOKE_MODEL_PATH | مسار أوزان الحريق والدخان | <FIRE_SMOKE_MODEL_CHECKPOINT_PATH> | إلزامي |
| PERSON_TRACKER_PATH | مسار أوزان التتبّع | <PERSON_TRACKER_CHECKPOINT_PATH> | إلزامي |
| MODEL_DEVICE | جهاز النموذج | cpu | إلزامي |
| DETECTION_CONF_THRESHOLD | عتبة ثقة الكشف | 0.25 | إلزامي |
| NMS_IOU_THRESHOLD | عتبة دمج الصناديق | 0.45 | إلزامي |
| INFERENCE_IMG_SIZE | حجم صورة الاستدلال | 640 | إلزامي |
| PERSIST_ALL_DETECTIONS | حفظ كل الكشوفات | false | اختياري |
| FRAME_SAMPLE_EVERY_N | تحليل إطار كل N إطار | 5 | إلزامي |
| MAX_FRAMES_PER_JOB | أقصى إطارات للمهمة | 20000 | اختياري |
| INCIDENT_CONFIRM_FRAMES | إطارات التأكيد | 3 | إلزامي |
| INCIDENT_COOLDOWN_SECONDS | منع تكرار الواقعة | 10 | إلزامي |
| PPE_CONF_THRESHOLD | عتبة ثقة معدات الوقاية | 0.35 | إلزامي |
| FIRE_CONF_THRESHOLD | عتبة ثقة الحريق | 0.40 | إلزامي |
| SMOKE_CONF_THRESHOLD | عتبة ثقة الدخان | 0.35 | إلزامي |
| PROXIMITY_MIN_OVERLAP | أدنى تداخل للتقارب | 0.15 | إلزامي |
| DANGER_ZONE_CONFIG | مناطق الخطر | <DANGER_ZONE_CONFIG> | إلزامي |
| WORKER_POLL_INTERVAL_SECONDS | فترة سؤال الطابور | 3 | إلزامي |
| WORKER_ID | معرّف الـ worker | worker-1 | اختياري |
| WORKER_CONCURRENCY | مهام الـ worker المتزامنة | 1 | إلزامي |
| JOB_MAX_RETRIES | أقصى محاولات الإعادة | 2 | اختياري |
| VITE_API_BASE_URL | عنوان الـ API للواجهة | http://localhost:8000/api/v1 | إلزامي |
| VITE_ENABLE_AUTH | تفعيل المصادقة بالواجهة | false | اختياري |

القيم الرقمية أمثلة بداية تُعاير بعد التجربة؛ والـ Placeholders مثل <SECRET_KEY> تُملأ بقيم حقيقية لا تُرفع إلى Git.

## 30.4 ملخص هيكل المستودع

### جدول 30-4: أهم مجلدات المستودع ووظائفها

| المسار | الوظيفة | داخل Git؟ |
|---|---|---|
| backend/ | تطبيق FastAPI (الحزمة app، الترحيلات، الاختبارات) | نعم |

| المسار | الوظيفة | داخل Git؟ |
|---|---|---|
| `frontend/` | تطبيق React + TypeScript + Vite | نعم |
| `models/` | أوزان النماذج | لا |
| `data/` | بيانات خام ومعالجة | جزئياً |
| `storage/` | الفيديوهات والدليل والتقارير | لا |
| `docs/` | التوثيق والقرارات | نعم |
| `scripts/` | سكربتات التشغيل | نعم |
| `docker-compose.yml` | تعريف الخدمات الموحّد | نعم |
| `.env.example` | قالب المتغيرات بقيم Placeholder | نعم |

## 30.5 ملخص عقد الـ API

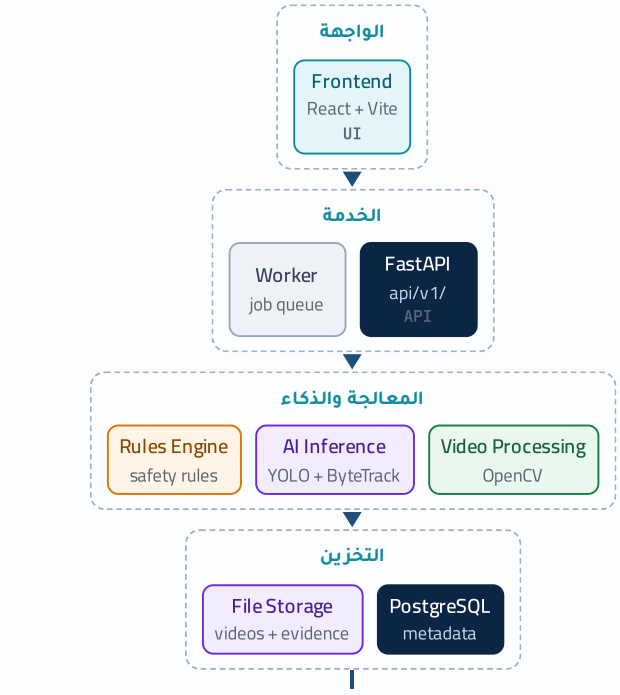
البادئة الموحّدة `/api/v1`. المصادقة تُفرض فقط عند `AUTH_ENABLED=true`؛ وفي وضع العرض يكون مستخدم افتراضي `safety_officer` مسجَّلاً.

### جدول 30-5: مسارات الـ API الأساسية

| Method | Endpoint | الغرض | Auth |
|---|---|---|---|
| `GET` | `/api/v1/health` | فحص صحة الخدمة | لا |
| `POST` | `/api/v1/videos` | رفع فيديو | نعم* |
| `GET` | `/api/v1/videos` | قائمة الفيديوهات | نعم* |
| `GET` | `/api/v1/videos/{video_id}` | فيديو مع آخر مهمة | نعم* |
| `DELETE` | `/api/v1/videos/{video_id}` | حذف الفيديو وملفاته | نعم* |
| `POST` | `/api/v1/videos/{video_id}/process` | إنشاء مهمة معالجة | نعم* |
| `GET` | `/api/v1/jobs/{job_id}` | حالة المهمة وتقدّمها | نعم* |
| `GET` | `/api/v1/jobs` | قائمة المهام | نعم* |
| `GET` | `/api/v1/incidents` | قائمة الوقائع | نعم* |
| `GET` | `/api/v1/incidents/{incident_id}` | تفاصيل واقعة | نعم* |
| `PATCH` | `/api/v1/incidents/{incident_id}` | مراجعة أو تغيير حالة | نعم* |
| `GET` | `/api/v1/incidents/{incident_id}/evidence` | قائمة أدلة الواقعة | نعم* |
| `GET` | `/api/v1/evidence/{evidence_id}/file` | بثّ الدليل | نعم* |
| `GET` | `/api/v1/analytics/overview` | مؤشرات لوحة التحكم | نعم* |
| `GET` | `/api/v1/analytics/timeseries` | الوقائع لكل يوم | نعم* |
| `GET` | `/api/v1/reports/daily` | ملخص يومي | نعم* |
| `GET` | `/api/v1/reports/export` | تصدير CSV | نعم* |
| `POST` | `/api/v1/auth/login` | الحصول على JWT | لا |
| `GET` | `/api/v1/auth/me` | المستخدم الحالي | نعم |

* تُفرض عند `AUTH_ENABLED=true`.

## 30.6 خريطة الاعتماديات النهائية بين المكوّنات




□□□□□□□□ → □□□□□□□□□ → □□□□□□□ → □□□□□□□□ :□□□□ □□□□□ □□ □□□□□□□□□

**شكل 30-1:** خريطة الاعتماديات النهائية بين مكوّنات النظام

### خلاصة الفصل 30

#### ما تعلّمناه

جمعنا كل المراجع في مكان واحد: روابط التوثيق الرسمي مع عمود التحقق اليدوي، وقاموس مصطلحات، وقائمة متغيرات البيئة كاملة، وملخص الهيكل وعقد الـ API، وخريطة الاعتماديات النهائية.

#### ما يجب تنفيذه

- فتح كل رابط رسمي وتغيير "يحتاج تحققاً يدوياً" إلى "تم التحقق" مع التاريخ.
- تعبئة قيم الـ Placeholders في .env غير المرفوع.
- تحديث هذا الفصل عند أي تغيير في عقد الـ API أو المتغيرات.

#### المخرجات المتوقعة

- مرجع مركزي يستخدمه الفريق واللجنة.
- قائمة متغيرات بيئة مطابقة تماماً لـ SPEC.md.
- خريطة اعتماديات مطابقة للمعمارية في الفصل 2.

#### كيف نتحقق من الإنجاز

- كل اسم متغير أو مسار في هذا الفصل مطابق حرفياً لـ SPEC.md.
- لا رابط مكتوب بصيغة غير رسمية أو غير مؤكد.
- كل Placeholder مذكور هنا معرَّف في مكان استخدامه.

#### أخطاء شائعة

- نسخ روابط لصفحات داخلية قد تتغيّر بدل جذور التوثيق الرسمية.
- إضافة متغير بيئة غير موجود في SPEC.md.
- وضع قيم حقيقية بدل الـ Placeholders داخل الدليل.
- الاعتماد على رابط دون فتحه والتأكد منه.